la idea de un criterio de decisión racional en todos los ámbitos de la actividad humana. En la siguiente sección se explica una teoría de la decisión más formalmente.

Matemáticas (aproximadamente desde el año 800 al presente)

- ¿Qué reglas formales son las adecuadas para obtener conclusiones válidas?
- ¿Qué se puede computar?
- ¿Cómo razonamos con información incierta?

Los filósofos delimitaron las ideas más importantes de la IA, pero para pasar de ahí a una ciencia formal es necesario contar con una formulación matemática en tres áreas fundamentales: lógica, computación y probabilidad.

El concepto de lógica formal se remonta a los filósofos de la antigua Grecia (*véase* el Capítulo 7), pero su desarrollo matemático comenzó realmente con el trabajo de George Boole (1815-1864) que definió la lógica proposicional o Booleana (Boole, 1847). En 1879, Gottlob Frege (1848-1925) extendió la lógica de Boole para incluir objetos y relaciones, y creó la lógica de primer orden que se utiliza hoy como el sistema más básico de representación de conocimiento[5]. Alfred Tarski (1902-1983) introdujo una teoría de referencia que enseña cómo relacionar objetos de una lógica con objetos del mundo real. El paso siguiente consistió en definir los límites de lo que se podía hacer con la lógica y la informática.

ALGORITMO

Se piensa que el primer **algoritmo** no trivial es el algoritmo Euclídeo para el cálculo del máximo común divisor. El considerar los algoritmos como objetos en sí mismos se remonta a la época de al-Khowarazmi, un matemático persa del siglo IX, con cuyos escritos también se introdujeron los números arábigos y el álgebra en Europa. Boole, entre otros, presentó algoritmos para llevar a cabo deducciones lógicas y hacia el final del siglo XIX se llevaron a cabo numerosos esfuerzos para formalizar el razonamiento matemático general con la lógica deductiva. En 1900, David Hilbert (1862-1943) presentó una lista de 23 problemas que acertadamente predijo ocuparían a los matemáticos durante todo ese siglo. En el último de ellos se preguntaba si existe un algoritmo que permita determinar la validez de cualquier proposición lógica en la que aparezcan números naturales (el famoso *Entscheidungsproblem*, o problema de decisión). Básicamente, lo que Hilbert se preguntaba es si hay límites fundamentales en la capacidad de los procedimientos efectivos de demostración. En 1930, Kurt Gödel (1906-1978) demostró que existe un procedimiento eficiente para demostrar cualquier aseveración verdadera en la lógica de primer orden de Frege y Russell, sin embargo con la lógica de primer orden no era posible capturar el principio de inducción matemática necesario para la caracterización de los números naturales. En 1931, demostró que, en efecto, existen límites reales. Mediante su **teorema de incompletitud** demostró que en cualquier lenguaje que tuviera la capacidad suficiente para expresar las propiedades de los números naturales, existen aseveraciones verdaderas no decidible en el sentido de que no es posible decidir su validez mediante ningún algoritmo.

TEOREMA DE INCOMPLETITUD

[5] La notación para la lógica de primer orden propuesta por Frege no se ha aceptado universalmente, por razones que son aparentemente obvias cuando se observa el ejemplo que aparece en la cubierta de este libro.

El resultado fundamental anterior se puede interpretar también como la indicación de que existen algunas funciones de los números enteros que no se pueden representar mediante un algoritmo, es decir no se pueden calcular. Lo anterior llevó a Alan Turing (1912-1954) a tratar de caracterizar exactamente aquellas funciones que sí *eran* susceptibles de ser caracterizadas. La noción anterior es de hecho problemática hasta cierto punto, porque no es posible dar una definición formal a la noción de cálculo o procedimiento efectivo. No obstante, la tesis de Church-Turing, que afirma que la máquina de Turing (Turing, 1936) es capaz de calcular cualquier función computable, goza de aceptación generalizada ya que proporciona una definición suficiente. Turing también demostró que existen algunas funciones que no se pueden calcular mediante la máquina de Turing. Por ejemplo, ninguna máquina puede decidir *en general* si un programa dado producirá una respuesta a partir de unas entradas, o si seguirá calculando indefinidamente.

INTRATABILIDAD

Si bien ser no decidible ni computable son importantes para comprender el proceso del cálculo, la noción de **intratabilidad** tuvo repercusiones más importantes. En términos generales se dice que un problema es intratable si el tiempo necesario para la resolución de casos particulares de dicho problema crece exponencialmente con el tamaño de dichos casos. La diferencia entre crecimiento polinomial y exponencial de la complejidad se destacó por primera vez a mediados de los años 60 (Cobham, 1964; Edmonds, 1965). Es importante porque un crecimiento exponencial implica la imposibilidad de resolver casos moderadamente grandes en un tiempo razonable. Por tanto, se debe optar por dividir el problema de la generación de una conducta inteligente en subproblemas que sean tratables en vez de manejar problemas intratables.

NP-COMPLETITUD

¿Cómo se puede reconocer un problema intratable? La teoría de la **NP-completitud**, propuesta por primera vez por Steven Cook (1971) y Richard Karp (1972) propone un método. Cook y Karp demostraron la existencia de grandes clases de problemas de razonamiento y búsqueda combinatoria canónica que son NP completos. Toda clase de problema a la que la clase de problemas NP completos se pueda reducir será seguramente intratable (aunque no se ha demostrado que los problemas NP completos son necesariamente intratables, la mayor parte de los teóricos así lo creen). Estos resultados contrastan con el optimismo con el que la prensa popular recibió a los primeros computadores, «Supercerebros Electrónicos» que eran «¡Más rápidos que Einstein!». A pesar del rápido incremento en la velocidad de los computadores, los sistemas inteligentes se caracterizarán por el uso cuidadoso que hacen de los recursos. De manera sucinta, ¡el mundo es un ejemplo de problema *extremadamente* grande! Recientemente la IA ha ayudado a explicar por qué algunos ejemplos de problemas NP completos son difíciles de resolver y otros son fáciles (Cheeseman *et al*., 1991).

PROBABILIDAD

Además de la lógica y el cálculo, la tercera gran contribución de las matemáticas a la IA es la teoría de la **probabilidad**. El italiano Gerolamo Cardano (1501-1576) fue el primero en proponer la idea de probabilidad, presentándola en términos de los resultados de juegos de apuesta. La probabilidad se convirtió pronto en parte imprescindible de las ciencias cuantitativas, ayudando en el tratamiento de mediciones con incertidumbre y de teorías incompletas. Pierre Fermat (1601-1665), Blaise Pascal (1623-1662), James Bernoulli (1654-1705), Pierre Laplace (1749-1827), entre otros, hicieron avanzar esta teoría e introdujeron nuevos métodos estadísticos. Thomas Bayes (1702-1761) propuso una

regla para la actualización de probabilidades subjetivas a la luz de nuevas evidencias. La regla de Bayes y el área resultante llamado análisis Bayesiano conforman la base de las propuestas más modernas que abordan el razonamiento incierto en sistemas de IA.

Economía (desde el año 1776 hasta el presente)

- ¿Cómo se debe llevar a cabo el proceso de toma de decisiones para maximizar el rendimiento?
- ¿Cómo se deben llevar a cabo acciones cuando otros no colaboren?
- ¿Cómo se deben llevar a cabo acciones cuando los resultados se obtienen en un futuro lejano?

La ciencia de la economía comenzó en 1776, cuando el filósofo escocés Adam Smith (1723-1790) publicó *An Inquiri into the Nature and Causes of the Wealth of Nations*. Aunque los antiguos griegos, entre otros, habían hecho contribuciones al pensamiento económico, Smith fue el primero en tratarlo como una ciencia, utilizando la idea de que las economías pueden concebirse como un conjunto de agentes individuales que intentan maximizar su propio estado de bienestar económico. La mayor parte de la gente cree que la economía sólo se trata de dinero, pero los economistas dicen que ellos realmente estudian cómo la gente toma decisiones que les llevan a obtener los beneficios esperados. Léon Walras (1834-1910) formalizó el tratamiento matemático del «beneficio deseado» o **utilidad,** y fue posteriormente mejorado por Frank Ramsey (1931) y después por John von Neumann y Oskar Morgenstern en su libro *The Theory of Games and Economic Behavior* (1944).

TEORÍA DE LA DECISIÓN

La **teoría de la decisión**, que combina la teoría de la probabilidad con la teoría de la utilidad, proporciona un marco completo y formal para la toma de decisiones (económicas o de otra índole) realizadas bajo incertidumbre, esto es, en casos en los que las descripciones probabilísticas capturan adecuadamente la forma en la que se toman las decisiones en el entorno; lo cual es adecuado para «grandes» economías en las que cada agente no necesita prestar atención a las acciones que lleven a cabo el resto de los agentes individualmente. Cuando se trata de «pequeñas» economías, la situación se asemeja más a la de un **juego**: las acciones de un jugador pueden afectar significativamente a la utilidad de otro (tanto positiva como negativamente). Los desarrollos de von Neumann y Morgenstern a partir de la **teoría de juegos** (*véase* también Luce y Raiffa, 1957) mostraban el hecho sorprendente de que, en algunos juegos, un agente racional debía actuar de forma aleatoria o, al menos, aleatoria en apariencia con respecto a sus contrincantes.

TEORÍA DE JUEGOS

La gran mayoría de los economistas no se preocuparon de la tercera cuestión mencionada anteriormente, es decir, cómo tomar decisiones racionales cuando los resultados de las acciones no son inmediatos y por el contrario se obtienen los resultados de las acciones de forma *secuencial*. El campo de la **investigación operativa** persigue este objetivo; dicho campo emergió en la Segunda Guerra Mundial con los esfuerzos llevados a cabo en el Reino Unido en la optimización de instalaciones de radar, y posteriormente en aplicaciones civiles relacionadas con la toma de decisiones de dirección complejas. El trabajo de Richard Bellman (1957) formaliza una clase de problemas de decisión secuencial llamados **procesos de decisión de Markov**, que se estudiarán en los Capítulos 17 y 21.

INVESTIGACIÓN OPERATIVA

El trabajo en la economía y la investigación operativa ha contribuido en gran medida a la noción de agente racional que aquí se presenta, aunque durante muchos años la investigación en el campo de la IA se ha desarrollado por sendas separadas. Una razón fue la **complejidad** aparente que trae consigo el tomar decisiones racionales. Herbert Simon (1916-2001), uno de los primeros en investigar en el campo de la IA, ganó el premio Nobel en Economía en 1978 por su temprano trabajo, en el que mostró que los modelos basados en **satisfacción** (que toman decisiones que son «suficientemente buenas», en vez de realizar cálculos laboriosos para alcanzar decisiones óptimas) proporcionaban una descripción mejor del comportamiento humano real (Simon, 1947). En los años 90, hubo un resurgimiento del interés en las técnicas de decisión teórica para sistemas basados en agentes (Wellman, 1995).

SATISFACCIÓN

Neurociencia (desde el año 1861 hasta el presente)

- ¿Cómo procesa información el cerebro?

NEUROCIENCIA

La **Neurociencia** es el estudio del sistema neurológico, y en especial del cerebro. La forma exacta en la que en un cerebro se genera el pensamiento es uno de los grandes misterios de la ciencia. Se ha observado durante miles de años que el cerebro está de alguna manera involucrado en los procesos de pensamiento, ya que fuertes golpes en la cabeza pueden ocasionar minusvalía mental. También es ampliamente conocido que los cerebros humanos son de alguna manera diferentes; aproximadamente en el 335 a.C. Aristóteles escribió, «de entre todos los animales el hombre tiene el cerebro más grande en proporción a su tamaño»[6]. Aunque, no fue hasta mediados del siglo XVIII cuando se aceptó mayoritariamente que el cerebro es la base de la conciencia. Hasta este momento, se pensaba que estaba localizado en el corazón, el bazo y la glándula pineal.

El estudio de Paul Broca (1824-1880) sobre la afasia (dificultad para hablar) en pacientes con el cerebro dañado, en 1861, le dio fuerza a este campo y convenció a la sociedad médica de la existencia de áreas localizadas en el cerebro responsables de funciones cognitivas específicas. En particular, mostró que la producción del habla se localizaba en una parte del hemisferio izquierdo; hoy en día conocida como el área de Broca[7]. En esta época ya se sabía que el cerebro estaba formado por células nerviosas o **neuronas**, pero no fue hasta 1873 cuando Camillo Golgi (1843-1926) desarrolló una técnica de coloración que permitió la observación de neuronas individuales en el cerebro (*véase* la Figura 1.2). Santiago Ramón y Cajal (1852-1934) utilizó esta técnica es sus estudios pioneros sobre la estructura neuronal del cerebro[8].

NEURONAS

En la actualidad se dispone de información sobre la relación existente entre las áreas del cerebro y las partes del cuerpo humano que controlan o de las que reciben impulsos

[6] Desde entonces, se ha descubierto que algunas especies de delfines y ballenas tienen cerebros relativamente grandes. Ahora se piensa que el gran tamaño de los cerebros humanos se debe en parte a la mejora reciente en su sistema de refrigeración.

[7] Muchos citan a Alexander Hood (1824) como una fuente posiblemente anterior.

[8] Golgi insistió en la creencia de que las funciones cerebrales se desarrollaron inicialmente en el medio continuo en el que las neuronas estaban inmersas, mientras que Cajal propuso la «doctrina neuronal». Ambos compartieron el premio Nobel en 1906 pronunciando un discurso de aceptación antagónico.

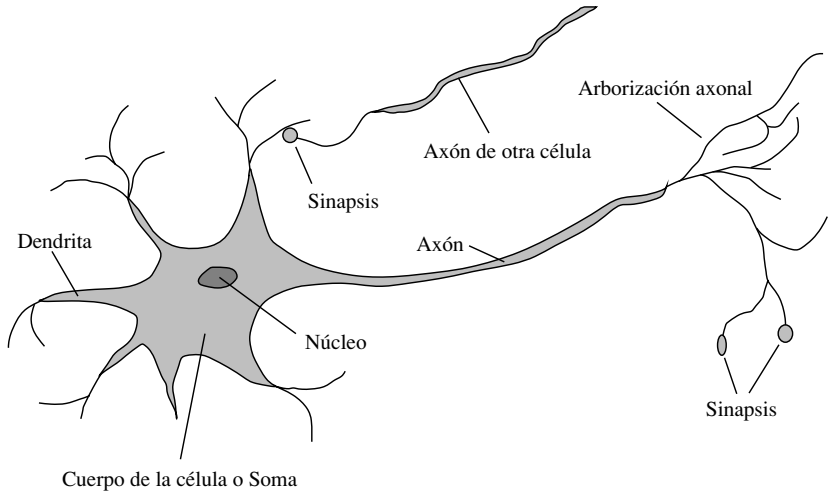


Figura 1.2 Partes de una célula nerviosa o neurona. Cada neurona contiene un cuerpo celular, o soma, que tiene un núcleo celular. Un número de fibras llamadas dendritas se ramifican a partir del cuerpo de la célula junto con una única fibra larga llamada axón. El axón se alarga considerablemente, mucho más que lo que se representa en esta imagen. Normalmente miden un centímetro (100 veces más que el diámetro del cuerpo de la célula), pero pueden alcanzar hasta un metro de longitud. Una neurona se conecta con entre 10 y 100.000 neuronas formando una maraña llamada sinapsis. Las señales se propagan de neurona a neurona mediante una reacción electroquímica complicada. Las señales controlan la actividad del cerebro a corto plazo, y permiten establecer cambios en la posición y conectividad de las neuronas a largo plazo. Se piensa que estos mecanismos forman la base del aprendizaje del cerebro. La mayoría del procesamiento de información se lleva a cabo en la corteza del cerebro, la capa más externa de éste. La unidad de organización básica es una columna de tejido de aproximadamente 0,5 mm de diámetro, con lo cual se amplía la profundidad total de la corteza cerebral, que en el caso de los humanos es de cuatro mm. Una columna contiene aproximadamente 20.000 neuronas.

sensoriales. Tales relaciones pueden cambiar de forma radical incluso en pocas semanas, y algunos animales parecen disponer de múltiples posibilidades. Más aún, no se tiene totalmente claro cómo algunas áreas se pueden encargar de ciertas funciones que eran responsabilidad de áreas dañadas. No hay prácticamente ninguna teoría que explique cómo se almacenan recuerdos individuales.

Los estudios sobre la actividad de los cerebros intactos comenzó en 1929 con el descubrimiento del electroencefalograma (EEG) desarrollado por Hans Berger. El reciente descubrimiento de las imágenes de resonancia magnética funcional (IRMF) (Ogawa *et al*., 1990) está proporcionando a los neurólogos imágenes detalladas de la actividad cerebral sin precedentes, permitiéndoles obtener medidas que se corresponden con procesos cognitivos en desarrollo de manera muy interesante. Este campo está evolucionando gracias a los avances en los estudios en celdas individuales y su actividad neuronal. A pesar de estos avances, nos queda un largo camino para llegar a comprender cómo funcionan todos estos procesos cognitivos.



La conclusión verdaderamente increíble es que *una colección de simples células puede llegar a generar razonamiento, acción, y conciencia* o, dicho en otras palabras, *los cerebros generan las inteligencias* (Searle, 1992). La única teoría alternativa es el misticismo: que nos dice que existe alguna esfera mística en la que las mentes operan fuera del control de la ciencia física.

Cerebros y computadores digitales realizan tareas bastante diferentes y tienen propiedades distintas. La Figura 1.3 muestra cómo hay 1.000 veces más neuronas en un cerebro humano medio que puertas lógicas en la UCP de un computador estándar. La ley de Moore[9] predice que el número de puertas lógicas de la UCP se igualará con el de neuronas del cerebro alrededor del año 2020. Por supuesto, poco se puede inferir de esta predicción; más aún, la diferencia en la capacidad de almacenamiento es insignificante comparada con las diferencias en la velocidad de intercambio y en paralelismo. Los circuitos de los computadores pueden ejecutar una instrucción en un nanosegundo, mientras que las neuronas son millones de veces más lentas. Las neuronas y las sinapsis del cerebro están activas simultáneamente, mientras que los computadores actuales tienen una o como mucho varias UCP. Por tanto, incluso sabiendo que un computador es un millón de veces más rápido en cuanto a su velocidad de intercambio, el cerebro acaba siendo 100.000 veces más rápido en lo que hace.



Psicología (desde el año 1879 hasta el presente)

- ¿Cómo piensan y actúan los humanos y los animales?

La psicología científica se inició con los trabajos del físico alemán Hermann von Helmholtz (1821-1894), según se referencia habitualmente, y su discípulo Wilhelm Wundt (1832-1920). Helmholtz aplicó el método científico al estudio de la vista humana, y su obra *Handbook of Physiological Optics*, todavía en nuestros días, se considera como «el tratado actual más importante sobre la física y la fisiología de la vista humana» (Nalwa, 1993, p. 15). En 1879, Wundt abrió el primer laboratorio de psicología experimental en

	Computador	Cerebro Humano
Unidades computacionales	1 UCP, 10^8 puertas	10^{11} neuronas
Unidades de Almacenamiento	10^{10} bits RAM	10^{11} neuronas
	10^{11} bits disco	10^{14} sinapsis
Duración de un ciclo	10^{-9} sec	10^{-3} sec
Ancho de banda	10^{10} bits/sec	10^{14} bits/sec
Memoria actualización/sec	10^9	10^{14}

Figura 1.3 Comparación básica entre los recursos de cómputo generales de que disponen los computadores (*circa 2003*) y el cerebro. Las cifras correspondientes a los computadores se han incrementado en al menos un factor 10 desde la primera edición de este libro, y se espera que suceda la mismo en esta década. Las cifras correspondientes al cerebro no han cambiado en los últimos 10.000 años.

[9] La ley de Moore dice que el número de transistores por pulgada cuadrada se duplica cada año o año y medio. La capacidad del cerebro humano se dobla aproximadamente cada dos o cuatro millones de años.

la Universidad de Leipzig. Wundt puso mucho énfasis en la realización de experimentos controlados cuidadosamente en la que sus operarios realizaban tareas de percepción o asociación al tiempo que sometían a introspección sus procesos mentales. Los meticulosos controles evolucionaron durante un largo período de tiempo hasta convertir la psicología en una ciencia, pero la naturaleza subjetiva de los datos hizo poco probable que un investigador pudiera contradecir sus propias teorías. Biólogos, estudiando el comportamiento humano, por el contrario, carecían de datos introspectivos y desarrollaron una metodología objetiva, tal y como describe H. S. Jennings (1906) en su influyente trabajo *Behavior of the Lower Organisms*. El movimiento **conductista**, liderado por John Watson (1878-1958) aplicó este punto de vista a los humanos, rechazando *cualquier* teoría en la que intervinieran procesos mentales, argumentando que la introspección no aportaba una evidencia fiable. Los conductistas insistieron en el estudio exclusivo de mediciones objetivas de percepciones (o *estímulos*) sobre animales y de las acciones resultantes (o *respuestas*). Construcciones mentales como conocimientos, creencias, objetivos y pasos en un razonamiento quedaron descartadas por ser consideradas «psicología popular» no científica. El conductismo hizo muchos descubrimientos utilizando ratas y palomas, pero tuvo menos éxito en la comprensión de los seres humanos. Aún así, su influencia en la psicología fue notable (especialmente en Estados Unidos) desde aproximadamente 1920 hasta 1960.

CONDUCTISMO

La conceptualización del cerebro como un dispositivo de procesamiento de información, característica principal de la **psicología cognitiva**, se remonta por lo menos a las obras de William James[10] (1842-1910). Helmholtz también pone énfasis en que la percepción entraña cierto tipo de inferencia lógica inconsciente. Este punto de vista cognitivo se vio eclipsado por el conductismo en Estados Unidos, pero en la Unidad de Psicología Aplicada de Cambridge, dirigida por Frederic Bartlett (1886-1969), los modelos cognitivos emergieron con fuerza. La obra *The Nature of Explanation*, de Kenneth Craik (1943), discípulo y sucesor de Bartlett, reestablece enérgicamente la legitimidad de términos «mentales» como creencias y objetivos, argumentando que son tan científicos como lo pueden ser la presión y la temperatura cuando se habla acerca de los gases, a pesar de que éstos estén formados por moléculas que no tienen ni presión ni temperatura. Craik establece tres elementos clave que hay que tener en cuenta para diseñar un agente basado en conocimiento: (1) el estímulo deberá ser traducido a una representación interna, (2) esta representación se debe manipular mediante procesos cognitivos para así generar nuevas representaciones internas, y (3) éstas, a su vez, se traducirán de nuevo en acciones. Dejó muy claro por qué consideraba que estos eran los requisitos idóneos para diseñar un agente:

PSICOLOGÍA COGNITIVA

> Si el organismo tiene en su cabeza «un modelo a pequeña escala» de la realidad externa y de todas sus posibles acciones, será capaz de probar diversas opciones, decidir cuál es la mejor, planificar su reacción ante posibles situaciones futuras antes de que éstas surjan, emplear lo aprendido de experiencias pasadas en situaciones presentes y futuras, y en todo momento, reaccionar ante los imprevistos que acontezcan de manera satisfactoria, segura y más competente (Craik, 1943).

[10] William James era hermano del novelista Henry James. Se comenta que Henry escribió novelas narrativas como si se tratara de psicología y William escribió sobre psicología como si se tratara de novelas narrativas.

Después de la muerte de Craik en un accidente de bicicleta en 1945, Donald Boadbent continuó su trabajo, y su libro *Perception and Communication* (1958) incluyó algunos de los primeros modelos de procesamiento de información del fenómeno psicológico. Mientras tanto, en Estados Unidos el desarrollo del modelo computacional llevo a la creación del campo de la **ciencia cognitiva**. Se puede decir que este campo comenzó en un simposio celebrado en el MIT, en septiembre de 1956 (como se verá a continuación este evento tuvo lugar sólo dos meses después de la conferencia en la que «nació» la IA). En este simposio, George Miller presentó *The Magic Number Seven*, Noam Chomsky presentó *Three Models of Language*, y Allen Newell y Herbert Simon presentaron *The Logic Theory Machine*. Estos tres artículos influyentes mostraron cómo se podían utilizar los modelos informáticos para modelar la psicología de la memoria, el lenguaje y el pensamiento lógico, respectivamente. Los psicólogos comparten en la actualidad el punto de vista común de que «la teoría cognitiva debe ser como un programa de computador» (Anderson, 1980), o dicho de otra forma, debe describir un mecanismo de procesamiento de información detallado, lo cual lleva consigo la implementación de algunas funciones cognitivas.

CIENCIA COGNITIVA

Ingeniería computacional (desde el año 1940 hasta el presente)

- ¿Cómo se puede construir un computador eficiente?

Para que la inteligencia artificial pueda llegar a ser una realidad se necesitan dos cosas: inteligencia y un artefacto. El computador ha sido el artefacto elegido. El computador electrónico digital moderno se inventó de manera independiente y casi simultánea por científicos en tres países involucrados en la Segunda Guerra Mundial. El equipo de Alan Turing construyó, en 1940, el primer computador *operacional* de carácter electromecánico, llamado Heath Robinson[11], con un único propósito: descifrar mensajes alemanes. En 1943 el mismo grupo desarrolló el Colossus, una máquina potente de propósito general basada en válvulas de vacío[12]. El primer computador operacional *programable* fue el Z-3, inventado por Konrad Zuse en Alemania, en 1941. Zuse también inventó los números de coma flotante y el primer lenguaje de programación de alto nivel, Plankalkül. El primer computador *electrónico*, el ABC, fue creado por John Atanasoff junto a su discípulo Clifford Berry entre 1940 y 1942 en la Universidad Estatal de Iowa. Las investigaciones de Atanasoff recibieron poco apoyo y reconocimiento; el ENIAC, desarrollado en el marco de un proyecto militar secreto, en la Universidad de Pensilvania, por un equipo en el que trabajaban entre otros John Mauchly y John Eckert, puede considerarse como el precursor de los computadores modernos.

Desde mediados del siglo pasado, cada generación de dispositivos *hardware* ha conllevado un aumento en la velocidad de proceso y en la capacidad de almacenamiento,

[11] Heath Robinson fue un caricaturista famoso por sus dibujos, que representaban artefactos de uso diario, caprichosos y absurdamente complicados, por ejemplo, uno para untar mantequilla en el pan tostado.

[12] En la postguerra, Turing quiso utilizar estos computadores para investigar en el campo de la IA, por ejemplo, desarrollando uno de los primeros programas para jugar a la ajedrez (Turing *et al*., 1953). El gobierno británico bloqueó sus esfuerzos.

así como una reducción de precios. La potencia de los computadores se dobla cada 18 meses aproximadamente y seguirá a este ritmo durante una o dos décadas más. Después, se necesitará ingeniería molecular y otras tecnologías novedosas.

Por supuesto que antes de la aparición de los computadores ya había dispositivos de cálculo. Las primeras máquinas automáticas, que datan del siglo XVII, ya se mencionaron en la página seis. La primera máquina programable fue un telar, desarrollado en 1805 por Joseph Marie Jacquard (1752-1834) que utilizaba tarjetas perforadas para almacenar información sobre los patrones de los bordados. A mediados del siglo XIX, Charles Babbage (1792-1871) diseñó dos máquinas, que no llegó a construir. La «Máquina de Diferencias», que aparece en la portada de este manuscrito, se concibió con la intención de facilitar los cálculos de tablas matemáticas para proyectos científicos y de ingeniería. Finalmente se construyó y se presentó en 1991 en el Museo de la Ciencia de Londres (Swade, 1993). La «Máquina Analítica» de Babbage era mucho más ambiciosa: incluía memoria direccionable, programas almacenados y saltos condicionales; fue el primer artefacto dotado de los elementos necesarios para realizar una computación universal. Ada Lovelace, colega de Babbage e hija del poeta Lord Byron, fue seguramente la primera programadora (el lenguaje de programación Ada se llama así en honor a esta programadora). Ella escribió programas para la inacabada Máquina Analítica e incluso especuló acerca de la posibilidad de que la máquina jugara al ajedrez y compusiese música.

La IA también tiene una deuda con la parte *software* de la informática que ha proporcionado los sistemas operativos, los lenguajes de programación, y las herramientas necesarias para escribir programas modernos (y artículos sobre ellos). Sin embargo, en este área la deuda se ha saldado: la investigación en IA ha generado numerosas ideas novedosas de las que se ha beneficiado la informática en general, como por ejemplo el tiempo compartido, los intérpretes imperativos, los computadores personales con interfaces gráficas y ratones, entornos de desarrollo rápido, listas enlazadas, administración automática de memoria, y conceptos claves de la programación simbólica, funcional, dinámica y orientada a objetos.

Teoría de control y cibernética (desde el año 1948 hasta el presente)

- ¿Cómo pueden los artefactos operar bajo su propio control?

Ktesibios de Alejandría (250 a.C.) construyó la primera máquina auto controlada: un reloj de agua con un regulador que mantenía el flujo de agua circulando por él, con un ritmo constante y predecible. Esta invención cambió la definición de lo que un artefacto podía hacer. Anteriormente, solamente seres vivos podían modificar su comportamiento como respuesta a cambios en su entorno. Otros ejemplos de sistemas de control auto regulables y retroalimentados son el motor de vapor, creado por James Watt (1736-1819), y el termostato, inventado por Cornelis Drebbel (1572-1633), que también inventó el submarino. La teoría matemática de los sistemas con retroalimentación estables se desarrolló en el siglo XIX.

TEORÍA DE CONTROL

La figura central del desarrollo de lo que ahora se llama la **teoría de control** fue Norbert Wiener (1894-1964). Wiener fue un matemático brillante que trabajó en sistemas de control biológicos y mecánicos y en sus vínculos con la cognición. De la misma forma que Craik (quien también utilizó sistemas de control como modelos psicológicos), Wiener y sus colegas Arturo Rosenblueth y Julian Bigelow desafiaron la ortodoxia conductista (Rosenblueth *et al*., 1943). Ellos veían el comportamiento determinista como algo emergente de un mecanismo regulador que intenta minimizar el «error» (la diferencia entre el estado presente y el estado objetivo). A finales de los años 40, Wiener, junto a Warren McCulloch, Walter Pitts y John von Neumann, organizaron una serie de conferencias en las que se exploraban los nuevos modelos cognitivos matemáticos y computacionales, e influyeron en muchos otros investigadores en el campo de las ciencias del comportamiento. El libro de Wiener, *Cybernetics* (1948), fue un *bestseller* y desveló al público las posibilidades de las máquinas con inteligencia artificial.

CIBERNÉTICA

FUNCIÓN OBJETIVO

La teoría de control moderna, especialmente la rama conocida como control óptimo estocástico, tiene como objetivo el diseño de sistemas que maximizan una **función objetivo** en el tiempo. Lo cual se asemeja ligeramente a nuestra visión de lo que es la IA: diseño de sistemas que se comportan de forma óptima. ¿Por qué, entonces, IA y teoría de control son dos campos diferentes, especialmente teniendo en cuenta la cercana relación entre sus creadores? La respuesta está en el gran acoplamiento existente entre las técnicas matemáticas con las que estaban familiarizados los investigadores y entre los conjuntos de problemas que se abordaban desde cada uno de los puntos de vista. El cálculo y el álgebra matricial, herramientas de la teoría de control, se utilizaron en la definición de sistemas que se podían describir mediante conjuntos fijos de variables continuas; más aún, el análisis exacto es sólo posible en sistemas *lineales*. La IA se fundó en parte para escapar de las limitaciones matemáticas de la teoría de control en los años 50. Las herramientas de inferencia lógica y computación permitieron a los investigadores de IA afrontar problemas relacionados con el lenguaje, visión y planificación, que estaban completamente fuera del punto de mira de la teoría de control.

Lingüística (desde el año 1957 hasta el presente)

- ¿Cómo está relacionado el lenguaje con el pensamiento?

En 1957, B. F. Skinner publicó *Verbal Behavior*. La obra presentaba una visión extensa y detallada desde el enfoque conductista al aprendizaje del lenguaje, y estaba escrita por los expertos más destacados de este campo. Curiosamente, una revisión de este libro llegó a ser tan famosa como la obra misma, y provocó el casi total desinterés por el conductismo. El autor de la revisión fue Noam Chomsky, quien acababa de publicar un libro sobre su propia teoría, *Syntactic Structures*. Chomsky mostró cómo la teoría conductista no abordaba el tema de la creatividad en el lenguaje: no explicaba cómo es posible que un niño sea capaz de entender y construir oraciones que nunca antes ha escuchado. La teoría de Chomsky (basada en modelos sintácticos que se remontaban al lingüista indio Panini, aproximadamente 350 a.C.) sí podía explicar lo anterior y, a diferencia de teorías anteriores, poseía el formalismo suficiente como para permitir su programación.

LINGÜÍSTICA COMPUTACIONAL

La lingüística moderna y la IA «nacieron», al mismo tiempo y maduraron juntas, solapándose en un campo híbrido llamado **lingüística computacional** o **procesamiento del lenguaje natural**. El problema del entendimiento del lenguaje se mostró pronto mucho más complejo de lo que se había pensado en 1957. El entendimiento del lenguaje requiere la comprensión de la materia bajo estudio y de su contexto, y no solamente el entendimiento de la estructura de las sentencias. Lo cual puede parecer obvio, pero no lo fue para la mayoría de la comunidad investigadora hasta los años 60. Gran parte de los primeros trabajos de investigación en el área de la **representación del conocimiento** (el estudio de cómo representar el conocimiento de forma que el computador pueda razonar a partir de dicha representación) estaban vinculados al lenguaje y a la búsqueda de información en el campo del lenguaje, y su base eran las investigaciones realizadas durante décadas en el análisis filosófico del lenguaje.

1.3 Historia de la inteligencia artificial

Una vez revisado el material básico estamos ya en condiciones de cubrir el desarrollo de la IA propiamente dicha.

Génesis de la inteligencia artificial (1943–1955)

Warren McCulloch y Walter Pitts (1943) han sido reconocidos como los autores del primer trabajo de IA. Partieron de tres fuentes: conocimientos sobre la fisiología básica y funcionamiento de las neuronas en el cerebro, el análisis formal de la lógica proposicional de Russell y Whitehead y la teoría de la computación de Turing. Propusieron un modelo constituido por neuronas artificiales, en el que cada una de ellas se caracterizaba por estar «activada» o «desactivada»; la «activación» se daba como respuesta a la estimulación producida por una cantidad suficiente de neuronas vecinas. El estado de una neurona se veía como «equivalente, de hecho, a una proposición con unos estímulos adecuados». Mostraron, por ejemplo, que cualquier función de cómputo podría calcularse mediante alguna red de neuronas interconectadas, y que todos los conectores lógicos (*and*, *or*, *not*, etc.) se podrían implementar utilizando estructuras de red sencillas. McCulloch y Pitts también sugirieron que redes adecuadamente definidas podrían aprender. Donald Hebb (1949) propuso y demostró una sencilla regla de actualización para modificar las intensidades de las conexiones entre neuronas. Su regla, ahora llamada **de aprendizaje Hebbiano o de Hebb**, sigue vigente en la actualidad.

Dos estudiantes graduados en el Departamento de Matemáticas de Princeton, Marvin Minsky y Dean Edmonds, construyeron el primer computador a partir de una red neuronal en 1951. El SNARC, como se llamó, utilizaba 3.000 válvulas de vacío y un mecanismo de piloto automático obtenido de los desechos de un avión bombardero B-24 para simular una red con 40 neuronas. El comité encargado de evaluar el doctorado de Minsky veía con escepticismo el que este tipo de trabajo pudiera considerarse como matemático, pero se dice que von Newmann dijo, «Si no lo es actualmente, algún día lo será».

Minsky posteriormente probó teoremas influyentes que mostraron las limitaciones de la investigación con redes neuronales.

Hay un número de trabajos iniciales que se pueden caracterizar como de IA, pero fue Alan Turing quien articuló primero una visión de la IA en su artículo *Computing Machinery and Intelligence*, en 1950. Ahí, introdujo la prueba de Turing, el aprendizaje automático, los algoritmos genéricos y el aprendizaje por refuerzo.

Nacimiento de la inteligencia artificial (1956)

Princeton acogió a otras de la figuras señeras de la IA, John McCarthy. Posteriormente a su graduación, McCarthy se transladó al Dartmouth College, que se erigiría en el lugar del nacimiento oficial de este campo. McCarthy convenció a Minsky, Claude Shannon y Nathaniel Rochester para que le ayudaran a aumentar el interés de los investigadores americanos en la teoría de autómatas, las redes neuronales y el estudio de la inteligencia. Organizaron un taller con una duración de dos meses en Darmouth en el verano de 1956. Hubo diez asistentes en total, entre los que se incluían Trenchard More de Princeton, Arthur Samuel de IBM, y Ray Solomonoff y Oliver Selfridge del MIT.

Dos investigadores del Carnegie Tech[13], Allen Newell y Herbert Simon, acapararon la atención. Si bien los demás también tenían algunas ideas y, en algunos casos, programas para aplicaciones determinadas como el juego de damas, Newell y Simon contaban ya con un programa de razonamiento, el Teórico Lógico (TL), del que Simon afirmaba: «Hemos inventado un programa de computación capaz de pensar de manera no numérica, con lo que ha quedado resuelto el venerable problema de la dualidad mente-cuerpo»[14]. Poco después del término del taller, el programa ya era capaz de demostrar gran parte de los teoremas del Capítulo 2 de *Principia Matemática* de Russell y Whitehead. Se dice que Russell se manifestó complacido cuando Simon le mostró que la demostración de un teorema que el programa había generado era más corta que la que aparecía en *Principia*. Los editores de la revista *Journal of Symbolic Logic* resultaron menos impresionados y rechazaron un artículo cuyos autores eran Newell, Simon y el Teórico Lógico (TL).

El taller de Dartmouth no produjo ningún avance notable, pero puso en contacto a las figuras importantes de este campo. Durante los siguientes 20 años, el campo estuvo dominado por estos personajes, así como por sus estudiantes y colegas del MIT, CMU, Stanford e IBM. Quizá lo último que surgió del taller fue el consenso en adoptar el nuevo nombre propuesto por McCarthy para este campo: **Inteligencia Artificial**. Quizá «racionalidad computacional» hubiese sido más adecuado, pero «IA» se ha mantenido.

Revisando la propuesta del taller de Dartmouth (McCarthy *et al*., 1955), se puede apreciar por qué fue necesario para la IA convertirse en un campo separado. ¿Por qué

[13] Actualmente Universidad Carnegie Mellon (UCM).

[14] Newell y Simon también desarrollaron un lenguaje de procesamiento de listas, IPL, para poder escribir el TL. No disponían de un compilador y lo tradujeron a código máquina a mano. Para evitar errores, trabajaron en paralelo, diciendo en voz alta números binarios, conforme escribían cada instrucción para asegurarse de que ambos coincidían.

no todo el trabajo hecho en el campo de la IA se ha realizado bajo el nombre de teoría de control, o investigación operativa, o teoría de la decisión, que, después de todo, persiguen objetivos similares a los de la IA? O, ¿por qué no es la IA una rama de las matemáticas? La primera respuesta es que la IA desde el primer momento abarcó la idea de duplicar facultades humanas como la creatividad, la auto-mejora y el uso del lenguaje. Ninguno de los otros campos tenían en cuenta esos temas. La segunda respuesta está relacionada con la metodología. La IA es el único de estos campos que es claramente una rama de la informática (aunque la investigación operativa comparte el énfasis en la simulación por computador), además la IA es el único campo que persigue la construcción de máquinas que funcionen automáticamente en medios complejos y cambiantes.

Entusiasmo inicial, grandes esperanzas (1952–1969)

Los primeros años de la IA estuvieron llenos de éxitos (aunque con ciertas limitaciones). Teniendo en cuenta lo primitivo de los computadores y las herramientas de programación de aquella época, y el hecho de que sólo unos pocos años antes, a los computadores se les consideraba como artefactos que podían realizar trabajos aritméticos y nada más, resultó sorprendente que un computador hiciese algo remotamente inteligente. La comunidad científica, en su mayoría, prefirió creer que «una máquina nunca podría hacer *tareas*» (*véase* el Capítulo 26 donde aparece una extensa lista de *tareas* recopilada por Turing). Naturalmente, los investigadores de IA responderían demostrando la realización de una *tarea* tras otra. John McCarthy se refiere a esta época como la era de «¡Mira, mamá, ahora sin manos!».

Al temprano éxito de Newell y Simon siguió el del sistema de resolución general de problemas, o SRGP. A diferencia del Teórico Lógico, desde un principio este programa se diseñó para que imitara protocolos de resolución de problemas de los seres humanos. Dentro del limitado número de puzles que podía manejar, resultó que la secuencia en la que el programa consideraba que los subobjetivos y las posibles acciones eran semejantes a la manera en que los seres humanos abordaban los mismos problemas. Es decir, el SRGP posiblemente fue el primer programa que incorporó el enfoque de «pensar como un ser humano». El éxito del SRGP y de los programas que le siguieron, como los modelos de cognición, llevaron a Newell y Simon (1976) a formular la famosa hipótesis del **sistema de símbolos físicos**, que afirma que «un sistema de símbolos físicos tiene los medios suficientes y necesarios para generar una acción inteligente». Lo que ellos querían decir es que cualquier sistema (humano o máquina) que exhibiese inteligencia debería operar manipulando estructuras de datos compuestas por símbolos. Posteriormente se verá que esta hipótesis se ha modificado atendiendo a distintos puntos de vista.

SISTEMA DE SÍMBOLOS FÍSICOS

En IBM, Nathaniel Rochester y sus colegas desarrollaron algunos de los primeros programas de IA. Herbert Gelernter (1959) construyó el demostrador de teoremas de geometría (DTG), el cual era capaz de probar teoremas que muchos estudiantes de matemáticas podían encontrar muy complejos de resolver. A comienzos 1952, Arthur Samuel escribió una serie de programas para el juego de las damas que eventualmente aprendieron a jugar hasta alcanzar un nivel equivalente al de un *amateur*. De paso, echó por

tierra la idea de que los computadores sólo pueden hacer lo que se les dice: su programa pronto aprendió a jugar mejor que su creador. El programa se presentó en la televisión en febrero de 1956 y causó una gran impresión. Como Turing, Samuel tenía dificultades para obtener el tiempo de cómputo. Trabajaba por las noches y utilizaba máquinas que aún estaban en período de prueba en la planta de fabricación de IBM. El Capítulo 6 trata el tema de los juegos, y en el Capítulo 21 se describe con detalle las técnicas de aprendizaje utilizadas por Samuel.

John McCarthy se trasladó de Darmouth al MIT, donde realizó tres contribuciones cruciales en un año histórico: 1958. En el Laboratorio de IA del MIT Memo Número 1, McCarthy definió el lenguaje de alto nivel **Lisp**, que se convertiría en el lenguaje de programación dominante en la IA. Lisp es el segundo lenguaje de programación más antiguo que se utiliza en la actualidad, ya que apareció un año después de FORTRAN. Con Lisp, McCarthy tenía la herramienta que necesitaba, pero el acceso a los escasos y costosos recursos de cómputo aún era un problema serio. Para solucionarlo, él, junto a otros miembros del MIT, inventaron el tiempo compartido. También, en 1958, McCarthy publicó un artículo titulado *Programs with Common Sense*, en el que describía el Generador de Consejos, un programa hipotético que podría considerarse como el primer sistema de IA completo. Al igual que el Teórico Lógico y el Demostrador de Teoremas de Geometría, McCarthy diseñó su programa para buscar la solución a problemas utilizando el conocimiento. Pero, a diferencia de los otros, manejaba el conocimiento general del mundo. Por ejemplo, mostró cómo algunos axiomas sencillos permitían a un programa generar un plan para conducirnos hasta el aeropuerto y tomar un avión. El programa se diseñó para que aceptase nuevos axiomas durante el curso normal de operación, permitiéndole así ser competente en áreas nuevas, sin *necesidad de reprogramación*. El Generador de Consejos incorporaba así los principios centrales de la representación del conocimiento y el razonamiento: es útil contar con una representación formal y explícita del mundo y de la forma en que la acción de un agente afecta al mundo, así como, ser capaces de manipular estas representaciones con procesos deductivos. Es sorprendente constatar cómo mucho de lo propuesto en el artículo escrito en 1958 permanece vigente incluso en la actualidad.

1958 fue el año en el que Marvin Minsky se trasladó al MIT. Sin embargo, su colaboración inicial no duró demasiado. McCarthy se centró en la representación y el razonamiento con lógica formal, mientras que Minsky estaba más interesado en lograr que los programas funcionaran y eventualmente desarrolló un punto de vista anti-lógico. En 1963 McCarthy creó el Laboratorio de IA en Stanford. Su plan para construir la versión más reciente del Generador de Consejos con ayuda de la lógica sufrió un considerable impulso gracias al descubrimiento de J. A. Robinson del método de resolución (un algoritmo completo para la demostración de teoremas para la lógica de primer orden; *véase* el Capítulo 9). El trabajo realizado en Stanford hacía énfasis en los métodos de propósito general para el razonamiento lógico. Algunas aplicaciones de la lógica incluían los sistemas de planificación y respuesta a preguntas de Cordell Green (1969b), así como el proyecto de robótica de Shakey en el nuevo Instituto de Investigación de Stanford (Stanford Research Institute, SRI). Este último proyecto, comentado en detalle en el Capítulo 25, fue el primero que demostró la total integración del razonamiento lógico y la actividad física.

Minsky supervisó el trabajo de una serie de estudiantes que eligieron un número de problemas limitados cuya solución pareció requerir inteligencia. Estos dominios limi-

MICROMUNDOS

tados se conocen como **micromundos**. El programa SAINT de James Slagle (1963a) fue capaz de resolver problemas de integración de cálculo en forma cerrada, habituales en los primeros cursos de licenciatura. El programa ANALOGY de Tom Evans (1968) resolvía problemas de analogía geométrica que se aplicaban en las pruebas de medición de inteligencia, semejante al de la Figura 1.4. El programa STUDENT de Daniel Bobrow (1967) podía resolver problemas de álgebra del tipo:

> Si el número de clientes de Tom es dos veces el cuadrado del 20 por ciento de la cantidad de anuncios que realiza, y éstos ascienden a 45, ¿cuántos clientes tiene Tom?

El micromundo más famoso fue el mundo de los bloques, que consiste en un conjunto de bloques sólidos colocados sobre una mesa (más frecuentemente, en la simulación de ésta), como se muestra en la Figura 1.5. Una tarea típica de este mundo es la reordenación de los bloques de cierta manera, con la ayuda de la mano de un robot que es capaz de tomar un bloque cada vez. El mundo de los bloques fue el punto de partida para el proyecto de visión de David Huffman (1971), la visión y el trabajo de propagación con restricciones de David Waltz (1975), la teoría del aprendizaje de Patrick Winston (1970), del programa para la comprensión de lenguaje natural de Terry Winograd (1972) y del planificador de Scott Fahlman (1974).

El trabajo realizado por McCulloch y Pitts con redes neuronales hizo florecer esta área. El trabajo de Winograd y Cowan (1963) mostró cómo un gran número de elementos podría representar un concepto individual de forma colectiva, lo cual llevaba consigo un aumento proporcional en robustez y paralelismo. Los métodos de aprendizaje de Hebb se reforzaron con las aportaciones de Bernie Widrow (Widrow y Hoff, 1960; Widrow, 1962), quien llamó **adalines** a sus redes, y por Frank Rosenblatt (1962) con sus **perceptrones**. Rosenblatt demostró el famoso teorema del perceptrón, con lo que mostró que su algoritmo de aprendizaje podría ajustar las intensidades de las conexiones de un perceptrón para que se adaptaran a los datos de entrada, siempre y cuando existiera una correspondencia. Estos temas se explicarán en el Capítulo 20.

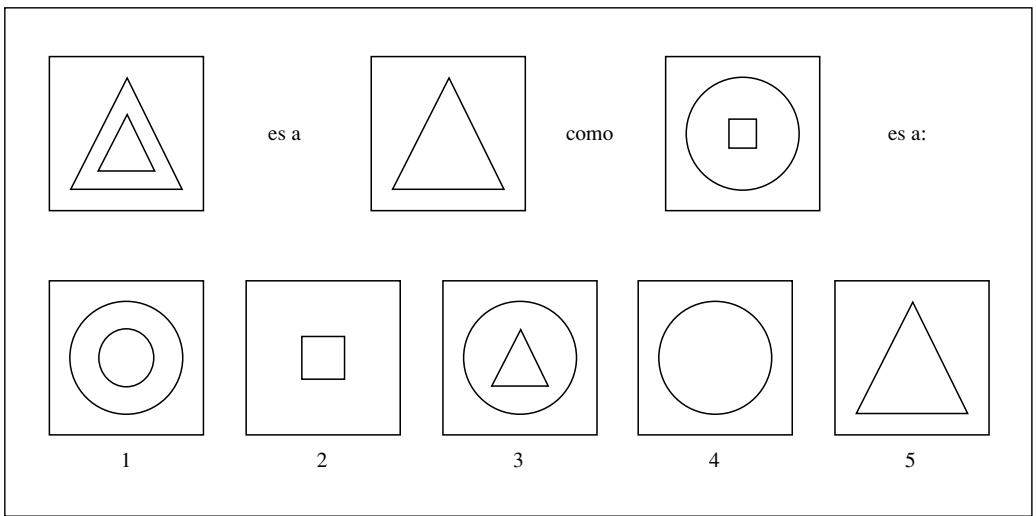


Figura 1.4 Ejemplo de un problema resuelto por el programa ANALOGY de Evans.

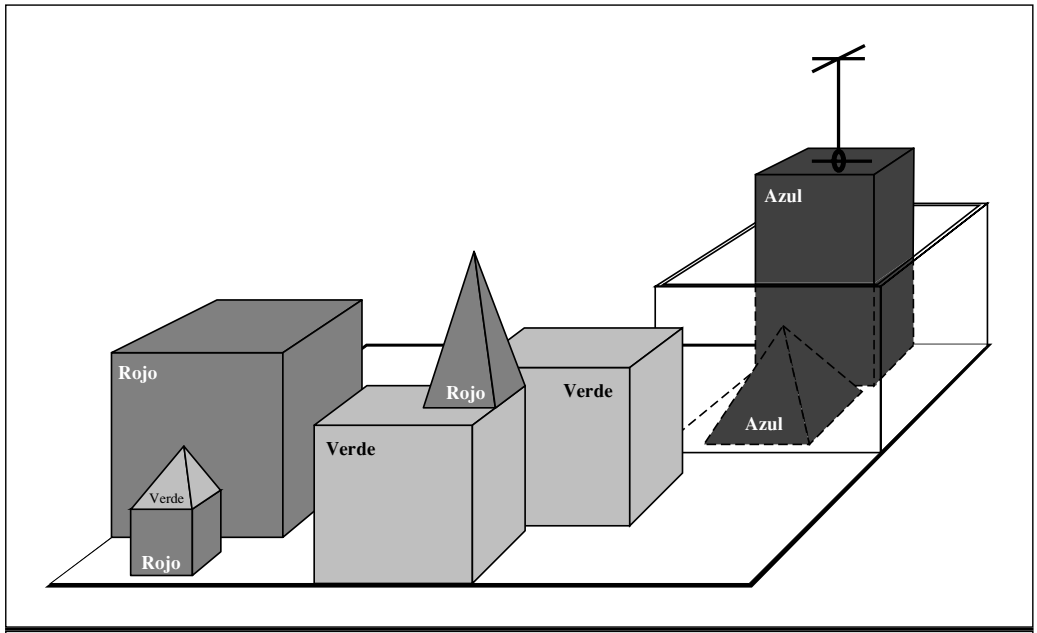


Figura 1.5 Esquema del mundo de los bloques. SHRDLU (Winograd, 1972) ha completado su tarea, «Busca un bloque que sea más alto que el que ahora tienes y ponlo en la caja».

Una dosis de realidad (1966–1973)

Desde el principio, los investigadores de IA hicieron públicas, sin timidez, predicciones sobre el éxito que les esperaba. Con frecuencia, se cita el siguiente comentario realizado por Herbert Simon en 1957:

> Sin afán de sorprenderlos y dejarlos atónitos, pero la forma más sencilla que tengo de resumirlo es diciéndoles que actualmente en el mundo existen máquinas capaces de pensar, aprender y crear. Además, su aptitud para hacer lo anterior aumentará rápidamente hasta que (en un futuro previsible) la magnitud de problemas que serán capaces de resolver irá a la par que la capacidad de la mente humana para hacer lo mismo.

Términos como «futuro previsible» pueden interpretarse de formas distintas, pero Simon también hizo predicciones más concretas: como que en diez años un computador llegaría a ser campeón de ajedrez, y que se podría demostrar un importante teorema matemático con una máquina. Estas predicciones se cumplirían (al menos en parte) dentro de los siguientes 40 años y no en diez. El exceso de confianza de Simon se debió a la prometedora actuación de los primeros sistemas de IA en problemas simples. En la mayor parte de los casos resultó que estos primeros sistemas fallaron estrepitosamente cuando se utilizaron en problemas más variados o de mayor dificultad.

El primer tipo de problemas surgió porque la mayoría de los primeros programas contaban con poco o ningún conocimiento de las materia objeto de estudio; obtenían resultados gracias a sencillas manipulaciones sintácticas. Una anécdota típica tuvo lugar cuando se comenzaba a trabajar en la traducción automática, actividad que recibía un

generoso patrocinio del Consejo Nacional para la Investigación de Estados Unidos en un intento de agilizar la traducción de artículos científicos rusos en vísperas del lanzamiento del Sputnik en 1957. Al principio se consideró que todo se reduciría a sencillas transformaciones sintácticas apoyadas en las gramáticas rusas e inglesa y al emplazamiento de palabras mediante un diccionario electrónico, lo que bastaría para obtener el significado exacto de las oraciones. La realidad es que para traducir es necesario contar con un conocimiento general sobre el tema, que permita resolver ambigüedades y así, precisar el contenido de una oración. La famosa retraducción del ruso al inglés de la frase «el espíritu es fuerte pero la carne es débil», cuyo resultado fue «el vodka es bueno pero la carne está podrida» es un buen ejemplo del tipo de dificultades que surgieron. En un informe presentado en 1966, el comité consultivo declaró que «no se ha logrado obtener ninguna traducción de textos científicos generales ni se prevé obtener ninguna en un futuro inmediato». Se canceló todo el patrocinio del gobierno estadounidense que se había asignado a los proyectos académicos sobre traducción. Hoy día, la traducción automática es una herramienta imperfecta pero de uso extendido en documentos técnicos, comerciales, gubernamentales y de Internet.

El segundo problema fue que muchos de los problemas que se estaban intentando resolver mediante la IA eran intratables. La mayoría de los primeros programas de IA resolvían problemas experimentando con diversos pasos hasta que se llegara a encontrar una solución. Esto funcionó en los primeros programas debido a que los micromundos con los que se trabajaba contenían muy pocos objetos, y por lo tanto muy pocas acciones posibles y secuencias de soluciones muy cortas. Antes de que se desarrollara la teoría de la complejidad computacional, se creía que para «aumentar» el tamaño de los programas de forma que estos pudiesen solucionar grandes problemas sería necesario incrementar la velocidad del *hardware* y aumentar las memorias. El optimismo que acompañó al logro de la demostración de problemas, por ejemplo, pronto se vio eclipsado cuando los investigadores fracasaron en la demostración de teoremas que implicaban más de unas pocas decenas de condiciones. *El hecho de que, en principio, un programa sea capaz de encontrar una solución no implica que tal programa encierre todos los mecanismos necesarios para encontrar la solución en la práctica.*

EVOLUCIÓN AUTOMÁTICA

La ilusoria noción de una ilimitada capacidad de cómputo no sólo existió en los programas para la resolución de problemas. Los primeros experimentos en el campo de la **evolución automática** (ahora llamados **algoritmos genéticos**) (Friedberg, 1958; Friedberg *et al*., 1959) estaban basados en la, sin duda correcta, premisa de que efectuando una adecuada serie de pequeñas mutaciones a un programa de código máquina se podría generar un programa con buen rendimiento aplicable en cualquier tarea sencilla. Después surgió la idea de probar con mutaciones aleatorias aplicando un proceso de selección con el fin de conservar aquellas mutaciones que hubiesen demostrado ser más útiles. No obstante, las miles de horas de CPU dedicadas, no dieron lugar a ningún avance tangible. Los algoritmos genéticos actuales utilizan representaciones mejores y han tenido más éxito.

La incapacidad para manejar la «explosión combinatoria» fue una de las principales críticas que se hicieron a la IA en el informe de Lighthill (Lighthill, 1973), informe en el que se basó la decisión del gobierno británico para retirar la ayuda a las investigaciones sobre IA, excepto en dos universidades. (La tradición oral presenta un cuadro un

poco distinto y más animado, en el que se vislumbran ambiciones políticas y animadversiones personales, cuya descripción está fuera del ámbito de esta obra.)

El tercer obstáculo se derivó de las limitaciones inherentes a las estructuras básicas que se utilizaban en la generación de la conducta inteligente. Por ejemplo, en 1969, en el libro de Minsky y Papert, *Perceptrons*, se demostró que si bien era posible lograr que los perceptrones (una red neuronal simple) aprendieran cualquier cosa que pudiesen representar, su capacidad de representación era muy limitada. En particular, un perceptrón con dos entradas no se podía entrenar para que aprendiese a reconocer cuándo sus dos entradas eran diferentes. Si bien los resultados que obtuvieron no eran aplicables a redes más complejas multicapa, los fondos para la investigación de las redes neuronales se redujeron a prácticamente nada. Es irónico que los nuevos algoritmos de aprendizaje de retroalimentación utilizados en las redes multicapa y que fueron la causa del gran resurgimiento de la investigación en redes neuronales de finales de los años 80, en realidad, se hayan descubierto por primera vez en 1969 (Bryson y Ho, 1969).

Sistemas basados en el conocimiento: ¿clave del poder? (1969–1979)

El cuadro que dibujaba la resolución de problemas durante la primera década de la investigación en la IA estaba centrado en el desarrollo de mecanismos de búsqueda de propósito general, en los que se entrelazaban elementos de razonamiento básicos para encontrar así soluciones completas. A estos procedimientos se les ha denominado **métodos débiles**, debido a que no tratan problemas más amplios o más complejos. La alternativa a los métodos débiles es el uso de conocimiento específico del dominio que facilita el desarrollo de etapas de razonamiento más largas, pudiéndose así resolver casos recurrentes en dominios de conocimiento restringido. Podría afirmarse que para resolver un problema en la práctica, es necesario saber de antemano la correspondiente respuesta.

MÉTODOS DÉBILES

El programa DENDRAL (Buchanan *et al*., 1969) constituye uno de los primeros ejemplos de este enfoque. Fue diseñado en Stanford, donde Ed Feigenbaum (discípulo de Herbert Simon), Bruce Buchanan (filósofo convertido en informático) y Joshua Lederberg (genetista ganador del Premio Nobel) colaboraron en la solución del problema de inferir una estructura molecular a partir de la información proporcionada por un espectrómetro de masas. El programa se alimentaba con la fórmula elemental de la molécula (por ejemplo, $C_6H_{13}NO_2$) y el espectro de masas, proporcionando las masas de los distintos fragmentos de la molécula generada después de ser bombardeada con un haz de electrones. Por ejemplo, un espectro de masas con un pico en $m = 15$, correspondería a la masa de un fragmento de metilo (CH_3).

La versión más simple del programa generaba todas las posibles estructuras que correspondieran a la fórmula, luego predecía el espectro de masas que se observaría en cada caso, y comparaba éstos con el espectro real. Como era de esperar, el método anterior resultó pronto inviable para el caso de moléculas con un tamaño considerable. Los creadores de DENDRAL consultaron con químicos analíticos y se dieron cuenta de que éstos trabajaban buscando patrones conocidos de picos en el espectro que sugerían estructuras comunes en la molécula. Por ejemplo, par identificar el subgrupo (con un peso de 28) de las cetonas (C=0) se empleó la siguiente regla:

si hay dos picos en x_1 y x_2 tales que
a) $x_1 + x_2 = M + 28$ (siendo M la masa de toda la molécula);
b) $x_1 - 28$ es un pico alto;
c) $x_2 - 28$ es un pico alto;
d) al menos una de x_1 y x_2 es alta.
entonces existe un subgrupo de cetonas

Al reconocer que la molécula contiene una subestructura concreta se reduce el número de posibles candidatos de forma considerable. La potencia de DENDRAL se basaba en que:

> Toda la información teórica necesaria para resolver estos problemas se ha proyectado desde su forma general [componente predicho por el espectro] («primeros principios») a formas eficientes especiales («recetas de cocina»). (Feigenbaum *et al*., 1971)

La trascendencia de DENDRAL se debió a ser el primer sistema de *conocimiento intenso* que tuvo éxito: su base de conocimiento estaba formada por grandes cantidades de reglas de propósito particular. En sistemas diseñados posteriormente se incorporaron también los elementos fundamentales de la propuesta de McCarthy para el Generador de Consejos, la nítida separación del conocimiento (en forma de reglas) de la parte correspondiente al razonamiento.

SISTEMAS EXPERTOS

Teniendo en cuenta esta lección, Feigenbaum junto con otros investigadores de Stanford dieron comienzo al Proyecto de Programación Heurística, PPH, dedicado a determinar el grado con el que la nueva metodología de los **sistemas expertos** podía aplicarse a otras áreas de la actividad humana. El siguiente gran esfuerzo se realizó en el área del diagnóstico médico. Feigenbaum, Buchanan y el doctor Edward Shortliffe diseñaron el programa MYCIN, para el diagnóstico de infecciones sanguíneas. Con 450 reglas aproximadamente, MYCIN era capaz de hacer diagnósticos tan buenos como los de un experto y, desde luego, mejores que los de un médico recién graduado. Se distinguía de DENDRAL en dos aspectos principalmente. En primer lugar, a diferencia de las reglas de DENDRAL, no se contaba con un modelo teórico desde el cual se pudiesen deducir las reglas de MYCIN. Fue necesario obtenerlas a partir de extensas entrevistas con los expertos, quienes las habían obtenido de libros de texto, de otros expertos o de su experiencia directa en casos prácticos. En segundo lugar, las reglas deberían reflejar la incertidumbre inherente al conocimiento médico. MYCIN contaba con un elemento que facilitaba el cálculo de incertidumbre denominado **factores de certeza** (*véase* el Capítulo 13), que al parecer (en aquella época) correspondía muy bien a la manera como los médicos ponderaban las evidencias al hacer un diagnóstico.

La importancia del conocimiento del dominio se demostró también en el área de la comprensión del lenguaje natural. Aunque el sistema SHRDLU de Winograd para la comprensión del lenguaje natural había suscitado mucho entusiasmo, su dependencia del análisis sintáctico provocó algunos de los mismos problemas que habían aparecido en los trabajos realizados en la traducción automática. Era capaz de resolver los problemas de ambigüedad e identificar los pronombres utilizados, gracias a que se había diseñado especialmente para un área (el mundo de los bloques). Fueron varios los investigadores que, como Eugene Charniak, estudiante de Winograd en el MIT, opinaron que para una sólida comprensión del lenguaje era necesario contar con un conocimiento general sobre el mundo y un método general para usar ese conocimiento.

En Yale, el lingüista transformado en informático Roger Schank reforzó lo anterior al afirmar: «No existe eso que llaman sintaxis», lo que irritó a muchos lingüistas, pero sirvió para iniciar un útil debate. Schank y sus estudiantes diseñaron una serie de programas (Schank y Abelson, 1977; Wilensky, 1978; Schank y Riesbeck, 1981; Dyer, 1983) cuyo objetivo era la comprensión del lenguaje natural. El foco de atención estaba menos en el lenguaje *per se* y más en los problemas vinculados a la representación y razonamiento del conocimiento necesario para la comprensión del lenguaje. Entre los problemas estaba el de la representación de situaciones estereotipo (Cullingford, 1981), la descripción de la organización de la memoria humana (Rieger, 1976; Kolodner, 1983) y la comprensión de planes y objetivos (Wilensky, 1983).

El crecimiento generalizado de aplicaciones para solucionar problemas del mundo real provocó el respectivo aumento en la demanda de esquemas de representación del conocimiento que funcionaran. Se desarrolló una considerable cantidad de lenguajes de representación y razonamiento diferentes. Algunos basados en la lógica, por ejemplo el lenguaje Prolog gozó de mucha aceptación en Europa, aceptación que en Estados Unidos fue para la familia del PLANNER. Otros, siguiendo la noción de **marcos** de Minsky (1975), se decidieron por un enfoque más estructurado, al recopilar información sobre objetos concretos y tipos de eventos, organizando estos tipos en grandes jerarquías taxonómicas, similares a las biológicas.

MARCOS

La IA se convierte en una industria (desde 1980 hasta el presente)

El primer sistema experto comercial que tuvo éxito, R1, inició su actividad en Digital Equipment Corporation (McDermott, 1982). El programa se utilizaba en la elaboración de pedidos de nuevos sistemas informáticos. En 1986 representaba para la compañía un ahorro estimado de 40 millones de dólares al año. En 1988, el grupo de Inteligencia Artificial de DEC había distribuido ya 40 sistemas expertos, y había más en camino. Du Pont utilizaba ya 100 y estaban en etapa de desarrollo 500 más, lo que le generaba ahorro de diez millones de dólares anuales aproximadamente. Casi todas las compañías importantes de Estados Unidos contaban con su propio grupo de IA, en el que se utilizaban o investigaban sistemas expertos.

En 1981 los japoneses anunciaron el proyecto «Quinta Generación», un plan de diez años para construir computadores inteligentes en los que pudiese ejecutarse Prolog. Como respuesta Estados Unidos constituyó la Microelectronics and Computer Technology Corporation (MCC), consorcio encargado de mantener la competitividad nacional en estas áreas. En ambos casos, la IA formaba parte de un gran proyecto que incluía el diseño de chips y la investigación de la relación hombre máquina. Sin embargo, los componentes de IA generados en el marco de MCC y del proyecto Quinta Generación nunca alcanzaron sus objetivos. En el Reino Unido, el informe Alvey restauró el patrocinio suspendido por el informe Lighthill[15].

[15] Para evitar confusiones, se creó un nuevo campo denominado Sistemas Inteligentes Basados en Conocimiento (IKBS, *Intelligent Knowledge-Based Systems*) ya que la Inteligencia Artificial había sido oficialmente cancelada.

En su conjunto, la industria de la IA creció rápidamente, pasando de unos pocos millones de dólares en 1980 a billones de dólares en 1988. Poco después de este período llegó la época llamada «El Invierno de la IA», que afectó a muchas empresas que no fueron capaces de desarrollar los extravagantes productos prometidos.

Regreso de las redes neuronales (desde 1986 hasta el presente)

Aunque la informática había abandonado de manera general el campo de las redes neuronales a finales de los años 70, el trabajo continuó en otros campos. Físicos como John Hopfield (1982) utilizaron técnicas de la mecánica estadística para analizar las propiedades de almacenamiento y optimización de las redes, tratando colecciones de nodos como colecciones de átomos. Psicólogos como David Rumelhart y Geoff Hinton continuaron con el estudio de modelos de memoria basados en redes neuronales. Como se verá en el Capítulo 20, el impulso más fuerte se produjo a mediados de la década de los 80, cuando por lo menos cuatro grupos distintos reinventaron el algoritmo de aprendizaje de retroalimentación, mencionado por vez primera en 1969 por Bryson y Ho. El algoritmo se aplicó a diversos problemas de aprendizaje en los campos de la informática y la psicología, y la gran difusión que conocieron los resultados obtenidos, publicados en la colección *Parallel Distributed Processing* (Rumelhart y McClelland, 1986), suscitó gran entusiasmo.

CONEXIONISTAS

Aquellos modelos de inteligencia artificial llamados **conexionistas**[16] fueron vistos por algunos como competidores tanto de los modelos simbólicos propuestos por Newell y Simon como de la aproximación lógica de McCarthy entre otros (Smolensky, 1988). Puede parecer obvio que los humanos manipulan símbolos hasta cierto nivel, de hecho, el libro *The Symbolic Species* (1997) de Terrence Deacon sugiere que esta es la *característica que define* a los humanos, pero los conexionistas más ardientes se preguntan si la manipulación de los símbolos desempeña algún papel justificable en determinados modelos de cognición. Este interrogante no ha sido aún clarificado, pero la tendencia actual es que las aproximaciones conexionistas y simbólicas son complementarias y no competidoras.

IA se convierte en una ciencia (desde 1987 hasta el presente)

En los últimos años se ha producido una revolución tanto en el contenido como en la metodología de trabajo en el campo de la inteligencia artificial.[17] Actualmente es más usual el desarrollo sobre teorías ya existentes que proponer teorías totalmente novedo-

[16] Se usa la traducción literal del término *connectionist* por no existir un término equivalente en español (*N. del RT*).

[17] Hay quien ha caracterizado este cambio como la victoria de los pulcros (aquellos que consideran que las teorías de IA deben basarse rigurosamente en las matemáticas) sobre los desaliñados (aquellos que después de intentar muchas ideas, escriben algunos programas y después evalúan las que aparentemente funcionan). Ambos enfoques son útiles. Esta tendencia en favor de una mayor pulcritud es señal de que el campo ha alcanzado cierto nivel de estabilidad y madurez. Lo cual no implica que tal estabilidad se puede ver alterada con el surgimiento de otras ideas poco estructuradas.

sas, tomar como base rigurosos teoremas o sólidas evidencias experimentales más que intuición, y demostrar la utilidad de las aplicaciones en el mundo real más que crear ejemplos de juguete.

La IA se fundó en parte en el marco de una rebelión en contra de las limitaciones de los campos existentes como la teoría de control o la estadística, y ahora abarca estos campos. Tal y como indica David McAllester (1998),

> En los primeros años de la IA parecía perfectamente posible que las nuevas formas de la computación simbólica, por ejemplo, los marcos y las redes semánticas, hicieran que la mayor parte de la teoría clásica pasara a ser obsoleta. Esto llevó a la IA a una especie de aislamiento, que la separó del resto de las ciencias de la computación. En la actualidad se está abandonando este aislamiento. Existe la creencia de que el aprendizaje automático no se debe separar de la teoría de la información, que el razonamiento incierto no se debe separar de los modelos estocásticos, de que la búsqueda no se debe aislar de la optimización clásica y el control, y de que el razonamiento automático no se debe separar de los métodos formales y del análisis estático.

En términos metodológicos, se puede decir, con rotundidad, que la IA ya forma parte del ámbito de los métodos científicos. Para que se acepten, las hipótesis se deben someter a rigurosos experimentos empíricos, y los resultados deben analizarse estadísticamente para identificar su relevancia (Cohen, 1995). El uso de Internet y el compartir repositorios de datos de prueba y código, ha hecho posible que ahora se puedan contrastar experimentos.

Un buen modelo de la tendencia actual es el campo del reconocimiento del habla. En la década de los 70 se sometió a prueba una gran variedad de arquitecturas y enfoques. Muchos de ellos fueron un tanto *ad hoc* y resultaban frágiles, y fueron probados sólo en unos pocos ejemplos elegidos especialmente. En años recientes, las aproximaciones basadas en los **modelos de Markov ocultos**, MMO, han pasado a dominar el área. Dos son las características de los MMO que tienen relevancia. Primero, se basan en una rigurosa teoría matemática, lo cual ha permitido a los investigadores del lenguaje basarse en los resultados de investigaciones matemáticas hechas en otros campos a lo largo de varias décadas. En segundo lugar, los modelos se han generado mediante un proceso de aprendizaje en grandes *corpus* de datos de lenguaje reales. Lo cual garantiza una funcionalidad robusta, y en sucesivas pruebas ciegas, los MMO han mejorado sus resultados a un ritmo constante. La tecnología del habla y el campo relacionado del reconocimiento de caracteres manuscritos están actualmente en transición hacia una generalizada utilización en aplicaciones industriales y de consumo.

Las redes neuronales también siguen esta tendencia. La mayor parte del trabajo realizado con redes neuronales en la década de los 80 se realizó con la idea de dejar a un lado lo que se podía hacer y de descubrir en qué se diferenciaban las redes neuronales de otras técnicas «tradicionales». La utilización de metodologías mejoradas y marcos teóricos, ha autorizado que este campo alcance un grado de conocimiento que ha permitido que ahora las redes neuronales se puedan comparar con otras técnicas similares de campos como la estadística, el reconocimiento de patrones y el aprendizaje automático, de forma que las técnicas más prometedoras pueden aplicarse a cualquier problema. Como resultado de estos desarrollos, la tecnología denominada **minería de datos** ha generado una nueva y vigorosa industria.

MINERÍA DE DATOS

La aparición de *Probabilistic Reasoning in Intelligent Systems* de Judea Pearl (1988) hizo que se aceptara de nuevo la probabilidad y la teoría de la decisión como parte de la IA, como consecuencia del resurgimiento del interés despertado y gracias especialmente al artículo *In Defense of Probability* de Peter Cheeseman (1985). El formalismo de las **redes de Bayes** apareció para facilitar la representación eficiente y el razonamiento riguroso en situaciones en las que se disponía de conocimiento incierto. Este enfoque supera con creces muchos de los problemas de los sistemas de razonamiento probabilístico de las décadas de los 60 y 70; y ahora domina la investigación de la IA en el razonamiento incierto y los sistemas expertos. Esta aproximación facilita el aprendizaje a partir de la experiencia, y combina lo mejor de la IA clásica y las redes neuronales. El trabajo de Judea Pearl (1982a) y de Eric Horvitz y David Heckerman (Horvitz y Heckerman, 1986; Horvitz *et al*., 1986) sirvió para promover la noción de sistemas expertos *normativos*: es decir, los que actúan racionalmente de acuerdo con las leyes de la teoría de la decisión, sin que intenten imitar las etapas de razonamiento de los expertos humanos. El sistema operativo Windows™ incluye varios sistemas expertos de diagnóstico normativos para la corrección de problemas. En los Capítulos 13 y 16 se aborda este tema.

Revoluciones similares y suaves se han dado en robótica, visión por computador, y aprendizaje automático. La comprensión mejor de los problemas y de su complejidad, junto a un incremento en la sofisticación de las matemáticas ha facilitado el desarrollo de una agenda de investigación y de métodos más robustos. En cualquier caso, la formalización y especialización ha llevado también a la fragmentación: áreas como la visión y la robótica están cada vez más aislados de la «rama central» de la IA. La concepción unificadora de IA como diseño de agentes racionales puede facilitar la unificación de estos campos diferentes.

Emergencia de los sistemas inteligentes (desde 1995 hasta el presente)

Quizás animados por el progreso en la resolución de subproblemas de IA, los investigadores han comenzado a trabajar de nuevo en el problema del «agente total». El trabajo de Allen Newell, John Laird, y Paul Rosenbloom en SOAR (Newell, 1990; Laird *et al*., 1987) es el ejemplo mejor conocido de una arquitectura de agente completa. El llamado «movimiento situado» intenta entender la forma de actuar de los agentes inmersos en entornos reales, que disponen de sensores de entradas continuas. Uno de los medios más importantes para los agentes inteligentes es Internet. Los sistemas de IA han llegado a ser tan comunes en aplicaciones desarrolladas para la Web que el sufijo «-bot» se ha introducido en el lenguaje común. Más aún, tecnologías de IA son la base de muchas herramientas para Internet, como por ejemplo motores de búsqueda, sistemas de recomendación, y los sistemas para la construcción de portales Web.

Además de la primera edición de este libro de texto (Russell y Norvig, 1995), otros libros de texto han adoptado recientemente la perspectiva de agentes (Poole *et al*., 1998; Nilsson, 1998). Una de las conclusiones que se han extraído al tratar de construir agentes completos ha sido que se deberían reorganizar los subcampos aislados de la IA para

que sus resultados se puedan interrelacionar. En particular, ahora se cree mayoritariamente que los sistemas sensoriales (visión, sónar, reconocimiento del habla, etc.) no pueden generar información totalmente fidedigna del medio en el que habitan. Otra segunda consecuencia importante, desde la perspectiva del agente, es que la IA se ha ido acercando a otros campos, como la teoría de control y la economía, que también tratan con agentes.

1.4 El estado del arte

¿Qué es capaz de hacer la IA hoy en día? Responder de manera concisa es difícil porque hay muchas actividades divididas en muchos subcampos. Aquí se presentan unas cuantas aplicaciones; otras aparecerán a lo largo del texto.

Planificación autónoma: a un centenar de millones de millas de la Tierra, el programa de la NASA Agente Remoto se convirtió en el primer programa de planificación autónoma a bordo que controlaba la planificación de las operaciones de una nave espacial desde abordo (Jonsson *et al*., 2000). El Agente Remoto generaba planes a partir de objetivos generales especificados desde tierra, y monitorizaba las operaciones de la nave espacial según se ejecutaban los planes (detección, diagnóstico y recuperación de problemas según ocurrían).

Juegos: Deep Blue de IBM fue el primer sistema que derrotó a un campeón mundial en una partida de ajedrez cuando superó a Garry Kasparov por un resultado de 3.5 a 2.5 en una partida de exhibición (Goodman y Keene, 1997). Kasparov dijo que había percibido un «nuevo tipo de inteligencia» al otro lado del tablero. La revista *Newsweek* describió la partida como «La partida final». El valor de las acciones de IBM se incrementó en 18 billones de dólares.

Control autónomo: el sistema de visión por computador ALVINN fue entrenado para dirigir un coche de forma que siguiese una línea. Se instaló en una furgoneta controlada por computador en el NAVLAB de UCM y se utilizó para dirigir al vehículo por Estados Unidos. Durante 2.850 millas controló la dirección del vehículo en el 98 por ciento del trayecto. Una persona lo sustituyó en el dos por ciento restante, principalmente en vías de salida. El NAVLAB posee videocámaras que transmiten imágenes de la carretera a ALVINN, que posteriormente calcula la mejor dirección a seguir, basándose en las experiencias acumuladas en los viajes de entrenamiento.

Diagnosis: los programas de diagnóstico médico basados en el análisis probabilista han llegado a alcanzar niveles similares a los de médicos expertos en algunas áreas de la medicina. Heckerman (1991) describe un caso en el que un destacado experto en la patología de los nodos linfáticos se mofó del diagnóstico generado por un programa en un caso especialmente difícil. El creador del programa le sugirió que le preguntase al computador cómo había generado el diagnóstico. La máquina indicó los factores más importantes en los que había basado su decisión y explicó la ligera interacción existente entre varios de los síntomas en este caso. Eventualmente, el experto aceptó el diagnóstico del programa.

Planificación logística: durante la crisis del Golfo Pérsico de 1991, las fuerzas de Estados Unidos desarrollaron la herramienta *Dynamic Analysis and Replanning Tool*

(DART) (Cross y Walker, 1994), para automatizar la planificación y organización logística del transporte. Lo que incluía hasta 50.000 vehículos, carga y personal a la vez, teniendo en cuenta puntos de partida, destinos, rutas y la resolución de conflictos entre otros parámetros. Las técnicas de planificación de IA permitieron que se generara un plan en cuestión de horas que podría haber llevado semanas con otros métodos. La agencia DARPA (*Defense Advanced Research Project Agency*) afirmó que esta aplicación por sí sola había más que amortizado los 30 años de inversión de DARPA en IA.

Robótica: muchos cirujanos utilizan hoy en día asistentes robot en operaciones de microcirugía. HipNav (DiGioia *et al*., 1996) es un sistema que utiliza técnicas de visión por computador para crear un modelo tridimensional de la anatomía interna del paciente y después utiliza un control robotizado para guiar el implante de prótesis de cadera.

Procesamiento de lenguaje y resolución de problemas: PROVER B (Littman *et al*., 1999) es un programa informático que resuelve crucigramas mejor que la mayoría de los humanos, utilizando restricciones en programas de relleno de palabras, una gran base de datos de crucigramas, y varias fuentes de información como diccionarios y bases de datos *online*, que incluyen la lista de películas y los actores que intervienen en ellas, entre otras cosas. Por ejemplo, determina que la pista «Historia de Niza» se puede resolver con «ETAGE» ya que su base de datos incluye el par pista/solución «Historia en Francia/ETAGE» y porque reconoce que los patrones «Niza X» y «X en Francia» a menudo tienen la misma solución. El programa no sabe que Niza es una ciudad de Francia, pero es capaz de resolver el puzle.

Estos son algunos de los ejemplos de sistemas de inteligencia artificial que existen hoy en día. No se trata de magia o ciencia ficción, son más bien ciencia, ingeniería y matemáticas, para los que este libro proporciona una introducción.

1.5 Resumen

En este capítulo se define la IA y se establecen los antecedentes culturales que han servido de base. Algunos de los aspectos más destacables son:

- Cada uno tiene una visión distinta de lo que es la IA. Es importante responder a las dos preguntas siguientes: ¿Está interesado en el razonamiento y el comportamiento? ¿Desea modelar seres humanos o trabajar a partir de un ideal estándar?
- En este libro se adopta el criterio de que la inteligencia tiene que ver principalmente con las **acciones racionales**. Desde un punto de vista ideal, un **agente inteligente** es aquel que emprende la mejor acción posible ante una situación dada. Se estudiará el problema de la construcción de agentes que sean inteligentes en este sentido.
- Los filósofos (desde el año 400 a.C.) facilitaron el poder imaginar la IA, al concebir la idea de que la mente es de alguna manera como una máquina que funciona a partir del conocimiento codificado en un lenguaje interno, y al considerar que el pensamiento servía para seleccionar la acción a llevar a cabo.
- Las matemáticas proporcionaron las herramientas para manipular tanto las aseveraciones de certeza lógicas, como las inciertas de tipo probabilista. Asimismo,

prepararon el terreno para un entendimiento de lo que es el cálculo y el razonamiento con algoritmos.

- Los economistas formalizaron el problema de la toma de decisiones para maximizar los resultados esperados.
- Los psicólogos adoptaron la idea de que los humanos y los animales podían considerarse máquinas de procesamiento de información. Los lingüistas demostraron que el uso del lenguaje se ajusta a ese modelo.
- Los informáticos proporcionaron los artefactos que hicieron posible la aplicación de la IA. Los programas de IA tienden a ser extensos y no podrían funcionar sin los grandes avances en velocidad y memoria aportados por la industria informática.
- La teoría de control se centra en el diseño de dispositivos que actúan de forma óptima con base en la retroalimentación que reciben del entorno en el que están inmersos. Inicialmente, las herramientas matemáticas de la teoría de control eran bastante diferentes a las técnicas que utilizaba la IA, pero ambos campos se están acercando.
- Las historia de la IA ha pasado por ciclos de éxito, injustificado optimismo y consecuente desaparición de entusiasmo y apoyos financieros. También ha habido ciclos caracterizados por la introducción de enfoques nuevos y creativos y de un perfeccionamiento sistemático de los mejores.
- La IA ha avanzado más rápidamente en la década pasada debido al mayor uso del método científico en la experimentación y comparación de propuestas.
- Los avances recientes logrados en el entendimiento de las bases teóricas de la inteligencia han ido aparejados con las mejoras realizadas en la optimización de los sistemas reales. Los subcampos de la IA se han integrado más y la IA ha encontrado elementos comunes con otras disciplinas.



NOTAS BIBLIOGRÁFICAS E HISTÓRICAS

El estatus metodológico de la inteligencia artificial se ha investigado en *The Sciences of the Artificial*, escrito por Herb Simon (1981), en el cual se analizan áreas de investigación interesadas en el desarrollo de artefactos complejos. Explica cómo la IA se puede ver como ciencia y matemática. Cohen (1995) proporciona una visión de la metodología experimental en el marco de la IA. Ford y Hayes (1995) presentan una revisión crítica de la utilidad de la Prueba de Turing.

Artificial Intelligence: The Very Idea, de John Haugeland (1985), muestra una versión amena de los problemas prácticos y filosóficos de la IA. La ciencia cognitiva está bien descrita en varios textos recientes (Johnson-Laird, 1988; Stillings *et al.*, 1995; Thagard, 1996) y en la *Encyclopedia of the Cognitive Sciences* (Wilson y Keil, 1999). Baker (1989) cubre la parte sintáctica de la lingüística moderna, y Chierchia y McConnell-Ginet (1990) la semántica. Jurafsky y Martin (2000) revisan la lingüística computacional.

Los primeros trabajos en el campo de la IA se citan en *Computers and Thought* (1963), de Feigenbaum y Feldman, en *Semantic Information Processing* de Minsky, y en la serie *Machine Intelligence*, editada por Donald Michie. Webber y Nilsson (1981)

y Luger (1995) han recogido una nutrida cantidad de artículos influyentes. Los primeros artículos sobre redes neuronales están reunidos en *Neurocomputing* (Anderson y Rosenfeld, 1988). La *Encyclopedia of AI* (Shapiro, 1992) contiene artículos de investigación sobre prácticamente todos los temas de IA. Estos artículos son muy útiles para iniciarse en las diversas áreas presentes en la literatura científica.

El trabajo más reciente se encuentra en las actas de las mayores conferencias de IA: la *International Joint Conference on AI* (IJCAI), de carácter bianual, la *European Conference on AI* (ECAI), de carácter bianual, y la *National Conference on AI*, conocida normalmente como AAAI por la organización que la patrocina. Las revistas científicas que presentan aspectos generales de la IA más importantes son *Artificial Intelligence, Computational Intelligence, IEEE Transactions on Pattern Analysis and Machine Intelligence, IEEE Intelligent Systems*, y la revista electrónica *Journal of Artificial Intelligence Research*. Hay también numerosas revistas y conferencias especializadas en áreas concretas, que se mencionarán en los capítulos apropiados. La asociaciones profesionales de IA más importantes son la American Association for Artificial Intelligence (AAAI), la ACM Special Interest Group in Artificial Intelligence (SIGART), y la Society for Artificial Intelligence and Simulation of Behaviour (AISB). La revista *AI Magazine* de AAAI contiene muchos artículos de interés general y manuales, y su página Web, aaai.org contiene noticias e información de referencia.



EJERCICIOS

El propósito de los siguientes ejercicios es estimular la discusión, y algunos de ellos se podrían utilizar como proyectos. Alternativamente, se podría hacer un esfuerzo inicial para solucionarlos ahora, de forma que una vez se haya leído todo el libro se puedan revisar estos primeros intentos.

1.1 Defina con sus propias palabras: (a) inteligencia, (b) inteligencia artificial, (c) agente.



1.2 Lea el artículo original de Turing sobre IA (Turing, 1950). En él se comentan algunas objeciones potenciales a su propuesta y a su prueba de inteligencia. ¿Cuáles de estas objeciones tiene todavía validez? ¿Son válidas sus refutaciones? ¿Se le ocurren nuevas objeciones a esta propuesta teniendo en cuenta los desarrollos realizados desde que se escribió el artículo? En el artículo, Turing predijo que para el año 2000 sería probable que un computador tuviera un 30 por ciento de posibilidades de superar una Prueba de Turing dirigida por un evaluador inexperto con una duración de cinco minutos. ¿Considera razonable lo anterior en el mundo actual? ¿Y en los próximos 50 años?



1.3 Todos los años se otorga el premio Loebner al programa que lo hace mejor en una Prueba de Turing concreta. Investigue y haga un informe sobre el último ganador del premio Loebner. ¿Qué técnica utiliza? ¿Cómo ha hecho que progrese la investigación en el campo de la IA?

1.4 Hay clases de problemas bien conocidos que son intratables para los computadores, y otras clases sobre los cuales un computador no pueda tomar una decisión. ¿Quiere esto decir que es imposible lograr la IA?

1.5 Supóngase que se extiende ANALOGY, el programa de Evans, como para alcanzar una puntuación de 200 en una prueba normal de cociente de inteligencia. ¿Quiere decir lo anterior que se ha creado un programa más inteligente que un ser humano? Explíquese.

1.6 ¿Cómo puede la introspección (revisión de los pensamientos íntimos) ser inexacta? ¿Se puede estar equivocado sobre lo que se cree? Discútase.



1.7 Consulte en la literatura existente sobre la IA si alguna de las siguientes tareas se puede efectuar con computadores:

a) Jugar una partida de tenis de mesa (ping-pong) decentemente.
b) Conducir un coche en el centro del Cairo.
c) Comprar comestibles para una semana en el mercado.
d) Comprar comestibles para una semana en la web.
e) Jugar una partida de *bridge* decentemente a nivel de competición.
f) Descubrir y demostrar nuevos teoremas matemáticos.
g) Escribir intencionadamente una historia divertida.
h) Ofrecer asesoría legal competente en un área determinada.
i) Traducir inglés hablado al sueco hablado en tiempo real.
j) Realizar una operación de cirugía compleja.

En el caso de las tareas que no sean factibles de realizar en la actualidad, trate de describir cuáles son las dificultades y calcule para cuándo se podrán superar.

1.8 Algunos autores afirman que la percepción y las habilidades motoras son la parte más importante de la inteligencia y que las capacidades de «alto nivel» son más bien parásitas (simples añadidos a las capacidades básicas). Es un hecho que la mayor parte de la evolución y que la mayor parte del cerebro se han concentrado en la percepción y las habilidades motoras, en tanto la IA ha descubierto que tareas como juegos e inferencia lógica resultan más sencillas, en muchos sentidos, que percibir y actuar en el mundo real. ¿Consideraría usted que ha sido un error la concentración tradicional de la IA en las capacidades cognitivas de alto nivel?

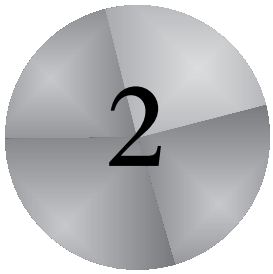
1.9 ¿Por qué la evolución tiende a generar sistemas que actúan racionalmente? ¿Qué objetivos deben intentar alcanzar estos sistemas?

1.10 ¿Son racionales las acciones reflejas (como retirar la mano de una estufa caliente)? ¿Son inteligentes?

1.11 «En realidad los computadores no son inteligentes, hacen solamente lo que le dicen los programadores». ¿Es cierta la última aseveración, e implica a la primera?

1.12 «En realidad los animales no son inteligentes, hacen solamente lo que le dicen sus genes». ¿Es cierta la última aseveración, e implica a la primera?

1.13 «En realidad los animales, los humanos y los computadores no pueden ser inteligentes, ellos sólo hacen lo que los átomos que los forman les dictan siguiendo las leyes de la física». ¿Es cierta la última aseveración, e implica a la primera?



2 Agentes inteligentes

Donde se discutirá la naturaleza de los agentes ideales, sus diversos hábitats y las formas de organizar los tipos de agentes existentes.

El Capítulo 1 identifica el concepto de **agente racional** como central en la perspectiva de la inteligencia artificial que presenta este libro. Esta noción se concreta más a lo largo de este capítulo. Se mostrará como el concepto de racionalidad se puede aplicar a una amplia variedad de agentes que operan en cualquier medio imaginable. En el libro, la idea es utilizar este concepto para desarrollar un pequeño conjunto de principios de diseño que sirvan para construir agentes útiles, sistemas que se puedan llamar razonablemente **inteligentes**.

Se comienza examinando los agentes, los medios en los que se desenvuelven, y la interacción entre éstos. La observación de que algunos agentes se comportan mejor que otros nos lleva naturalmente a la idea de agente racional, aquel que se comporta tan bien como puede. La forma de actuar del agente depende de la naturaleza del medio; algunos hábitats son más complejos que otros. Se proporciona una categorización cruda del medio y se muestra cómo las propiedades de un hábitat influyen en el diseño de agentes adecuados para ese entorno. Se presenta un número de «esquemas» básicos para el diseño de agentes, a los que se dará cuerpo a lo largo del libro.

2.1 Agentes y su entorno

MEDIOAMBIENTE

Un **agente** es cualquier cosa capaz de percibir su **medioambiente** con la ayuda de **sensores** y actuar en ese medio utilizando **actuadores**[1]. La Figura 2.1 ilustra esta idea sim-

[1] Se usa este término para indicar el elemento que reacciona a un estímulo realizando una acción (*N. del RT*).

PROGRAMA DEL AGENTE

mentará mediante el **programa del agente**. Es importante diferenciar estas dos ideas. La función del agente es una descripción matemática abstracta; el programa del agente es una implementación completa, que se ejecuta sobre la arquitectura del agente.

Para ilustrar esta idea se utilizará un ejemplo muy simple, el mundo de la aspiradora presentado en la Figura 2.2. Este mundo es tan simple que se puede describir todo lo que en él sucede; es un mundo hecho a medida, para el que se pueden inventar otras variaciones. Este mundo en particular tiene solamente dos localizaciones: cuadrícula *A* y *B*. La aspiradora puede percibir en qué cuadrante se encuentra y si hay suciedad en él. Puede elegir si se mueve hacia la izquierda, derecha, aspirar la suciedad o no hacer nada. Una función muy simple para el agente vendría dada por: si la cuadrícula en la que se encuentra está sucia, entonces aspirar, de otra forma cambiar de cuadrícula. Una muestra parcial de la función del agente representada en forma de tabla aparece en la Figura 2.3. Un programa de agente simple para esta función de agente se mostrará posteriormente en la Figura 2.8.

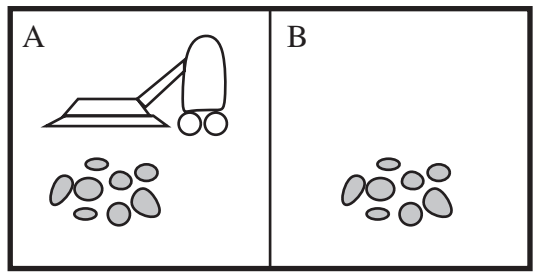


Figura 2.2 El mundo de la aspiradora con dos localizaciones solamente.

Secuencia de percepciones	Acción
[*A, Limpio*]	*Derecha*
[*A, Sucio*]	*Aspirar*
[*B, Limpio*]	*Izquierda*
[*B, Sucio*]	*Aspirar*
[*A, Limpio*], [*A, Limpio*]	*Derecha*
[*A, Limpio*], [*A, Sucio*]	*Aspirar*
—	—
—	—
—	—
[*A, Limpio*], [*A, Limpio*], [*A, Limpio*]	*Derecha*
[*A, Limpio*], [*A, Limpio*], [*A, Sucio*]	*Aspirar*
—	—
—	—
—	—

Figura 2.3 Tabla parcial de una función de agente sencilla para el mundo de la aspiradora que se muestra en la Figura 2.2.

Revisando la Figura 2.3, se aprecia que se pueden definir varios agentes para el mundo de la aspiradora simplemente rellenando la columna de la derecha de formas distintas. La pregunta obvia, entonces es: *¿cuál es la mejor forma de rellenar una tabla*? En otras palabras, ¿qué hace que un agente sea bueno o malo, inteligente o estúpido? Estas preguntas se responden en la siguiente sección.

Antes de terminar esta sección, es necesario remarcar que la noción de agente es supuestamente una herramienta para el análisis de sistemas, y no una caracterización absoluta que divida el mundo entre agentes y no agentes. Se puede ver una calculadora de mano como un agente que elige la acción de mostrar «4» en la pantalla, dada la secuencia de percepciones «2 + 2 =». Pero este análisis difícilmente puede mejorar nuestro conocimiento acerca de las calculadoras.

2.2 Buen comportamiento: el concepto de racionalidad

AGENTE RACIONAL

Un **agente racional** es aquel que hace lo correcto; en términos conceptuales, cada elemento de la tabla que define la función del agente se tendría que rellenar correctamente. Obviamente, hacer lo correcto es mejor que hacer algo incorrecto, pero ¿qué significa hacer lo correcto? Como primera aproximación, se puede decir que lo correcto es aquello que permite al agente obtener un resultado mejor. Por tanto, se necesita determinar una forma de medir el éxito. Ello, junto a la descripción del entorno y de los sensores y actuadores del agente, proporcionará una especificación completa de la tarea que desempeña el agente. Dicho esto, ahora es posible definir de forma más precisa qué significa la racionalidad.

Medidas de rendimiento

MEDIDAS DE RENDIMIENTO

Las **medidas de rendimiento** incluyen los criterios que determinan el éxito en el comportamiento del agente. Cuando se sitúa un agente en un medio, éste genera una secuencia de acciones de acuerdo con las percepciones que recibe. Esta secuencia de acciones hace que su hábitat pase por una secuencia de estados. Si la secuencia es la deseada, entonces el agente habrá actuado correctamente. Obviamente, no hay una única medida adecuada para todos los agentes. Se puede preguntar al agente por su opinión subjetiva acerca de su propia actuación, pero muchos agentes serían incapaces de contestar, y otros podrían engañarse a sí mismos[3]. Por tanto hay que insistir en la importancia de utilizar medidas de rendimiento objetivas, que normalmente determinará el diseñador encargado de la construcción del agente.

Si retomamos el ejemplo de la aspiradora de la sección anterior, se puede proponer utilizar como medida de rendimiento la cantidad de suciedad limpiada en un período de

[3] Los agentes humanos son conocidos en particular por su «acidez», hacen creer que no quieren algo después de no haberlo podido conseguir, por ejemplo, «Ah bueno, de todas formas no quería ese estúpido Premio Nobel».

ocho horas. Con agentes racionales, por supuesto, se obtiene lo que se demanda. Un agente racional puede maximizar su medida de rendimiento limpiando la suciedad, tirando la basura al suelo, limpiándola de nuevo, y así sucesivamente. Una medida de rendimiento más adecuada recompensaría al agente por tener el suelo limpio. Por ejemplo, podría ganar un punto por cada cuadrícula limpia en cada período de tiempo (quizás habría que incluir algún tipo de penalización por la electricidad gastada y el ruido generado). *Como regla general, es mejor diseñar medidas de utilidad de acuerdo con lo que se quiere para el entorno, más que de acuerdo con cómo se cree que el agente debe comportarse*.



La selección de la medida de rendimiento no es siempre fácil. Por ejemplo, la noción de «suelo limpio» del párrafo anterior está basada en un nivel de limpieza medio a lo largo del tiempo. Además, este nivel medio de limpieza se puede alcanzar de dos formas diferentes, llevando a cabo una limpieza mediocre pero continua o limpiando en profundidad, pero realizando largos descansos. La forma más adecuada de hacerlo puede venir dada por la opinión de un encargado de la limpieza profesional, pero en realidad es una cuestión filosófica profunda con fuertes implicaciones. ¿Qué es mejor, una vida temeraria con altos y bajos, o una existencia segura pero aburrida? ¿Qué es mejor, una economía en la que todo el mundo vive en un estado de moderada pobreza o una en la que algunos viven en la abundancia y otros son muy pobres? Estas cuestiones se dejan como ejercicio para los lectores diligentes.

Racionalidad

La racionalidad en un momento determinado depende de cuatro factores:

- La medida de rendimiento que define el criterio de éxito.
- El conocimiento del medio en el que habita acumulado por el agente.
- Las acciones que el agente puede llevar a cabo.
- La secuencia de percepciones del agente hasta este momento.

DEFINICIÓN DE AGENTE RACIONAL

Esto nos lleva a la **definición de agente racional**:



> *En cada posible secuencia de percepciones, un agente racional deberá emprender aquella acción que supuestamente maximice su medida de rendimiento, basándose en las evidencias aportadas por la secuencia de percepciones y en el conocimiento que el agente mantiene almacenado.*

Considerando que el agente aspiradora limpia una cuadrícula si está sucia y se mueve a la otra si no lo está (ésta es la función del agente que aparece en la tabla de la Figura 2.3), ¿se puede considerar racional? ¡Depende! Primero, se debe determinar cuál es la medida de rendimiento, qué se conoce del entorno, y qué sensores y actuadores tiene el agente. Si asumimos que:

- La medida de rendimiento premia con un punto al agente por cada recuadro limpio en un período de tiempo concreto, a lo largo de una «vida» de 1.000 períodos.
- La «geografía» del medio se conoce *a priori* (Figura 2.2), pero que la distribución de la suciedad y la localización inicial del agente no se conocen. Las cuadrículas se mantienen limpias y aspirando se limpia la cuadrícula en que se encuentre el agente. Las acciones *Izquierda* y *Derecha* mueven al agente hacia la izquierda y

derecha excepto en el caso de que ello pueda llevar al agente fuera del recinto, en este caso el agente permanece donde se encuentra.
- Las únicas acciones permitidas son *Izquierda*, *Derecha*, *Aspirar* y *NoOp* (no hacer nada).
- El agente percibe correctamente su localización y si esta localización contiene suciedad.

Puede afirmarse que *bajo estas circunstancias* el agente es verdaderamente racional; el rendimiento que se espera de este agente es por lo menos tan alto como el de cualquier otro agente. El Ejercicio 2.4 pide que se pruebe este hecho.

Fácilmente se puede observar que el agente puede resultar irracional en circunstancias diferentes. Por ejemplo, cuando toda la suciedad se haya eliminado el agente oscilará innecesariamente hacia delante y atrás; si la medida de rendimiento incluye una penalización de un punto por cada movimiento hacia la derecha e izquierda, la respuesta del agente será pobre. Un agente más eficiente no hará nada si está seguro de que todas las cuadrículas están limpias. Si una cuadrícula se ensucia de nuevo, el agente debe identificarlo en una de sus revisiones ocasionales y limpiarla. Si no se conoce la geografía del entorno, el agente tendrá que explorarla y no quedarse parado en las cuadrículas *A* y *B*. El Ejercicio 2.4 pide que se diseñen agentes para estos casos.

Omnisciencia, aprendizaje y autonomía

OMNISCIENCIA

Es necesario tener cuidado al distinguir entre racionalidad y **omnisciencia**. Un agente omnisciente conoce el resultado de su acción y actúa de acuerdo con él; sin embargo, en realidad la omnisciencia no es posible. Considerando el siguiente ejemplo: estoy paseando por los Campos Elíseos y veo un amigo al otro lado de la calle. No hay tráfico alrededor y no tengo ningún compromiso, entonces, actuando racionalmente, comenzaría a cruzar la calle. Al mismo tiempo, a 33.000 pies de altura, se desprende la puerta de un avión[4], y antes de que termine de cruzar al otro lado de la calle me encuentro aplastado. ¿Fue irracional cruzar la calle? Sería de extrañar que en mi nota necrológica apareciera «Un idiota intentando cruzar la calle».

Este ejemplo muestra que la racionalidad no es lo mismo que la perfección. La racionalidad maximiza el rendimiento esperado, mientras la perfección maximiza el resultado real. Alejarse de la necesidad de la perfección no es sólo cuestión de hacer justicia con los agentes. El asunto es que resulta imposible diseñar un agente que siempre lleve a cabo, de forma sucesiva, las mejores acciones después de un acontecimiento, a menos que se haya mejorado el rendimiento de las bolas de cristal o las máquinas de tiempo.

La definición propuesta de racionalidad no requiere omnisciencia, ya que la elección racional depende sólo de la secuencia de percepción hasta *la fecha*. Es necesario asegurase de no haber permitido, por descuido, que el agente se dedique decididamente a llevar a cabo acciones poco inteligentes. Por ejemplo, si el agente no mirase a ambos lados de la calle antes de cruzar una calle muy concurrida, entonces su secuencia de per-

[4] *Véase* N. Henderson, «New door latches urged for Boeing 747 jumbo jets» (es urgente dotar de nuevas cerraduras a las puertas de los Boeing jumbo 747), *Washington Post*, 24 de agosto de 1989.

cepción no le indicaría que se está acercando un gran camión a gran velocidad. ¿La definición de racionalidad nos está indicando que está bien cruzar la calle? ¡Todo lo contrario! Primero, no sería racional cruzar la calle sólo teniendo esta secuencia de percepciones incompleta: el riesgo de accidente al cruzarla sin mirar es demasiado grande. Segundo, un agente racional debe elegir la acción de «mirar» antes de intentar cruzar la calle, ya que el mirar maximiza el rendimiento esperado. Llevar a cabo acciones *con la intención de modificar percepciones futuras*, en ocasiones proceso denominado **recopilación de información**, es una parte importante de la racionalidad y se comenta en profundidad en el Capítulo 16. Un segundo ejemplo de recopilación de información lo proporciona la **exploración** que debe llevar a cabo el agente aspiradora en un medio inicialmente desconocido.

RECOPILACIÓN DE INFORMACIÓN

EXPLORACIÓN

La definición propuesta implica que el agente racional no sólo recopile información, sino que **aprenda** lo máximo posible de lo que está percibiendo. La configuración inicial del agente puede reflejar un conocimiento preliminar del entorno, pero a medida que el agente adquiere experiencia éste puede modificarse y aumentar. Hay casos excepcionales en los que se conoce totalmente el entorno *a priori*. En estos casos, el agente no necesita percibir y aprender; simplemente actúa de forma correcta. Por supuesto, estos agentes son muy frágiles. Considérese el caso del humilde escarabajo estercolero. Después de cavar su nido y depositar en él su huevos, tomó una bola de estiércol de una pila cercana para tapar su entrada. Si *durante el trayecto* se le quita la bola, el escarabajo continuará su recorrido y hará como si estuviera tapando la entrada del nido, sin tener la bola y sin darse cuanta de ello. La evolución incorporó una suposición en la conducta del escarabajo, y cuando se viola, el resultado es un comportamiento insatisfactorio. La avispa cavadora es un poco más inteligente. La avispa hembra cavará una madriguera, saldrá de ella, picará a una oruga y la llevará a su madriguera, se introducirá en la madriguera para comprobar que todo está bien, arrastrará la oruga hasta el fondo y pondrá sus huevos. La oruga servirá como fuente de alimento cuando los huevos se abran. Hasta ahora todo bien, pero si un entomólogo desplaza la oruga unos centímetros fuera cuando la avispa está revisando la situación, ésta volverá a la etapa de «arrastre» que figura en su plan, y continuará con el resto del plan sin modificación alguna, incluso después de que se intervenga para desplazar la oruga. La avispa cavadora no es capaz de aprender que su plan innato está fallando, y por tanto no lo cambiará.

APRENDIZAJE

Los agentes con éxito dividen las tareas de calcular la función del agente en tres períodos diferentes: cuando se está diseñando el agente, y están los diseñadores encargados de realizar algunos de estos cálculos; cuando está pensando en la siguiente operación, el agente realiza más cálculos; y cuando está aprendiendo de la experiencia, el agente lleva a cabo más cálculos para decidir cómo modificar su forma de comportarse.

AUTONOMÍA

Se dice que un agente carece de **autonomía** cuando se apoya más en el conocimiento inicial que le proporciona su diseñador que en sus propias percepciones. Un agente racional debe ser autónomo, debe saber aprender a determinar cómo tiene que compensar el conocimiento incompleto o parcial inicial. Por ejemplo, el agente aspiradora que aprenda a prever dónde y cuándo aparecerá suciedad adicional lo hará mejor que otro que no aprenda. En la práctica, pocas veces se necesita autonomía completa desde el comienzo: cuando el agente haya tenido poca o ninguna experiencia, tendrá que actuar de forma aleatoria a menos que el diseñador le haya proporcionado ayuda. Así, de la

misma forma que la evolución proporciona a los animales sólo los reactivos necesarios para que puedan sobrevivir lo suficiente para aprender por ellos mismos, sería razonable proporcionar a los agentes que disponen de inteligencia artificial un conocimiento inicial, así como de la capacidad de aprendizaje. Después de las suficientes experiencias interaccionando con el entorno, el comportamiento del agente racional será efectivamente *independiente* del conocimiento que poseía inicialmente. De ahí, que la incorporación del aprendizaje facilite el diseño de agentes racionales individuales que tendrán éxito en una gran cantidad de medios.

2.3 La naturaleza del entorno

ENTORNOS DE TRABAJO

Ahora que se tiene una definición de racionalidad, se está casi preparado para pensar en la construcción de agentes racionales. Primero, sin embargo, hay que centrarse en los **entornos de trabajo**, que son esencialmente los «problemas» para los que los agentes racionales son las «soluciones». Para ello se comienza mostrando cómo especificar un entorno de trabajo, ilustrando el proceso con varios ejemplos. Posteriormente se mostrará que el entorno de trabajo ofrece diferentes posibilidades, de forma que cada una de las posibilidades influyen directamente en el diseño del programa del agente.

Especificación del entorno de trabajo

REAS

En la discusión de la racionalidad de un agente aspiradora simple, hubo que especificar las medidas de rendimiento, el entorno, y los actuadores y sensores del agente. Todo ello forma lo que se llama el **entorno de trabajo**, para cuya denominación se utiliza el acrónimo **REAS** (**R**endimiento, **E**ntorno, **A**ctuadores, **S**ensores). En el diseño de un agente, el primer paso debe ser siempre especificar el entorno de trabajo de la forma más completa posible.

El mundo de la aspiradora fue un ejemplo simple; considérese ahora un problema más complejo: un taxista automático. Este ejemplo se utilizará a lo largo del capítulo. Antes de alarmar al lector, conviene aclarar que en la actualidad la construcción de un taxi automatizado está fuera del alcance de la tecnología actual. *Véase* en la página 31 la descripción de un robot conductor que ya existe en la actualidad, o lea las actas de la conferencia *Intelligent Transportation Systems*. La tarea de conducir un automóvil, en su totalidad, es extremadamente *ilimitada*. No hay límite en cuanto al número de nuevas combinaciones de circunstancias que pueden surgir (por esta razón se eligió esta actividad en la presente discusión). La Figura 2.4 resume la descripción REAS para el entorno de trabajo del taxi. El próximo párrafo explica cada uno de sus elementos en más detalle.

Primero, ¿cuál es el **entorno de trabajo** en el que el taxista automático aspira a conducir? Dentro de las cualidades deseables que debería tener se incluyen el que llegue al destino correcto; que minimice el consumo de combustible; que minimice el tiempo de viaje y/o coste; que minimice el número de infracciones de tráfico y de molestias a otros conductores; que maximice la seguridad, la comodidad del pasajero y el

Tipo de agente	Medidas de rendimiento	Entorno	Actuadores	Sensores
Taxista	Seguro, rápido, legal, viaje confortable, maximización del beneficio	Carreteras, otro tráfico, peatones, clientes	Dirección, acelerador, freno, señal, bocina, visualizador	Cámaras, sónar, velocímetro, GPS, tacómetro, visualizador de la aceleración, sensores del motor, teclado

Figura 2.4 Descripción REAS del entorno de trabajo de un taxista automático.

beneficio. Obviamente, alguno de estos objetivos entran en conflicto por lo que habrá que llegar a acuerdos.

Siguiente, ¿cuál es el **entorno** en el que se encontrará el taxi? Cualquier taxista debe estar preparado para circular por distintas carreteras, desde caminos rurales y calles urbanas hasta autopistas de 12 carriles. En las carreteras se pueden encontrar con tráfico, peatones, animales, obras, coches de policía, charcos y baches. El taxista también tiene que comunicarse tanto con pasajeros reales como potenciales. Hay también elecciones opcionales. El taxi puede operar en California del Sur, donde la nieve es raramente un problema, o en Alaska, donde raramente no lo es. Puede conducir siempre por la derecha, o puede ser lo suficientemente flexible como para que circule por la izquierda cuando se encuentre en el Reino Unido o en Japón. Obviamente, cuanto más restringido esté el entorno, más fácil será el problema del diseño.

Los **actuadores** disponibles en un taxi automático serán más o menos los mismos que los que tiene a su alcance un conductor humano: el control del motor a través del acelerador y control sobre la dirección y los frenos. Además, necesitará tener una pantalla de visualización o un sintetizador de voz para responder a los pasajeros, y quizás algún mecanismo para comunicarse, educadamente o de otra forma, con otros vehículos.

Para alcanzar sus objetivos en el entorno en el que circula, el taxi necesita saber dónde está, qué otros elementos están en la carretera, y a qué velocidad circula. Sus **sensores** básicos deben, por tanto, incluir una o más cámaras de televisión dirigidas, un velocímetro y un tacómetro. Para controlar el vehículo adecuadamente, especialmente en las curvas, debe tener un acelerador; debe conocer el estado mecánico del vehículo, de forma que necesitará sensores que controlen el motor y el sistema eléctrico. Debe tener instrumentos que no están disponibles para un conductor medio: un sistema de posicionamiento global vía satélite (GPS) para proporcionarle información exacta sobre su posición con respecto a un mapa electrónico, y sensores infrarrojos o sonares para detectar las distancias con respecto a otros coches y obstáculos. Finalmente, necesitará un teclado o micrófono para que el pasajero le indique su destino.

La Figura 2.5 muestra un esquema con los elementos REAS básicos para diferentes clases de agentes adicionales. Más ejemplos aparecerán en el Ejercicio 2.5. Puede sorprender a algunos lectores que se incluya en la lista de tipos de agente algunos programas que operan en la totalidad del entorno artificial definido por las entradas del teclado y los caracteres impresos en el monitor. «Seguramente», nos podamos pregun-

Tipo de agente	Medidas de rendimiento	Entorno	Actuadores	Sensores
Sistema de diagnóstico médico	Pacientes sanos, reducir costes, demandas	Pacientes, hospital, personal	Visualizar preguntas, pruebas, diagnósticos, tratamientos, casos	Teclado para la entrada de síntomas, conclusiones, respuestas de pacientes
Sistema de análisis de imágenes de satélites	Categorización de imagen correcta	Conexión con el satélite en órbita	Visualizar la categorización de una escena	Matriz de pixels de colores
Robot para la selección de componentes	Porcentaje de componentes clasificados en los cubos correctos	Cinta transportadora con componentes, cubos	Brazo y mano articulados	Cámara, sensor angular
Controlador de una refinería	Maximizar la pureza, producción y seguridad	Refinería, operadores	Válvulas, bombas, calentadores, monitores	Temperatura, presión, sensores químicos
Tutor de inglés interactivo	Maximizar la puntuación de los estudiantes en los exámenes	Conjunto de estudiantes, agencia examinadora	Visualizar los ejercicios, sugerencias, correcciones	Teclado de entrada

Figura 2.5 Ejemplos de tipos de agentes y sus descripciones REAS.

tar, «¿este no es un entorno real, verdad?». De hecho, lo que importa no es la distinción entre un medio «real» y «artificial», sino la complejidad de la relación entre el comportamiento del agente, la secuencia de percepción generada por el medio y la medida de rendimiento. Algunos entornos «reales» son de hecho bastante simples. Por ejemplo, un robot diseñado para inspeccionar componentes según pasan por una cinta transportadora puede hacer uso de varias suposiciones simples: que la cinta siempre estará iluminada, que conocerá todos los componentes que circulen por la cinta, y que hay solamente dos acciones (aceptar y rechazar).

AGENTES *SOFTWARE*

SOFTBOTS

En contraste, existen algunos **agentes *software*** (o robots *software* o **softbots**) en entornos ricos y prácticamente ilimitados. Imagine un softbot diseñado para pilotar el simulador de vuelo de un gran avión comercial. El simulador constituye un medio muy detallado y complejo que incluye a otros aviones y operaciones de tierra, y el agente *software* debe elegir, en tiempo real, una de entre un amplio abanico de posibilidades. O imagine un robot diseñado para que revise fuentes de información en Internet y para que muestre aquellas que sean interesantes a sus clientes. Para lograrlo, deberá poseer cierta habilidad en el procesamiento de lenguaje natural, tendrá que aprender qué es lo que le interesa a cada cliente, y tendrá que ser capaz de cambiar sus planes dinámica-

mente, por ejemplo, cuando se interrumpa la conexión con una fuente de información o cuando aparezca una nueva. Internet es un medio cuya complejidad rivaliza con la del mundo físico y entre cuyos habitantes se pueden incluir muchos agentes artificiales.

Propiedades de los entornos de trabajo

El rango de los entornos de trabajo en los que se utilizan técnicas de IA es obviamente muy grande. Sin embargo, se puede identificar un pequeño número de dimensiones en las que categorizar estos entornos. Estas dimensiones determinan, hasta cierto punto, el diseño más adecuado para el agente y la utilización de cada una de las familias principales de técnicas en la implementación del agente. Primero se enumeran la dimensiones, y después se analizan varios entornos de trabajo para ilustrar estas ideas. Las definiciones dadas son informales; capítulos posteriores proporcionan definiciones más precisas y ejemplos de cada tipo de entorno.

TOTALMENTE OBSEVABLE

- **Totalmente observable** *vs.* **parcialmente observable.**
  Si los sensores del agente le proporcionan acceso al estado completo del medio en cada momento, entonces se dice que el entorno de trabajo es totalmente observable[5]. Un entorno de trabajo es, efectivamente, totalmente observable si los sensores detectan todos los aspectos que son relevantes en la toma de decisiones; la relevancia, en cada momento, depende de las medidas de rendimiento. Entornos totalmente observables son convenientes ya que el agente no necesita mantener ningún estado interno para saber qué sucede en el mundo. Un entorno puede ser parcialmente observable debido al ruido y a la existencia de sensores poco exactos o porque los sensores no reciben información de parte del sistema, por ejemplo, un agente aspiradora con sólo un sensor de suciedad local no puede saber si hay suciedad en la otra cuadrícula, y un taxi automatizado no pude saber qué están pensando otros conductores.

DETERMINISTA

ESTOCÁSTICO

- **Determinista** *vs.* **estocástico.**
  Si el siguiente estado del medio está totalmente determinado por el estado actual y la acción ejecutada por el agente, entonces se dice que el entorno es determinista; de otra forma es estocástico. En principio, un agente no se tiene que preocupar de la incertidumbre en un medio totalmente observable y determinista. Sin embargo, si el medio es parcialmente observable entonces puede *parecer* estocástico. Esto es particularmente cierto si se trata de un medio complejo, haciendo difícil el mantener constancia de todos las aspectos observados. Así, a menudo es mejor pensar en entornos deterministas o estocásticos *desde el punto de vista del agente*. El agente taxi es claramente estocástico en este sentido, ya que no se puede predecir el comportamiento del tráfico exactamente; más aún, una rueda se puede reventar y un motor se puede gripar sin previo aviso. El mundo de la aspiradora es deter-

[5] La primera edición de este libro utiliza los términos **accesible** e **inaccesible** en vez de **total** y **parcialmente observable; no determinista** en vez de **estocástico;** y **no episódico** en vez de **secuencial.** La nueva terminología es más consistente con el uso establecido.

minista, como ya se describió, pero las variaciones pueden incluir elementos estocásticos como la aparición de suciedad aleatoria y un mecanismo de succión ineficiente (Ejercicio 2.12). Si el medio es determinista, excepto para las acciones de otros agentes, decimos que el medio es **estratégico**.

ESTRATÉGICO

EPISÓDICO

SECUENCIAL

- **Episódico** *vs.* **secuencial**[6]**.**
  En un entorno de trabajo episódico, la experiencia del agente se divide en episodios atómicos. Cada episodio consiste en la percepción del agente y la realización de una única acción posterior. Es muy importante tener en cuenta que el siguiente episodio no depende de las acciones que se realizaron en episodios previos. En los medios episódicos la elección de la acción en cada episodio depende sólo del episodio en sí mismo. Muchas tareas de clasificación son episódicas. Por ejemplo, un agente que tenga que seleccionar partes defectuosas en una cadena de montaje basa sus decisiones en la parte que está evaluando en cada momento, sin tener en cuenta decisiones previas; más aún, a la decisión presente no le afecta el que la próxima fase sea defectuosa. En entornos secuenciales, por otro lado, la decisión presente puede afectar a decisiones futuras. El ajedrez y el taxista son secuenciales: en ambos casos, las acciones que se realizan a corto plazo pueden tener consecuencias a largo plazo. Los medios episódicos son más simples que los secuenciales porque la gente no necesita pensar con tiempo.

ESTÁTICO

DINAMICO

- **Estático** *vs.* **dinámico.**
  Si el entorno puede cambiar cuando el agente está deliberando, entonces se dice que el entorno es dinámico para el agente; de otra forma se dice que es estático. Los medios estáticos son fáciles de tratar ya que el agente no necesita estar pendiente del mundo mientras está tomando una decisión sobre una acción, ni necesita preocuparse sobre el paso del tiempo. Los medios dinámicos, por el contrario, están preguntando continuamente al agente qué quiere hacer; si no se ha decidido aún, entonces se entiende que ha tomado la decisión de no hacer nada. Si el entorno no cambia con el paso del tiempo, pero el rendimiento del agente cambia, entonces se dice que el medio es **semidinámico**. El taxista es claramente dinámico: tanto los otros coches como el taxi se están moviendo mientras el algoritmo que guía la conducción indica qué es lo próximo a hacer. El ajedrez, cuando se juega con un reloj, es semideterminista. Los crucigramas son estáticos.

SEMIDINÁMICO

DISCRETO

CONTINUO

- **Discreto** *vs.* **continuo.**
  La distinción entre discreto y continuo se puede aplicar al *estado* del medio, a la forma en la que se maneja el *tiempo* y a las *percepciones* y *acciones* del agente. Por ejemplo, un medio con estados discretos como el del juego del ajedrez tiene un número finito de estados distintos. El ajedrez tiene un conjunto discreto de percepciones y acciones. El taxista conduciendo define un estado continuo y un problema de tiempo continuo: la velocidad y la ubicación del taxi y de los otros vehículos pasan por un rango de valores continuos de forma suave a lo largo del

[6] La palabra «secuencial» se utiliza también en el campo de la informática como antónimo de «paralelo». Los dos significados no están relacionados.

tiempo. Las conducción del taxista es también continua (ángulo de dirección, etc.). Las imágenes captadas por cámaras digitales son discretas, en sentido estricto, pero se tratan típicamente como representaciones continuas de localizaciones e intensidades variables.

AGENTE INDIVIDUAL

MULTIAGENTE

- **Agente individual** *vs.* **multiagente.**
  La distinción entre el entorno de un agente individual y el de un sistema multiagente puede parecer suficientemente simple. Por ejemplo, un agente resolviendo un crucigrama por sí mismo está claramente en un entorno de agente individual, mientras que un agente que juega al ajedrez está en un entorno con dos agentes. Sin embargo hay algunas diferencias sutiles. Primero, se ha descrito que una entidad *puede* percibirse como un agente, pero no se ha explicado qué entidades se *deben* considerar agentes. ¿Tiene el agente *A* (por ejemplo el agente taxista) que tratar un objeto *B* (otro vehículo) como un agente, o puede tratarse méramente como un objeto con un comportamiento estocástico, como las olas de la playa o las hojas que mueve el viento? La distinción clave está en identificar si el comportamiento de *B* está mejor descrito por la maximización de una medida de rendimiento cuyo valor depende del comportamiento de *A*. Por ejemplo, en el ajedrez, la entidad oponente *B* intenta maximizar su medida de rendimiento, la cual, según las reglas, minimiza la medida de rendimiento del agente *A*. Por tanto, el ajedrez es un entorno multiagente **competitivo**. Por otro lado, en el medio definido por el taxista circulando, el evitar colisiones maximiza la medida de rendimiento de todos los agentes, así pues es un entorno multiagente parcialmente **cooperativo**. Es también parcialmente competitivo ya que, por ejemplo, sólo un coche puede ocupar una plaza de aparcamiento. Los problemas en el diseño de agentes que aparecen en los entornos multiagente son a menudo bastante diferentes de los que aparecen en entornos con un único agente; por ejemplo, la **comunicación** a menudo emerge como un comportamiento racional en entornos multiagente; en algunos entornos competitivos parcialmente observables el **comportamiento estocástico** es racional ya que evita las dificultades de la predicción.

COMPETITIVO

COOPERATIVO

Como es de esperar, el caso más complejo es el *parcialmente observable*, *estocástico*, *secuencial*, *dinámico*, *continuo* y *multiagente*. De hecho, suele suceder que la mayoría de las situaciones reales son tan complejas que sería discutible clasificarlas como *realmente* deterministas. A efectos prácticos, se deben tratar como estocásticas. Un taxista circulando es un problema, complejo a todos los efectos.

La Figura 2.6 presenta las propiedades de un número de entornos familiares. Hay que tener en cuenta que las respuestas no están siempre preparadas de antemano. Por ejemplo, se ha presentado el ajedrez como totalmente observable; en sentido estricto, esto es falso porque ciertas reglas que afectan al movimiento de las torres, el enroque y a movimientos por repetición requieren que se recuerden algunos hechos sobre la historia del juego que no están reflejados en el estado del tablero. Estas excepciones, por supuesto, no tienen importancia si las comparamos con aquellas que aparecen en el caso del taxista, el tutor de inglés, o el sistema de diagnóstico médico.

Entornos de trabajo	Observable	Determinista	Episódico	Estático	Discreto	Agentes
Crucigrama	Totalmente	Determinista	Secuencial	Estático	Discreto	Individual
Ajedrez con reloj	Totalmente	Estratégico	Secuencial	Semi	Discreto	Multi
Póker	Parcialmente	Estratégico	Secuencial	Estático	Discreto	Multi
Backgammon	Totalmente	Estocástico	Secuencial	Estático	Discreto	Multi
Taxi circulando	Parcialmente	Estocástico	Secuencial	Dinámico	Continuo	Multi
Diagnóstico médico	Parcialmente	Estocástico	Secuencial	Dinámico	Continuo	Individual
Análisis de imagen	Totalmente	Determinista	Episódico	Semi	Continuo	Individual
Robot clasificador	Parcialmente	Estocástico	Episódico	Dinámico	Continuo	Individual
Controlador de refinería	Parcialmente	Estocástico	Secuencial	Dinámico	Continuo	Individual
Tutor interactivo de inglés	Parcialmente	Estocástico	Secuencial	Dinámico	Discreto	Multi

Figura 2.6 Ejemplos de entornos de trabajo y sus características.

Otras entradas de la tabla dependen de cómo se haya definido el entorno de trabajo. Se ha definido el sistema de diagnóstico médico como un único agente porque no es rentable modelar el proceso de la enfermedad en un paciente como un agente; pero incluso el sistema de diagnóstico médico podría necesitar tener en cuenta a pacientes recalcitrantes y empleados escépticos, de forma que el entorno podría tener un aspecto multiagente. Más aún, el diagnóstico médico es episódico si se concibe como proporcionar un diagnóstico a partir de una lista de síntomas; el problema es secuencial si ello trae consigo la propuesta de una serie de pruebas, un proceso de evaluación a lo largo del tratamiento, y demás aspectos. Muchos entornos son, también, episódicos si se observan desde un nivel de abstracción más alto que el de las acciones individuales del agente. Por ejemplo, un torneo de ajedrez consiste en una secuencia de juegos; cada juego es un episodio, pero (a la larga) la contribución de los movimientos en una partida al resultado general que obtenga el agente no se ve afectada por los movimientos realizados en la partida anterior. Por otro lado, las decisiones tomadas en una partida concreta son ciertamente de tipo secuencial.

El repositorio de código asociado a este libro (aima.cs.berkeley.edu) incluye la implementación de un número de entornos, junto con un simulador de entornos de propósito general que sitúa uno o más agentes en un entorno simulado, observa su comportamiento a lo largo del tiempo, y los evalúa de acuerdo con una medida de rendimiento dada. Estos experimentos no sólo se han realizado para un medio concreto, sino que se han realizado con varios problemas obtenidos de una **clase de entornos**. Por ejemplo, para evaluar un taxista en un tráfico simulado, sería interesante hacer varias simulaciones con diferente tipo de tráfico, claridad y condiciones atmosféricas. Si se diseña un agente para un escenario concreto, se pueden sacar ventajas de las propiedades específicas de ese caso en particular, pero puede no identificarse un buen diseño para conducir en general. Por esta razón, el repositorio de código también incluye un **generador de entornos** para cada clase de medios que selecciona hábitats particulares (con ciertas posibilidades) en los que ejecutar los agentes. Por ejemplo, el generador de un entorno

CLASE DE ENTORNOS

GENERADOR DE ENTORNOS

para un agente aspiradora inicializa el patrón de suciedad y la localización del agente de forma aleatoria. Después, es interesante evaluar la eficacia media del agente en el contexto de la clase del entorno. Un agente racional para una clase de entorno maximiza el rendimiento medio. Los Ejercicios del 2.7 al 2.12 guían el proceso de desarrollo de una clase de entornos y la evaluación de varios agentes.

2.4 Estructura de los agentes

PROGRAMA DEL AGENTE

ARQUITECTURA

Hasta este momento se ha hablado de los agentes describiendo su *conducta*, la acción que se realiza después de una secuencia de percepciones dada. Ahora, se trata de centrarse en el núcleo del problema y hablar sobre cómo trabajan internamente. El trabajo de la IA es diseñar el **programa del agente** que implemente la función del agente que proyecta las percepciones en las acciones. Se asume que este programa se ejecutará en algún tipo de computador con sensores físicos y actuadores, lo cual se conoce como **arquitectura:**

$$Agente = arquitectura + programa$$

Obviamente, el programa que se elija tiene que ser apropiado para la arquitectura. Si el programa tiene que recomendar acciones como *Caminar*, la arquitectura tiene que tener piernas. La arquitectura puede ser un PC común, o puede ser un coche robotizado con varios computadores, cámaras, y otros sensores a bordo. En general, la arquitectura hace que las percepciones de los sensores estén disponibles para el programa, ejecuta los programas, y se encarga de que los actuadores pongan en marcha las acciones generadas. La mayor parte de este libro se centra en el diseño de programas para agentes, aunque los Capítulos 24 y 25 tratan sobre sensores y actuadores.

Programas de los agentes

Los programas de los agentes que se describen en este libro tienen la misma estructura: reciben las percepciones actuales como entradas de los sensores y devuelven una acción a los actuadores[7]. Hay que tener en cuenta la diferencia entre los programas de los agentes, que toman la percepción actual como entrada, y la función del agente, que recibe la percepción histórica completa. Los programas de los agentes reciben sólo la percepción actual como entrada porque no hay nada más disponible en el entorno; si las acciones del agente dependen de la secuencia completa de percepciones, el agente tendría que recordar las percepciones.

Los programas de los agente se describirán con la ayuda de un sencillo lenguaje pseudocódigo que se define en el Apéndice B. El repositorio de código disponible en Inter-

[7] Hay otras posibilidades para definir la estructura del programa para el agente; por ejemplo, los programas para agentes pueden ser **subrutinas** que se ejecuten asincrónicamente en el entorno de trabajo. Cada una de estas subrutinas tienen un puerto de entrada y salida y consisten en un bucle que interpreta las entradas del puerto como percepciones y escribe acciones en el puerto de salida.

```
función AGENTE-DIRIGIDO-MEDIANTE TABLA(percepción) devuelve una acción
    variables estáticas: percepciones, una secuencia, vacía inicialmente
                         tabla, una tabla de acciones, indexada por las secuencias de
                         percepciones, totalmente definida inicialmente

    añadir la percepción al final de las percepciones
    acción ← CONSULTA(percepciones, tabla)
    devolver acción
```

Figura 2.7 El programa AGENTE-DIRIGIDO-MEDIANTE TABLA se invoca con cada nueva percepción y devuelve una acción en cada momento. Almacena la secuencia de percepciones utilizando su propia estructura de datos privada.

net contiene implementaciones en lenguajes de programación reales. Por ejemplo, la Figura 2.7 muestra un programa de agente muy sencillo que almacena la secuencia de percepciones y después las compara con las secuencias almacenadas en la tabla de acciones para decidir qué hacer. La tabla representa explícitamente la función que define el programa del agente. Para construir un agente racional de esta forma, los diseñadores deben realizar una tabla que contenga las acciones apropiadas para cada secuencia posible de percepciones.

Intuitivamente se puede apreciar por qué la propuesta de dirección-mediante-tabla para la construcción de agentes está condenada al fracaso. Sea P el conjunto de posibles percepciones y T el tiempo de vida del agente (el número total de percepciones que recibirá). La tabla de búsqueda contendrá $\sum_{t=1}^{T} |P|^t$ entradas. Si consideramos ahora el taxi automatizado: la entrada visual de una cámara individual es de 27 megabytes por segundo (30 fotografías por segundo, 640 × 480 pixels con 24 bits de información de colores). Lo cual genera una tabla de búsqueda con más de $10^{250.000.000.000}$ entradas por hora de conducción. Incluso la tabla de búsqueda del ajedrez (un fragmento del mundo real pequeño y obediente) tiene por lo menos 10^{150} entradas. El tamaño exageradamente grande de estas tablas (el número de átomos en el universo observable es menor que 10^{80}) significa que (a) no hay agente físico en este universo que tenga el espacio suficiente como para almacenar la tabla, (b) el diseñador no tendrá tiempo para crear la tabla, (c) ningún agente podría aprender todas las entradas de la tabla a partir de su experiencia, y (d) incluso si el entorno es lo suficientemente simple para generar una tabla de un tamaño razonable, el diseñador no tiene quien le asesore en la forma en la que rellenar la tabla.

A pesar de todo ello, el AGENTE-DIRIGIDO-MEDIANTE TABLA *hace* lo que nosotros queremos: implementa la función deseada para el agente. El desafío clave de la IA es encontrar la forma de escribir programas, que en la medida de lo posible, reproduzcan un comportamiento racional a partir de una pequeña cantidad de código en vez de a partir de una tabla con un gran número de entradas. Existen bastantes ejemplos que muestran qué se puede hacer con éxito en otras áreas: por ejemplo, las grandes tablas de las raíces cuadradas utilizadas por ingenieros y estudiantes antes de 1970 se han reemplazado por un programa de cinco líneas que implementa el método de Newton en las calculadoras electrónicas. La pregunta es, en el caso del comportamiento inteligente general, ¿puede la IA hacer lo que Newton hizo con las raíces cuadradas? Creemos que la respuesta es afirmativa.

En lo que resta de esta sección se presentan los cuatro tipos básicos de programas para agentes que encarnan los principios que subyacen en casi todos los sistemas inteligentes.

- Agentes reactivos simples.
- Agentes reactivos basados en modelos.
- Agentes basados en objetivos.
- Agentes basados en utilidad.

Después se explica, en términos generales, cómo convertir todos ellos en *agentes que aprendan*.

Agentes reactivos simples

AGENTE REACTIVO SIMPLE

El tipo de agente más sencillo es el **agente reactivo simple**. Estos agentes seleccionan las acciones sobre la base de las percepciones *actuales*, ignorando el resto de las percepciones históricas. Por ejemplo, el agente aspiradora cuya función de agente se presentó en la Figura 2.3 es un agente reactivo simple porque toma sus decisiones sólo con base en la localización actual y si ésta está sucia. La Figura 2.8 muestra el programa para este agente.

Hay que tener en cuenta que el programa para el agente aspiradora es muy pequeño comparado con su tabla correspondiente. La reducción más clara se obtiene al ignorar la historia de percepción, que reduce el número de posibilidades de 4^T a sólo 4. Otra reducción se basa en el hecho de que cuando la cuadrícula actual está sucia, la acción no depende de la localización.

Imagínese que es el conductor del taxi automático. Si el coche que circula delante frena, y las luces de freno se encienden, entonces lo advertiría y comenzaría a frenar. En otras palabras, se llevaría a cabo algún tipo de procesamiento sobre las señales visuales para establecer la condición que se llama «El coche que circula delante está frenando». Esto dispara algunas conexiones establecidas en el programa del agente para que se ejecute la acción «iniciar frenado». Esta conexión se denomina **regla de condición-acción**[8], y se representa por

REGLA DE CONDICIÓN-ACCIÓN

si *el-coche-que-circula-delante-está-frenando* **entonces** *iniciar-frenada*.

función AGENTE-ASPIRADORA-REACTIVO([*localización*, *estado*]) **devuelve** una acción

si *estado* = *Sucio* **entonces devolver** *Aspirar*
de otra forma, si *localización* = *A* **entonces devolver** *Derecha*
de otra forma, si *localización* = *B* **entonces devolver** *Izquierda*

Figura 2.8 Programa para el agente aspiradora de reactivo simple en el entorno definido por las dos cuadrículas. Este programa implementa la función de agente presentada en la Figura 2.3.

[8] También llamadas **reglas de situación-acción**, **producciones**, o **reglas si-entonces**.

Los humanos también tienen muchas de estas conexiones, algunas de las cuales son respuestas aprendidas (como en el caso de la conducción) y otras son reacciones innatas (como parpadear cuando algo se acerca al ojo). A lo largo de esta obra, se estudiarán diferentes formas en las que se pueden aprender e implementar estas conexiones.

El programa de la Figura 2.8 es específico para el entorno concreto de la aspiradora. Una aproximación más general y flexible es la de construir primero un intérprete de propósito general para reglas de condición-acción y después crear conjuntos de reglas para entornos de trabajo específicos. La Figura 2.9 presenta la estructura de este programa general de forma esquemática, mostrando cómo las reglas de condición-acción permiten al agente generar la conexión desde las percepciones a las acciones. No se preocupe si le parece trivial; pronto se complicará. Se utilizan rectángulos para denotar el estado interno actual del proceso de toma de decisiones del agente y óvalos para representar la información base utilizada en el proceso. El programa del agente, que es también muy simple, se muestra en la Figura 2.10. La función INTERPRETAR-ENTRADA genera una descripción abstracta del estado actual a partir de la percepción, y la función REGLA-COINCIDENCIA devuelve la primera regla del conjunto de reglas que coincide con la descripción del estado dada. Hay que tener en cuenta que la descripción en términos de «reglas»

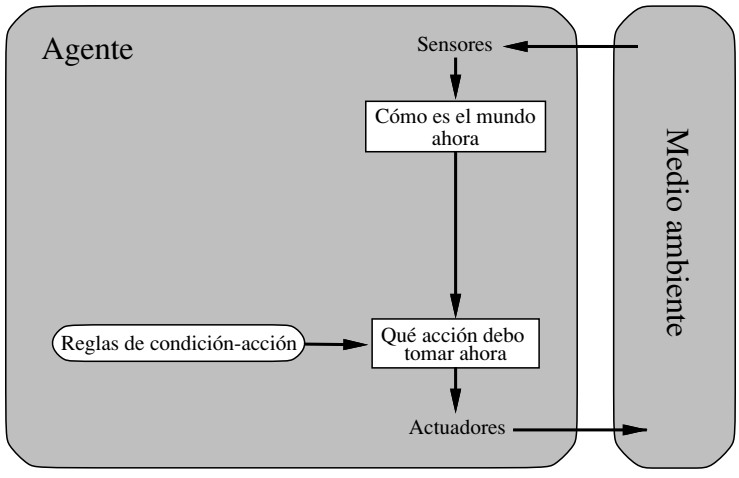


Figura 2.9 Diagrama esquemático de un agente reactivo simple.

```
función AGENTE-REACTIVO-SIMPLE(percepción) devuelve una acción
    estático: reglas, un conjunto de reglas condición-acción

    estado ← INTERPRETAR-ENTRADA(percepción)
    regla ← REGLA-COINCIDENCIA(estado, reglas)
    acción ← REGLA-ACCIÓN[regla]
    devolver acción
```

Figura 2.10 Un agente reactivo simple, que actúa de acuerdo a la regla cuya condición coincida con el estado actual, definido por la percepción.

y «coincidencias» es puramente conceptual; las implementaciones reales pueden ser tan simples como colecciones de puertas lógicas implementando un circuito booleano.



Los agentes reactivos simples tienen la admirable propiedad de ser simples, pero poseen una inteligencia muy limitada. El agente de la Figura 2.10 funcionará *sólo si se puede tomar la decisión correcta sobre la base de la percepción actual, lo cual es posible sólo si el entorno es totalmente observable*. Incluso el que haya una pequeña parte que no se pueda observar puede causar serios problemas. Por ejemplo, la regla de frenado dada anteriormente asume que la condición *el-coche-que-circula-delante-está-frenando* se puede determinar a partir de la percepción actual (imagen de vídeo actual) si el coche de enfrente tiene un sistema centralizado de luces de freno. Desafortunadamente, los modelos antiguos tienen diferentes configuraciones de luces traseras, luces de frenado, y de intermitentes, y no es siempre posible saber a partir de una única imagen si el coche está frenando. Un agente reactivo simple conduciendo detrás de un coche de este tipo puede frenar continuamente y de manera innecesaria, o peor, no frenar nunca.

Un problema similar aparece en el mundo de la aspiradora. Supongamos que se elimina el sensor de localización de un agente aspiradora reactivo simple, y que sólo tiene un sensor de suciedad. Un agente de este tipo tiene sólo dos percepciones posibles: [*Sucio*] y [*Limpio*]. Puede *Aspirar* cuando se encuentra con [*Sucio*]. ¿Qué debe hacer cuando se encuentra con [*Limpio*]? Si se desplaza a la *Izquierda* se equivoca (siempre) si está en la cuadrícula *A*, y si de desplaza a la *Derecha* se equivoca (siempre) si está en la cuadrícula *B*. Los bucles infinitos son a menudo inevitables para los agentes reactivos simples que operan en algunos entornos parcialmente observables.

ALEATORIO

Salir de los bucles infinitos es posible si los agentes pueden seleccionar sus acciones **aleatoriamente**. Por ejemplo, si un agente aspiradora percibe [*Limpio*], puede lanzar una moneda y elegir entre *Izquierda* y *Derecha*. Es fácil mostrar que el agente se moverá a la otra cuadrícula en una media de dos pasos. Entonces, si la cuadrícula está sucia, la limpiará y la tarea de limpieza se completará. Por tanto, un agente reactivo simple con capacidad para elegir acciones de manera aleatoria puede mejorar los resultados que proporciona un agente reactivo simple determinista.

En la Sección 2.3 se mencionó que un comportamiento aleatorio de un tipo adecuado puede resultar racional en algunos entornos multiagente. En entornos de agentes individuales, el comportamiento aleatorio *no* es normalmente racional. Es un truco útil que ayuda a los agentes reactivos simples en algunas situaciones, pero en la mayoría de los casos se obtendrán mejores resultados con agentes deterministas más sofisticados.

Agentes reactivos basados en modelos

La forma más efectiva que tienen los agentes de manejar la visibilidad parcial es *almacenar información de las partes del mundo que no pueden ver*. O lo que es lo mismo,

ESTADO INTERNO

el agente debe mantener algún tipo de **estado interno** que dependa de la historia percibida y que de ese modo refleje por lo menos alguno de los aspectos no observables del estado actual. Para el problema de los frenos, el estado interno no es demasiado extenso, sólo la fotografía anterior de la cámara, facilitando al agente la detección de dos luces rojas encendiéndose y apagándose simultáneamente a los costados del vehículo. Para

otros aspectos de la conducción, como un cambio de carril, el agente tiene que mantener información de la posición del resto de los coches si no los puede ver.

La actualización de la información de estado interno según pasa el tiempo requiere codificar dos tipos de conocimiento en el programa del agente. Primero, se necesita alguna información acerca de cómo evoluciona el mundo independientemente del agente, por ejemplo, que un coche que está adelantando estará más cerca, detrás, que en un momento inmediatamente anterior. Segundo, se necesita más información sobre cómo afectan al mundo las acciones del agente, por ejemplo, que cuando el agente gire hacia la derecha, el coche gira hacia la derecha o que después de conducir durante cinco minutos hacia el norte en la autopista se avanzan cinco millas hacia el norte a partir del punto en el que se estaba cinco minutos antes. Este conocimiento acerca de «cómo funciona el mundo», tanto si está implementado con un circuito booleano simple o con teorías científicas completas, se denomina **modelo** del mundo. Un agente que utilice este modelo es un **agente basado en modelos**.

AGENTE BASADO EN MODELOS

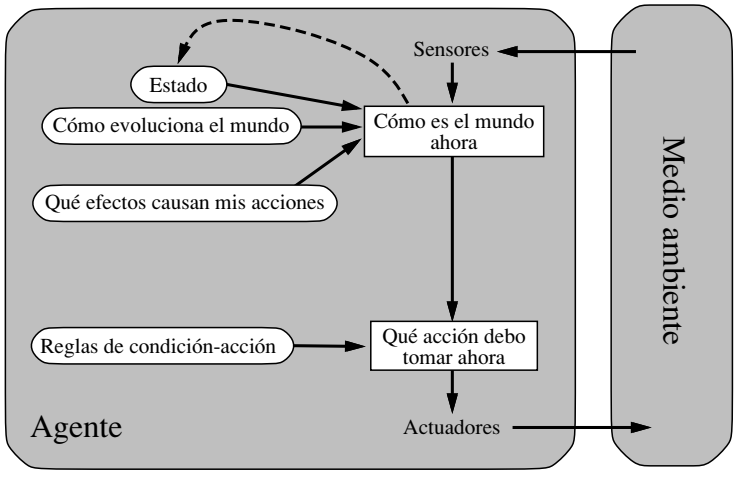


Figura 2.11 Un agente reactivo basado en modelos.

función AGENTE-REACTIVO-CON-ESTADO(percepción) **devuelve** una acción
estático: *estado*, una descripción actual del estado del mundo
reglas, un conjunto de reglas condición-acción
acción, la acción más reciente, inicialmente ninguna

estado ← ACTUALIZAR-ESTADO(*estado, acción, percepción*)
regla ← REGLA-COINCIDENCIA(*estado, reglas*)
acción ← REGLA-ACCIÓN[*regla*]
devolver *acción*

Figura 2.12 Un agente reactivo basado en modelos, que almacena información sobre el estado actual del mundo utilizando un modelo interno. Después selecciona una acción de la misma forma que el agente reactivo.

La Figura 2.11 proporciona la estructura de un agente reactivo simple con estado interno, muestra cómo la percepción actual se combina con el estado interno antiguo para generar la descripción actualizada del estado actual. La Figura 2.12 muestra el programa del agente. La parte interesante es la correspondiente a la función Actualizar-Estado, que es la responsable de la creación de la nueva descripción del estado interno. Además de interpretar la nueva percepción a partir del conocimiento existente sobre el estado, utiliza información relativa a la forma en la que evoluciona el mundo para conocer más sobre las partes del mundo que no están visibles; para ello debe conocer cuál es el efecto de las acciones del agente sobre el estado del mundo. Los Capítulos 10 y 17 ofrecen ejemplos detallados.

Agentes basados en objetivos

El conocimiento sobre el estado actual del mundo no es siempre suficiente para decidir qué hacer. Por ejemplo, en un cruce de carreteras, el taxista puede girar a la izquierda, girar a la derecha o seguir hacia adelante. La decisión correcta depende de dónde quiere ir el taxi. En otras palabras, además de la descripción del estado actual, el agente necesita algún tipo de información sobre su **meta** que describa las situaciones que son deseables, por ejemplo, llegar al destino propuesto por el pasajero. El programa del agente se puede combinar con información sobre los resultados de las acciones posibles (la misma información que se utilizó para actualizar el estado interno en el caso del agente reflexivo) para elegir las acciones que permitan alcanzar el objetivo. La Figura 2.13 muestra la estructura del agente basado en objetivos.

META

En algunas ocasiones, la selección de acciones basadas en objetivos es directa, cuando alcanzar los objetivos es el resultado inmediato de una acción individual. En otras oca-

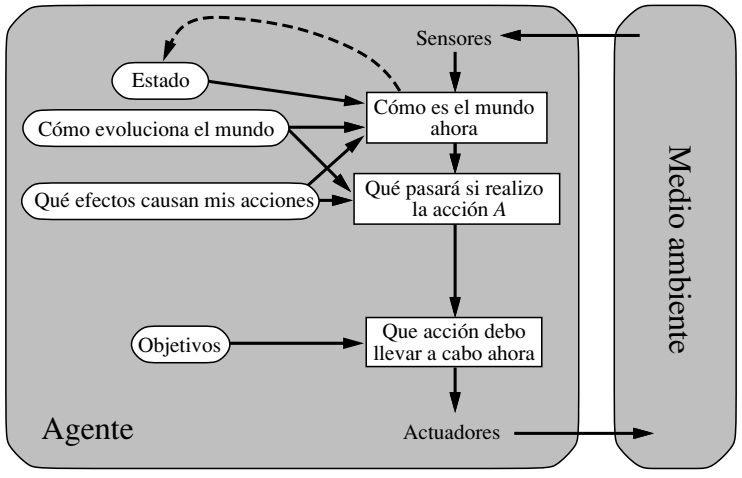


Figura 2.13 Un agente basado en objetivos y basado en modelos, que almacena información del estado del mundo así como del conjunto de objetivos que intenta alcanzar, y que es capaz de seleccionar la acción que eventualmente lo guiará hacia la consecución de sus objetivos.

siones, puede ser más complicado, cuando el agente tiene que considerar secuencias complejas para encontrar el camino que le permita alcanzar el objetivo. **Búsqueda** (Capítulos del 3 al 6) y **planificación** (Capítulos 11 y 12) son los subcampos de la IA centrados en encontrar secuencias de acciones que permitan a los agentes alcanzar sus metas.

Hay que tener en cuenta que la toma de decisiones de este tipo es fundamentalmente diferente de las reglas de condición–acción descritas anteriormente, en las que hay que tener en cuenta consideraciones sobre el futuro (como «¿qué pasará si yo hago esto y esto?» y «¿me hará esto feliz?»). En los diseños de agentes reactivos, esta información no está representada explícitamente, porque las reglas que maneja el agente proyectan directamente las percepciones en las acciones. El agente reactivo frena cuando ve luces de freno. Un agente basado en objetivos, en principio, puede razonar que si el coche que va delante tiene encendidas las luces de frenado, está reduciendo su velocidad. Dada la forma en la que el mundo evoluciona normalmente, la única acción que permite alcanzar la meta de no chocarse con otros coches, es frenar.

Aunque el agente basado en objetivos pueda parecer menos eficiente, es más flexible ya que el conocimiento que soporta su decisión está representado explícitamente y puede modificarse. Si comienza a llover, el agente puede actualizar su conocimiento sobre cómo se comportan los frenos; lo cual implicará que todas las formas de actuar relevantes se alteren automáticamente para adaptarse a las nuevas circunstancias. Para el agente reactivo, por otro lado, se tendrán que rescribir muchas reglas de condición-acción. El comportamiento del agente basado en objetivos puede cambiarse fácilmente para que se dirija a una localización diferente. Las reglas de los agentes reactivos relacionadas con cuándo girar y cuándo seguir recto son válidas sólo para un destino concreto y tienen que modificarse cada vez que el agente se dirija a cualquier otro lugar distinto.

Agentes basados en utilidad

Las metas por sí solas no son realmente suficientes para generar comportamiento de gran calidad en la mayoría de los entornos. Por ejemplo, hay muchas secuencias de acciones que llevarán al taxi a su destino (y por tanto a alcanzar su objetivo), pero algunas son más rápidas, más seguras, más fiables, o más baratas que otras. Las metas sólo proporcionan una cruda distinción binaria entre los estados de «felicidad» y «tristeza», mientras que una medida de eficiencia más general debería permitir una comparación entre estados del mundo diferentes de acuerdo al nivel exacto de felicidad que el agente alcance cuando se llegue a un estado u otro. Como el término «felicidad» no suena muy científico, la terminología tradicional utilizada en estos casos para indicar que se prefiere un estado del mundo a otro es que un estado tiene más **utilidad** que otro para el agente[9].

UTILIDAD

FUNCIÓN DE UTILIDAD

Una **función de utilidad** proyecta un estado (o una secuencia de estados) en un número real, que representa un nivel de felicidad. La definición completa de una función de utilidad permite tomar decisiones racionales en dos tipos de casos en los que las metas son inadecuadas. Primero, cuando haya objetivos conflictivos, y sólo se puedan al-

[9] La palabra «utilidad» aquí se refiere a «la cualidad de ser útil».

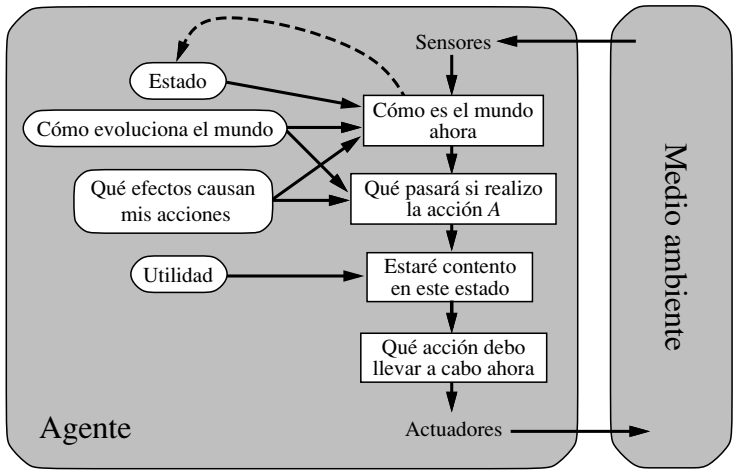


Figura 2.14 Un agente basado en utilidad y basado en modelos. Utiliza un modelo del mundo, junto con una función de utilidad que calcula sus preferencias entre los estados del mundo. Después selecciona la acción que le lleve a alcanzar la mayor utilidad esperada, que se calcula haciendo la media de todos los estados resultantes posibles, ponderado con la probabilidad del resultado.

canzar algunos de ellos (por ejemplo, velocidad y seguridad), la función de utilidad determina el equilibrio adecuado. Segundo, cuando haya varios objetivos por los que se pueda guiar el agente, y ninguno de ellos se pueda alcanzar con certeza, la utilidad proporciona un mecanismo para ponderar la probabilidad de éxito en función de la importancia de los objetivos.

En el Capítulo 16, se mostrará cómo cualquier agente racional debe comportarse *como si* tuviese una función de utilidad cuyo valor esperado tiene que maximizar. Por tanto, un agente que posea una función de utilidad *explícita* puede tomar decisiones racionales, y lo puede hacer con la ayuda de un algoritmo de propósito general que no dependa de la función específica de utilidad a maximizar. De esta forma, la definición «global» de racionalidad (identificando como racionales aquellas funciones de los agentes que proporcionan el mayor rendimiento) se transforma en una restricción «local» en el diseño de agentes racionales que se puede expresar con un simple programa.

La Figura 2.14 muestra la estructura de un agente basado en utilidad. En la Parte IV aparecen programas de agentes basados en utilidad, donde se presentan agentes que toman decisiones y que deben trabajar con la incertidumbre inherente a los entornos parcialmente observables.

Agentes que aprenden

Se han descrito programas para agentes que poseen varios métodos para seleccionar acciones. Hasta ahora no se ha explicado cómo *poner en marcha* estos programas de agentes. Turing (1950), en su temprano y famoso artículo, consideró la idea de programar sus máquinas inteligentes a mano. Estimó cuánto tiempo podía llevar y concluyó que «Se-

ría deseable utilizar algún método más rápido». El método que propone es construir máquinas que aprendan y después enseñarlas. En muchas áreas de IA, éste es ahora el método más adecuado para crear sistemas novedosos. El aprendizaje tiene otras ventajas, como se ha explicado anteriormente: permite que el agente opere en medios inicialmente desconocidos y que sea más competente que si sólo utilizase un conocimiento inicial. En esta sección, se introducen brevemente las principales ideas en las que se basan los agentes que aprenden. En casi todos los capítulos de este libro se comentan las posibilidades y métodos de aprendizaje de tipos de agentes concretos. La Parte VI profundiza más en los algoritmos de aprendizaje en sí mismos.

ELEMENTO DE APRENDIZAJE

ELEMENTO DE ACTUACIÓN

CRÍTICA

Un agente que aprende se puede dividir en cuatro componentes conceptuales, tal y como se muestra en la Figura 2.15. La distinción más importante entre el **elemento de aprendizaje** y el **elemento de actuación** es que el primero está responsabilizado de hacer mejoras y el segundo se responsabiliza de la selección de acciones externas. El elemento de actuación es lo que anteriormente se había considerado como el agente completo: recibe estímulos y determina las acciones a realizar. El elemento de aprendizaje se realimenta con las **críticas** sobre la actuación del agente y determina cómo se debe modificar el elemento de actuación para proporcionar mejores resultados en el futuro.

El diseño del elemento de aprendizaje depende mucho del diseño del elemento de actuación. Cuando se intenta diseñar un agente que tenga capacidad de aprender, la primera cuestión a solucionar no es ¿cómo se puede enseñar a aprender?, sino ¿qué tipo de elemento de actuación necesita el agente para llevar a cabo su objetivo, cuando haya aprendido cómo hacerlo? Dado un diseño para un agente, se pueden construir los mecanismos de aprendizaje necesarios para mejorar cada una de las partes del agente.

La crítica indica al elemento de aprendizaje qué tal lo está haciendo el agente con respecto a un nivel de actuación fijo. La crítica es necesaria porque las percepciones por sí mismas no prevén una indicación del éxito del agente. Por ejemplo, un programa de

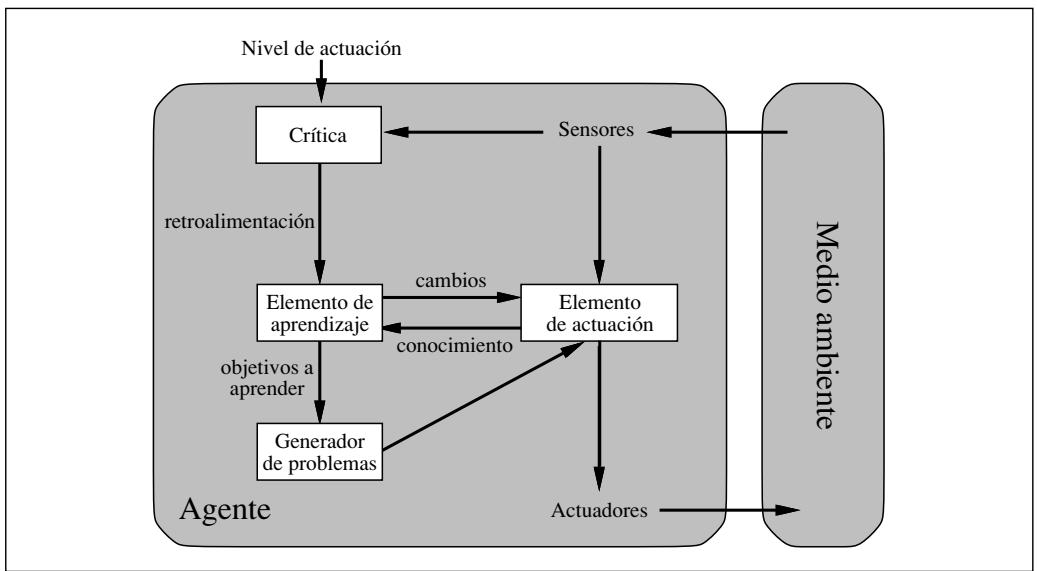


Figura 2.15 Modelo general para agentes que aprenden.

ajedrez puede recibir una percepción indicando que ha dado jaque mate a su oponente, pero necesita tener un nivel de actuación que le indique que ello es bueno; la percepción por sí misma no lo indica. Es por tanto muy importante fijar el nivel de actuación. Conceptualmente, se debe tratar con él como si estuviese fuera del agente, ya que éste no debe modificarlo para satisfacer su propio interés.

GENERADOR DE PROBLEMAS

El último componente del agente con capacidad de aprendizaje es el **generador de problemas**. Es responsable de sugerir acciones que lo guiarán hacia experiencias nuevas e informativas. Lo interesante es que si el elemento de actuación sigue su camino, puede continuar llevando a cabo las acciones que sean mejores, dado su conocimiento. Pero si el agente está dispuesto a explorar un poco, y llevar a cabo algunas acciones que no sean totalmente óptimas a corto plazo, puede descubrir acciones mejores a largo plazo. El trabajo del generador de problemas es sugerir estas acciones exploratorias. Esto es lo que los científicos hacen cuando llevan a cabo experimentos. Galileo no pensaba que tirar piedras desde lo alto de una torre en Pisa tenía un valor por sí mismo. Él no trataba de romper piedras ni de cambiar la forma de pensar de transeúntes desafortunados que paseaban por el lugar. Su intención era adaptar su propia mente, para identificar una teoría que definiese mejor el movimiento de los objetos.

Para concretar el diseño total, se puede volver a utilizar el ejemplo del taxi automatizado. El elemento de actuación consiste en la colección de conocimientos y procedimientos que tiene el taxi para seleccionar sus acciones de conducción. El taxi se pone en marcha y circula utilizando este elemento de actuación. La crítica observa el mundo y proporciona información al elemento de aprendizaje. Por ejemplo, después de que el taxi se sitúe tres carriles hacia la izquierda de forma rápida, la crítica observa el lenguaje escandaloso que utilizan otros conductores. A partir de esta experiencia, el elemento de aprendizaje es capaz de formular una regla que indica que ésta fue una mala acción, y el elemento de actuación se modifica incorporando la nueva regla. El generador de problemas debe identificar ciertas áreas de comportamiento que deban mejorarse y sugerir experimentos, como probar los frenos en carreteras con tipos diferentes de superficies y bajo condiciones distintas.

El elemento de aprendizaje puede hacer cambios en cualquiera de los componentes de «conocimiento» que se muestran en los diagramas de agente (Figuras 2.9, 2.11, 2.13, y 2.14). Los casos más simples incluyen el aprendizaje directo a partir de la secuencia percibida. La observación de pares de estados sucesivos del entorno puede permitir que el agente aprenda «cómo evoluciona el mundo», y la observación de los resultados de sus acciones puede permitir que el agente aprenda «qué hacen sus acciones». Por ejemplo, si el taxi ejerce una cierta presión sobre los frenos cuando está circulando por una carretera mojada, acto seguido conocerá cómo decelera el coche. Claramente, estas dos tareas de aprendizaje son más difíciles si sólo existe una vista parcial del medio.

Las formas de aprendizaje mostradas en los párrafos precedentes no necesitan el acceso a niveles de actuación externo, de alguna forma, el nivel es el que se utiliza universalmente para hacer pronósticos de acuerdo con la experimentación. La situación es ligeramente más compleja para un agente basado en utilidad que desee adquirir información para crear su función de utilidad. Por ejemplo, se supone que el agente conductor del taxi no recibe propina de los pasajeros que han recorrido un trayecto de forma incómoda debido a una mala conducción. El nivel de actuación externo debe informar

al agente de que la pérdida de propinas tiene una contribución negativa en su nivel de actuación medio; entonces el agente puede aprender que «maniobras violentas no contribuyen a su propia utilidad». De alguna manera, el nivel de actuación identifica parte de las percepciones entrantes como **recompensas** (o **penalizaciones**) que generan una respuesta directa en la calidad del comportamiento del agente. Niveles de actuación integrados como el dolor y el hambre en animales se pueden enmarcar en este contexto. El Capítulo 21 discute estos asuntos.

En resumen, los agentes tienen una gran variedad de componentes, y estos componentes se pueden representar de muchas formas en los programas de agentes, por lo que, parece haber una gran variedad de métodos de aprendizaje. Existe, sin embargo, una visión unificada sobre un tema fundamental. El aprendizaje en el campo de los agentes inteligentes puede definirse como el proceso de modificación de cada componente del agente, lo cual permite a cada componente comportarse más en consonancia con la información que se recibe, lo que por tanto permite mejorar el nivel medio de actuación del agente.

2.5 Resumen

En este capítulo se ha realizado un recorrido rápido por el campo de la IA, que se ha presentado como la ciencia del diseño de los agentes. Los puntos más importantes a tener en cuenta son:

- Un **agente** es algo que percibe y actúa en un medio. La **función del agente** para un agente especifica la acción que debe realizar un agente como respuesta a cualquier secuencia percibida.
- La **medida de rendimiento** evalúa el comportamiento del agente en un medio. Un **agente racional** actúa con la intención de maximizar el valor esperado de la medida de rendimiento, dada la secuencia de percepciones que ha observado hasta el momento.
- Las especificaciones del **entorno de trabajo** incluyen la medida de rendimiento, el medio externo, los actuadores y los sensores. El primer paso en el diseño de un agente debe ser siempre la especificación, tan completa como sea posible, del entorno de trabajo.
- El entorno de trabajo varía según distintos parámetros. Pueden ser total o parcialmente visibles, deterministas o estocásticos, episódicos o secuenciales, estáticos o dinámicos, discretos o continuos, y formados por un único agente o por varios agentes.
- El **programa del agente** implementa la función del agente. Existe una gran variedad de diseños de programas de agentes, y reflejan el tipo de información que se hace explícita y se utiliza en el proceso de decisión. Los diseños varían en eficiencia, solidez y flexibilidad. El diseño apropiado del programa del agente depende en gran medida de la naturaleza del medio.
- **Los gentes reactivos simples** responden directamente a las percepciones, mientras que los **agentes reactivos basados en modelos** mantienen un estado interno

que les permite seguir el rastro de aspectos del mundo que no son evidentes según las percepciones actuales. Los agentes basados en objetivos actúan con la intención de alcanzar sus metas, y los agentes basados en utilidad intentan maximizar su «felicidad» deseada.

- Todos los agentes pueden mejorar su eficacia con la ayuda de mecanismos de **aprendizaje**.



NOTAS BIBLIOGRÁFICAS E HISTÓRICAS

El papel central de la acción en la inteligencia (la noción del razonamiento práctico) se remonta por lo menos a la obra *Nicomachean Ethics* de Aristóteles. McCarthy (1958) trató también el tema del razonamiento práctico en su influyente artículo *Programs with Common Sense*. Los campos de la robótica y la teoría de control tienen interés, por su propia naturaleza, en la construcción de agentes físicos. El concepto de un **controlador**, en el ámbito de la teoría de control, es idéntico al de un agente en IA. Quizá sorprendentemente, la IA se ha concentrado durante la mayor parte de su historia en componentes aislados de agentes (sistemas que responden a preguntas, demostración de teoremas, sistemas de visión, y demás) en vez de en agentes completos. La discusión sobre agentes que se presenta en el libro de Genesereth y Nilsson (1987) fue una influyente excepción. El concepto de agente en sí está aceptado ampliamente ahora en el campo y es un tema central en libros recientes (Poole *et al*., 1998; Nilsson, 1998).

CONTROLADOR

El Capítulo 1 muestra las raíces del concepto de racionalidad en la Filosofía y la Economía. En la IA, el concepto tuvo un interés periférico hasta mediados de los 80, donde comenzó a suscitar muchas discusiones sobre los propios fundamentos técnicos del campo. Un artículo de Jon Doyle (1983) predijo que el diseño de agentes racionales podría llegar a ser la misión central de la IA, mientras otras áreas populares podrían separarse dando lugar a nuevas disciplinas.

Es muy importante tener muy en cuenta las propiedades del medio y sus consecuencias cuando se realiza el diseño de los agentes racionales ya que forma parte de la tradición ligada a la teoría de control [por ejemplo los sistemas de control clásicos (Dorf y Bishop, 1999) manejan medios deterministas y totalmente observables; el control óptimo estocástico (Kumar y Varaiya, 1986) maneja medios parcialmente observables y estocásticos y un control híbrido (Henzinger y Sastry, 1998) maneja entornos que contienen elementos discretos y continuos]. La distinción entre entornos totalmente y parcialmente observables es también central en la literatura sobre **programación dinámica** desarrollada en el campo de la investigación operativa (Puterman, 1994), como se comentará en el Capítulo 17.

Los agentes reactivos fueron los primeros modelos para psicólogos conductistas como Skinner (1953), que intentó reducir la psicología de los organismos estrictamente a correspondencias entrada/salida o estímulo/respuesta. La evolución del behaviourismo hacia el funcionalismo en el campo de la psicología, que estuvo, al menos de forma parcial, dirigida por la aplicación de la metáfora del computador a los agentes (Putnam, 1960; Lewis, 1966) introdujo el estado interno del agente en el nuevo escenario. La mayor par-

te del trabajo realizado en el campo de la IA considera que los agentes reactivos puros con estado interno son demasiado simples para ser muy influyentes, pero los trabajos de Rosenschein (1985) y Brooks (1986) cuestionan esta hipótesis (*véase* el Capítulo 25). En los últimos años, se ha trabajado intensamente para encontrar algoritmos eficientes capaces de hacer un buen seguimiento de entornos complejos (Hamscher *et al*., 1992). El programa del Agente Remoto que controla la nave espacial Deep Space One (descrito en la página 27) es un admirable ejemplo concreto (Muscettola *et al*., 1998; Jonsson *et al*., 2000).

Los agentes basados en objetivos están presentes tanto en las referencias de Aristóteles sobre el razonamiento práctico como en los primeros artículos de McCarthy sobre IA lógica. El robot Shakey (Fikes y Nilsson, 1971; Nilsson, 1984) fue el primer robot construido como un agente basado en objetivos. El análisis lógico completo de un agente basado en objetivos aparece en Genesereth y Nilsson (1987), y Shoham (1993) ha desarrollado una metodología de programación basada en objetivos llamada programación orientada a agentes.

La perspectiva orientada a objetivos también predomina en la psicología cognitiva tradicional, concretamente en el área de la resolución de problemas, como se muestra tanto en el influyente *Human Problem Solving* (Newell y Simon, 1972) como en los últimos trabajos de Newell (1990). Los objetivos, posteriormente definidos como *deseos* (generales) y las *intenciones* (perseguidas en un momento dado), son fundamentales en la teoría de agentes desarrollada por Bratman (1987). Esta teoría ha sido muy influyente tanto en el entendimiento del lenguaje natural como en los sistemas multiagente.

Horvitz *et al*. (1988) sugieren específicamente el uso de la maximización de la utilidad esperada concebida racionalmente como la base de la IA. El texto de Pearl (1988) fue el primero en IA que cubrió las teorías de la probabilidad y la utilidad en profundidad; su exposición de métodos prácticos de razonamiento y toma de decisiones con incertidumbre fue, posiblemente, el factor individual que más influyó en el desarrollo de los agentes basados en utilidad en los 90 (*véase* la Parte V).

El diseño general de agentes que aprenden representado en la Figura 2.15 es un clásico de la literatura sobre aprendizaje automático (Buchanan *et al*., 1978; Mitchell, 1997). Ejemplos de diseños, implementados en programas, se remontan, como poco, hasta los programas que aprendían a jugar al ajedrez de Arthur Samuel (1959, 1967). La Parte VI está dedicada al estudio en profundidad de los agentes que aprenden.

El interés en los agentes y en el diseño de agentes ha crecido rápidamente en los últimos años, en parte por la expansión de Internet y la necesidad observada de desarrollar **softbots** (robots *software*) automáticos y móviles (Etzioni y Weld, 1994). Artículos relevantes pueden encontrarse en *Readings in Agents* (Huhns y Singh, 1998) y *Fundations of Rational Agency* (Wooldridge y Rao, 1999). *Multiagent Systems* (Weiss, 1999) proporciona una base sólida para muchos aspectos del diseño de agentes. Conferencias dedicadas a agentes incluyen la International Conference on Autonomous Agents, la International Workshop on Agent Theories, Architectures, and Languages, y la International Conference on Multiagent Systems. Finalmente, *Dung Beetle Ecology* (Hanski y Cambefort, 1991) proporciona gran cantidad de información interesante sobre el comportamiento de los escarabajos estercoleros.



EJERCICIOS

2.1 Defina con sus propias palabras los siguientes términos: agente, función de agente, programa de agente, racionalidad, autonomía, agente reactivo, agente basado en modelo, agente basado en objetivos, agente basado en utilidad, agente que aprende.

2.2 Tanto la medida de rendimiento como la función de utilidad miden la eficiencia del agente. Explique la diferencia entre los dos conceptos.

2.3 Este ejercicio explora las diferencias entre las funciones de los agentes y los programas de los agentes.

- ***a***) ¿Puede haber más de un programa de agente que implemente una función de agente dada? Proponga un ejemplo, o muestre por qué una no es posible.
- ***b***) ¿Hay funciones de agente que no se pueden implementar con algún programa de agente?
- ***c***) Dada una arquitectura máquina, ¿implementa cada programa de agente exactamente una función de agente?
- ***d***) Dada una arquitectura con n bits de almacenamiento, ¿cuántos posibles programas de agente diferentes puede almacenar?

2.4 Examínese ahora la racionalidad de varias funciones de agentes aspiradora.

- ***a***) Muestre que la función de agente aspiradora descrita en la Figura 2.3 es realmente racional bajo la hipótesis presentada en la página 36.
- ***b***) Describa una función para un agente racional cuya medida de rendimiento modificada deduzca un punto por cada movimiento. ¿Requiere el correspondiente programa de agente estado interno?
- ***c***) Discuta posibles diseños de agentes para los casos en los que las cuadrículas limpias puedan ensuciarse y la geografía del medio sea desconocida. ¿Tiene sentido que el agente aprenda de su experiencia en estos casos? ¿Si es así, qué debe aprender?

2.5 Identifique la descripción REAS que define el entorno de trabajo para cada uno de los siguientes agentes:

- ***a***) Robot que juega al fútbol;
- ***b***) Agente para comprar libros en Internet;
- ***c***) Explorador autónomo de Marte;
- ***d***) Asistente matemático para la demostración de teoremas.

2.6 Para cada uno de los tipos de agente enumerados en el Ejercicio 2.5, caracterice el medio de acuerdo con las propiedades dadas en la Sección 2.3, y seleccione un diseño de agente adecuado.



Los siguientes ejercicios están relacionados con la implementación de entornos y agentes para el mundo de la aspiradora.

2.7 Implemente un simulador que determine la medida de rendimiento para el entorno del mundo de la aspiradora descrito en la Figura 2.2 y especificado en la página 36. La implementación debe ser modular, de forma que los sensores, actuadores, y las características del entorno (tamaño, forma, localización de la suciedad, etc.) puedan modificar-

se fácilmente. (*Nota:* hay implementaciones disponibles en el repositorio de Internet que pueden ayudar a decidir qué lenguaje de programación y sistema operativo seleccionar).

2.8 Implemente un agente reactivo simple para el entorno de la aspiradora del Ejercicio 2.7. Ejecute el simulador del entorno con este agente para todas las configuraciones iniciales posibles de suciedad y posiciones del agente. Almacene la puntuación de la actuación del agente para cada configuración y la puntuación media global.

2.9 Considere una versión modificada del entorno de la aspiradora del Ejercicio 2.7, en el que se penalice al agente con un punto en cada movimiento.

- ***a***) ¿Puede un agente reactivo simple ser perfectamente racional en este medio? Explíquese.
- ***b***) ¿Qué sucedería con un agente reactivo con estado? Diseñe este agente.
- ***c***) ¿Cómo se responderían las preguntas **a** y **b** si las percepciones proporcionan al agente información sobre el nivel de suciedad/limpieza de todas las cuadrículas del entorno?

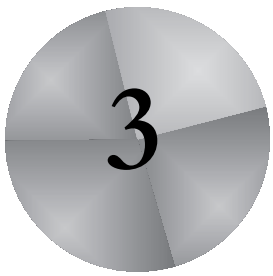
2.10 Considere una versión modificada del entorno de la aspiradora del Ejercicio 2.7, en el que la geografía del entorno (su extensión, límites, y obstáculos) sea desconocida, así como, la disposición inicial de la suciedad. (El agente puede ir hacia *arriba*, *abajo,* así como, hacia la *derecha* y a la *izquierda*.)

- ***a***) ¿Puede un agente reactivo simple ser perfectamente racional en este medio? Explíquese.
- ***b***) ¿Puede un agente reactivo simple con una función de agente aleatoria superar a un agente reactivo simple? Diseñe un agente de este tipo y medir su rendimiento en varios medios.
- ***c***) ¿Se puede diseñar un entorno en el que el agente con la función aleatoria obtenga una actuación muy pobre? Muestre los resultados.
- ***d***) ¿Puede un agente reactivo con estado mejorar los resultados de un agente reactivo simple? Diseñe un agente de este tipo y medir su eficiencia en distintos medios. ¿Se puede diseñar un agente racional de este tipo?

2.11 Repítase el Ejercicio 2.10 para el caso en el que el sensor de localización sea reemplazado por un sensor «de golpes» que detecte si el agente golpea un obstáculo o si se sale fuera de los límites del entorno. Supóngase que el sensor de golpes deja de funcionar. ¿Cómo debe comportarse el agente?

2.12 Los entornos de la aspiradora en los ejercicios anteriores han sido todos deterministas. Discuta posibles programas de agentes para cada una de las siguientes versiones estocásticas:

- ***a***) Ley de Murphy: el 25 por ciento del tiempo, la acción de *Aspirar* falla en la limpieza del suelo si está sucio y deposita suciedad en el suelo si el suelo está limpio. ¿Cómo se ve afectado el agente si el sensor de suciedad da una respuesta incorrecta el diez por ciento de las veces?
- ***b***) Niño pequeño: en cada lapso de tiempo, cada recuadro limpio tiene un diez por ciento de posibilidad de ensuciarse. ¿Puede identificar un diseño para un agente racional en este caso?



3 Resolver problemas mediante búsqueda

En donde veremos cómo un agente puede encontrar una secuencia de acciones que alcance sus objetivos, cuando ninguna acción simple lo hará.

Los agentes más simples discutidos en el Capítulo 2 fueron los agentes reactivos, los cuales basan sus acciones en una aplicación directa desde los estados a las acciones. Tales agentes no pueden funcionar bien en entornos en los que esta aplicación sea demasiado grande para almacenarla y que tarde mucho en aprenderla. Por otra parte, los agentes basados en objetivos pueden tener éxito considerando las acciones futuras y lo deseable de sus resultados.

AGENTE RESOLVENTE-PROBLEMAS

Este capítulo describe una clase de agente basado en objetivos llamado **agente resolvente-problemas**. Los agentes resolventes-problemas deciden qué hacer para encontrar secuencias de acciones que conduzcan a los estados deseables. Comenzamos definiendo con precisión los elementos que constituyen el «problema» y su «solución», y daremos diferentes ejemplos para ilustrar estas definiciones. Entonces, describimos diferentes algoritmos de propósito general que podamos utilizar para resolver estos problemas y así comparar las ventajas de cada algoritmo. Los algoritmos son **no informados**, en el sentido que no dan información sobre el problema salvo su definición. El Capítulo 4 se ocupa de los algoritmos de búsqueda **informada**, los que tengan cierta idea de dónde buscar las soluciones.

Este capítulo utiliza los conceptos de análisis de algoritmos. Los lectores no familiarizados con los conceptos de complejidad asintótica (es decir, notación $O()$) y la NP completitud, debería consultar el Apéndice A.

3.1 Agentes resolventes-problemas

Se supone que los agentes inteligentes deben maximizar su medida de rendimiento. Como mencionamos en el Capítulo 2, esto puede simplificarse algunas veces si el agente puede

elegir **un objetivo** y trata de satisfacerlo. Primero miraremos el porqué y cómo puede hacerlo.

Imagine un agente en la ciudad de Arad, Rumanía, disfrutando de un viaje de vacaciones. La medida de rendimiento del agente contiene muchos factores: desea mejorar su bronceado, mejorar su rumano, tomar fotos, disfrutar de la vida nocturna, evitar resacas, etcétera. El problema de decisión es complejo implicando muchos elementos y por eso, lee cuidadosamente las guías de viaje. Ahora, supongamos que el agente tiene un billete no reembolsable para volar a Bucarest al día siguiente. En este caso, tiene sentido que el agente elija **el objetivo** de conseguir Bucarest. Las acciones que no alcanzan Bucarest se pueden rechazar sin más consideraciones y el problema de decisión del agente se simplifica enormemente. Los objetivos ayudan a organizar su comportamiento limitando las metas que intenta alcanzar el agente. El primer paso para solucionar un problema es la **formulación del objetivo**, basado en la situación actual y la medida de rendimiento del agente.

FORMULACIÓN DEL OBJETIVO

Consideraremos un objetivo como un conjunto de estados del mundo (exactamente aquellos estados que satisfacen el objetivo). La tarea del agente es encontrar qué secuencia de acciones permite obtener un estado objetivo. Para esto, necesitamos decidir qué acciones y estados considerar. Si se utilizaran acciones del tipo «mueve el pie izquierdo hacia delante» o «gira el volante un grado a la izquierda», probablemente el agente nunca encontraría la salida del aparcamiento, no digamos por tanto llegar a Bucarest, porque a ese nivel de detalle existe demasiada incertidumbre en el mundo y habría demasiados pasos en una solución. Dado un objetivo, la **formulación del problema** es el proceso de decidir qué acciones y estados tenemos que considerar. Discutiremos con más detalle este proceso. Por ahora, suponemos que el agente considerará acciones del tipo conducir de una ciudad grande a otra. Consideraremos los estados que corresponden a estar en una ciudad[1] determinada.

FORMULACIÓN DEL PROBLEMA

Ahora, nuestro agente ha adoptado el objetivo de conducir a Bucarest, y considera a dónde ir desde Arad. Existen tres carreteras desde Arad, una hacia Sibiu, una a Timisoara, y una a Zerind. Ninguna de éstas alcanza el objetivo, y, a menos que el agente este familiarizado con la geografía de Rumanía, no sabría qué carretera seguir[2]. En otras palabras, el agente no sabrá cuál de las posibles acciones es mejor, porque no conoce lo suficiente los estados que resultan al tomar cada acción. Si el agente no tiene conocimiento adicional, entonces estará en un callejón sin salida. Lo mejor que puede hacer es escoger al azar una de las acciones.

Pero, supongamos que el agente tiene un mapa de Rumanía, en papel o en su memoria. El propósito del mapa es dotar al agente de información sobre los estados en los que podría encontrarse, así como las acciones que puede tomar. El agente puede usar esta información para considerar los siguientes estados de un viaje hipotético por cada una de las tres ciudades, intentando encontrar un viaje que llegue a Bucarest. Una vez que

---

[1] Observe que cada uno de estos «estados» se corresponde realmente a un conjunto de estados del mundo, porque un estado del mundo real especifica todos los aspectos de realidad. Es importante mantener en mente la distinción entre estados del problema a resolver y los estados del mundo.

[2] Suponemos que la mayoría de los lectores están en la misma situación y pueden fácilmente imaginarse cómo de desorientado está nuestro agente. Pedimos disculpas a los lectores rumanos quienes no pueden aprovecharse de este recurso pedagógico.

ha encontrado un camino en el mapa desde Arad a Bucarest, puede alcanzar su objetivo, tomando las acciones de conducir que corresponden con los tramos del viaje. En general, *un agente con distintas opciones inmediatas de valores desconocidos puede decidir qué hacer, examinando las diferentes secuencias posibles de acciones que le conduzcan a estados de valores conocidos, y entonces escoger la mejor secuencia*.



BÚSQUEDA

SOLUCIÓN

EJECUCIÓN

Este proceso de hallar esta secuencia se llama **búsqueda**. Un algoritmo de búsqueda toma como entrada un problema y devuelve una **solución** de la forma secuencia de acciones. Una vez que encontramos una solución, se procede a ejecutar las acciones que ésta recomienda. Esta es la llamada fase de **ejecución**. Así, tenemos un diseño simple de un agente «formular, buscar, ejecutar», como se muestra en la Figura 3.1. Después de formular un objetivo y un problema a resolver, el agente llama al procedimiento de búsqueda para resolverlo. Entonces, usa la solución para guiar sus acciones, haciendo lo que la solución le indica como siguiente paso a hacer —generalmente, primera acción de la secuencia— y procede a eliminar este paso de la secuencia. Una vez ejecutada la solución, el agente formula un nuevo objetivo.

Primero describimos el proceso de formulación del problema, y después dedicaremos la última parte del capítulo a diversos algoritmos para la función BÚSQUEDA. En este capítulo no discutiremos las funciones ACTUALIZAR-ESTADO y FORMULAR-OBJETIVO.

Antes de entrar en detalles, hagamos una breve pausa para ver dónde encajan los agentes resolventes de problemas en la discusión de agentes y entornos del Capítulo 2. El agente diseñado en la Figura 3.1 supone que el entorno es **estático**, porque la formulación y búsqueda del problema se hace sin prestar atención a cualquier cambio que puede ocurrir en el entorno. El agente diseñado también supone que se conoce el estado inicial; conocerlo es fácil si el entorno es **observable**. La idea de enumerar «las lí-

```
función AGENTE-SENCILLO-RESOLVENTE-PROBLEMAS(percepción) devuelve una acción
  entradas: percepción, una percepción
  estático: sec, una secuencia de acciones, vacía inicialmente
            estado, una descripción del estado actual del mundo
            objetivo, un objetivo, inicialmente nulo
            problema, una formulación del problema

  estado ← ACTUALIZAR-ESTADO(estado, percepción)
  si sec está vacía entonces hacer
    objetivo ← FORMULAR-OBJETIVO(estado)
    problema ← FORMULAR-PROBLEMA(estado,objetivo)
    sec ← BÚSQUEDA(problema)
  acción ← PRIMERO(secuencia)
  sec ← RESTO(secuencia)
  devolver acción
```

Figura 3.1 Un sencillo agente resolvente de problemas. Primero formula un objetivo y un problema, busca una secuencia de acciones que deberían resolver el problema, y entonces ejecuta las acciones una cada vez. Cuando se ha completado, formula otro objetivo y comienza de nuevo. Notemos que cuando se ejecuta la secuencia, se ignoran sus percepciones: se supone que la solución encontrada trabajará bien.

neas de acción alternativas» supone que el entorno puede verse como **discreto**. Finalmente, y más importante, el agente diseñado supone que el entorno es **determinista**. Las soluciones a los problemas son simples secuencias de acciones, así que no pueden manejar ningún acontecimiento inesperado; además, las soluciones se ejecutan sin prestar atención a las percepciones. Los agentes que realizan sus planes con los ojos cerrados, por así decirlo, deben estar absolutamente seguros de lo que pasa (los teóricos de control lo llaman sistema de **lazo abierto**, porque ignorar las percepciones rompe el lazo entre el agente y el entorno). Todas estas suposiciones significan que tratamos con las clases más fáciles de entornos, razón por la que este capítulo aparece tan pronto en el libro. La Sección 3.6 echa una breve ojeada sobre lo que sucede cuando relajamos las suposiciones de observancia y de determinismo. Los Capítulos 12 y 17 entran más en profundidad.

LAZO ABIERTO

Problemas y soluciones bien definidos

PROBLEMA

Un **problema** puede definirse, formalmente, por cuatro componentes:

ESTADO INICIAL

- El **estado inicial** en el que comienza el agente. Por ejemplo, el estado inicial para nuestro agente en Rumanía se describe como *En*(*Arad*).

FUNCIÓN SUCESOR

- Una descripción de las posibles **acciones** disponibles por el agente. La formulación[3] más común utiliza una **función sucesor**. Dado un estado particular x, SUCESOR-FN(x) devuelve un conjunto de pares ordenados $\langle$*acción, sucesor*$\rangle$, donde cada acción es una de las acciones legales en el estado x y cada sucesor es un estado que puede alcanzarse desde x, aplicando la acción. Por ejemplo, desde el estado *En(Arad)*, la función sucesor para el problema de Rumanía devolverá

  $$\{\langle Ir(Sibiu), En(Sibiu)\rangle, \langle Ir(Timisoara), En(Timisoara)\rangle, \langle Ir(Zerind), En(Zerind)\rangle\}$$

ESPACIO DE ESTADOS

  Implícitamente el estado inicial y la función sucesor definen el **espacio de estados** del problema (el conjunto de todos los estados alcanzables desde el estado inicial). El espacio de estados forma un grafo en el cual los nodos son estados y los arcos entre los nodos son acciones. (El mapa de Rumanía que se muestra en la Figura 3.2 puede interpretarse como un grafo del espacio de estados si vemos cada carretera como dos acciones de conducir, una en cada dirección). Un **camino** en el espacio de estados es una secuencia de estados conectados por una secuencia de acciones.

CAMINO

TEST OBJETIVO

- El **test objetivo**, el cual determina si un estado es un estado objetivo. Algunas veces existe un conjunto explícito de posibles estados objetivo, y el test simplemente comprueba si el estado es uno de ellos. El objetivo del agente en Rumanía es el conjunto {*En*(*Bucarest*)}. Algunas veces el objetivo se especifica como una propiedad abstracta más que como un conjunto de estados enumerados explícitamente. Por ejemplo, en el ajedrez, el objetivo es alcanzar un estado llamado «jaque mate», donde el rey del oponente es atacado y no tiene escapatoria.

[3] Una formulación alternativa utiliza un conjunto de **operadores** que pueden aplicarse a un estado para generar así los sucesores.

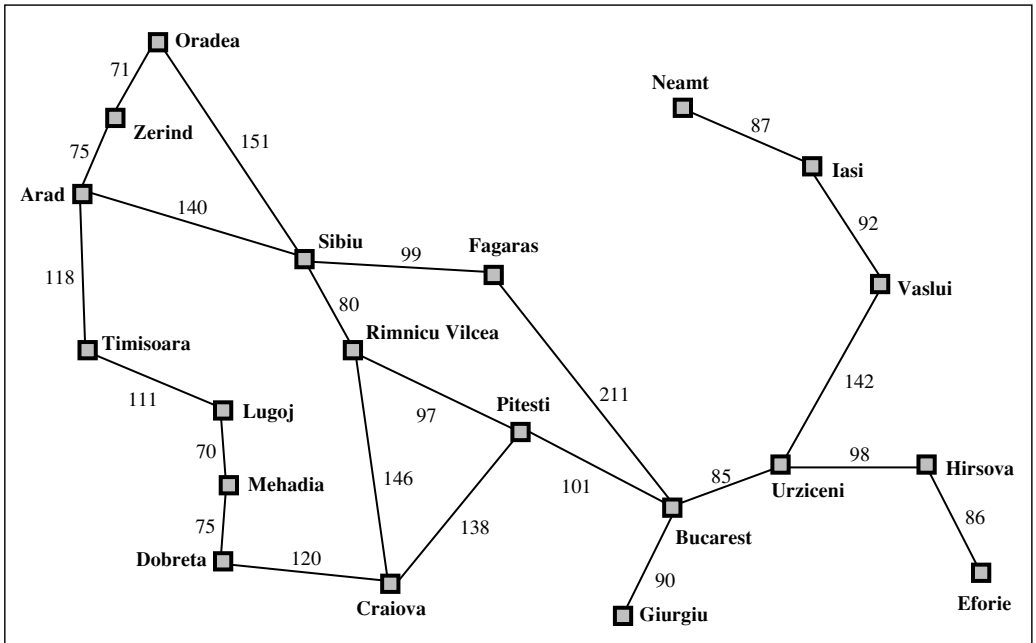


Figura 3.2 Un mapa de carreteras simplificado de parte de Rumanía.

COSTO DEL CAMINO

- Una función **costo del camino** que asigna un costo numérico a cada camino. El agente resolvente de problemas elige una función costo que refleje nuestra medida de rendimiento. Para el agente que intenta llegar a Bucarest, el tiempo es esencial, así que el costo del camino puede describirse como su longitud en kilómetros. En este capítulo, suponemos que el costo del camino puede describirse como la suma de los costos de las acciones individuales a lo largo del camino. El **costo individual** de una acción a que va desde un estado x al estado y se denota por $c(x,a,y)$. Los costos individuales para Rumanía se muestran en la Figura 3.2 como las distancias de las carreteras. Suponemos que los costos son no negativos[4].

COSTO INDIVIDUAL

Los elementos anteriores definen un problema y pueden unirse en una estructura de datos simple que se dará como entrada al algoritmo resolvente del problema. Una **solución** de un problema es un camino desde el estado inicial a un estado objetivo. La calidad de la solución se mide por la función costo del camino, y una **solución óptima** tiene el costo más pequeño del camino entre todas las soluciones.

SOLUCIÓN ÓPTIMA

Formular los problemas

En la sección anterior propusimos una formulación del problema de ir a Bucarest en términos de estado inicial, función sucesor, test objetivo y costo del camino. Esta formulación parece razonable, a pesar de omitir muchos aspectos del mundo real. Para

[4] Las implicaciones de costos negativos se exploran en el Ejercicio 3.17.

comparar la descripción de un estado simple, hemos escogido, *En*(*Arad*), para un viaje real por el país, donde el estado del mundo incluye muchas cosas: los compañeros de viaje, lo que está en la radio, el paisaje, si hay algunos policías cerca, cómo está de lejos la parada siguiente, el estado de la carretera, el tiempo, etcétera. Todas estas consideraciones se dejan fuera de nuestras descripciones del estado porque son irrelevantes para el problema de encontrar una ruta a Bucarest. Al proceso de eliminar detalles de una representación se le llama **abstracción**.

ABSTRACCIÓN

Además de abstraer la descripción del estado, debemos abstraer sus acciones. Una acción de conducir tiene muchos efectos. Además de cambiar la localización del vehículo y de sus ocupantes, pasa el tiempo, consume combustible, genera contaminación, y cambia el agente (como dicen, el viaje ilustra). En nuestra formulación, tenemos en cuenta solamente el cambio en la localización. También, hay muchas acciones que omitiremos: encender la radio, mirar por la ventana, el retraso de los policías, etcétera. Y por supuesto, no especificamos acciones a nivel de «girar la rueda tres grados a la izquierda».

¿Podemos ser más precisos para definir los niveles apropiados de abstracción? Piense en los estados y las acciones abstractas que hemos elegido y que se corresponden con grandes conjuntos de estados detallados del mundo y de secuencias detalladas de acciones. Ahora considere una solución al problema abstracto: por ejemplo, la trayectoria de Arad a Sibiu a Rimnicu Vilcea a Pitesti a Bucarest. Esta solución abstracta corresponde a una gran cantidad de trayectorias más detalladas. Por ejemplo, podríamos conducir con la radio encendida entre Sibiu y Rimnicu Vilcea, y después lo apagamos para el resto del viaje. La abstracción es *válida* si podemos ampliar cualquier solución abstracta a una solución en el mundo más detallado; una condición suficiente es que para cada estado detallado de «en Arad», haya una trayectoria detallada a algún estado «en Sibiu», etcétera. La abstracción es útil si al realizar cada una de las acciones en la solución es más fácil que en el problema original; en este caso pueden ser realizadas por un agente que conduce sin búsqueda o planificación adicional. La elección de una buena abstracción implica quitar tantos detalles como sean posibles mientras que se conserve la validez y se asegure que las acciones abstractas son fáciles de realizar. Si no fuera por la capacidad de construir abstracciones útiles, los agentes inteligentes quedarían totalmente absorbidos por el mundo real.

3.2 Ejemplos de problemas

La metodología para resolver problemas se ha aplicado a un conjunto amplio de entornos. Enumeramos aquí algunos de los más conocidos, distinguiendo entre problemas de *juguete* y del *mundo-real*. Un **problema de juguete** se utiliza para ilustrar o ejercitar los métodos de resolución de problemas. Éstos se pueden describir de forma exacta y concisa. Esto significa que diferentes investigadores pueden utilizarlos fácilmente para comparar el funcionamiento de los algoritmos. Un **problema del mundo-real** es aquel en el que la gente se preocupa por sus soluciones. Ellos tienden a no tener una sola descripción, pero nosotros intentaremos dar la forma general de sus formulaciones.

PROBLEMA DE JUGUETE

PROBLEMA DEL MUNDO REAL

Problemas de juguete

El primer ejemplo que examinaremos es el **mundo de la aspiradora**, introducido en el Capítulo 2. (*Véase* Figura 2.2.) Éste puede formularse como sigue:

- **Estados:** el agente está en una de dos localizaciones, cada una de las cuales puede o no contener suciedad. Así, hay $2 \times 2^2 = 8$ posibles estados del mundo.
- **Estado inicial:** cualquier estado puede designarse como un estado inicial.
- **Función sucesor:** ésta genera los estados legales que resultan al intentar las tres acciones (*Izquierda, Derecha* y *Aspirar*). En la Figura 3.3 se muestra el espacio de estados completo.
- **Test objetivo:** comprueba si todos los cuadrados están limpios.
- **Costo del camino:** cada costo individual es 1, así que el costo del camino es el número de pasos que lo compone.

Comparado con el mundo real, este problema de juguete tiene localizaciones discretas, suciedad discreta, limpieza fiable, y nunca se ensucia una vez que se ha limpiado. (En la Sección 3.6 relajaremos estas suposiciones). Una cosa a tener en cuenta es que el estado está determinado por la localización del agente y por las localizaciones de la suciedad. Un entorno grande con n localizaciones tiene $n\ 2^n$ estados.

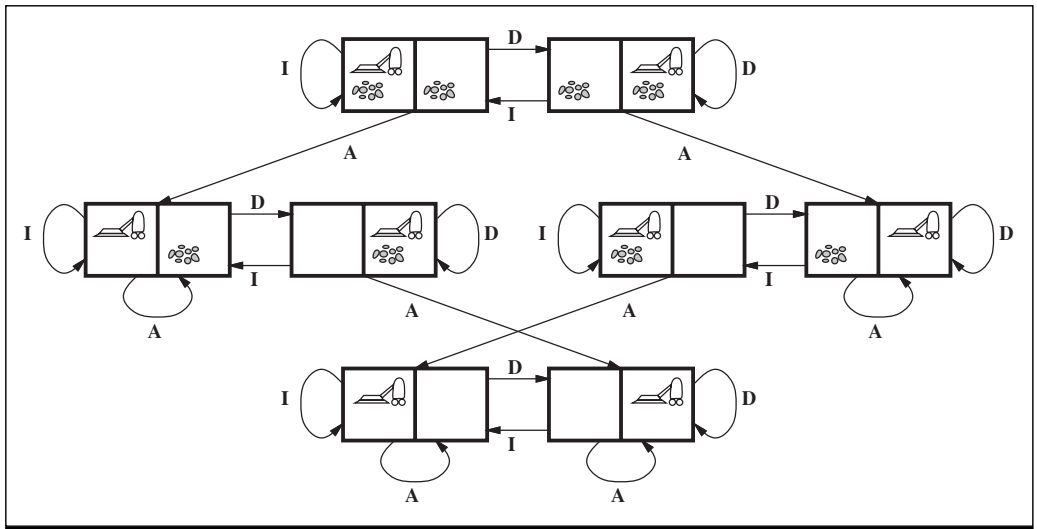


Figura 3.3 Espacio de estados para el mundo de la aspiradora. Los arcos denotan las acciones: I = *Izquierda*, D = *Derecha*, A = *Aspirar*.

8-PUZZLE

El **8-puzle**, la Figura 3.4 muestra un ejemplo, consiste en un tablero de 3×3 con ocho fichas numeradas y un espacio en blanco. Una ficha adyacente al espacio en blanco puede deslizarse a éste. La meta es alcanzar el estado objetivo especificado, tal como se muestra a la derecha de la figura. La formulación estándar es como sigue:

- **Estados:** la descripción de un estado especifica la localización de cada una de las ocho fichas y el blanco en cada uno de los nueve cuadrados.

- **Estado inicial:** cualquier estado puede ser un estado inicial. Nótese que cualquier objetivo puede alcanzarse desde exactamente la mitad de los estados iniciales posibles (Ejercicio 3.4).
- **Función sucesor:** ésta genera los estados legales que resultan de aplicar las cuatro acciones (mover el blanco a la *Izquierda*, *Derecha*, *Arriba* y *Abajo*).
- **Test objetivo:** comprueba si el estado coincide con la configuración objetivo que se muestra en la Figura 3.4. (son posibles otras configuraciones objetivo).
- **Costo del camino:** el costo de cada paso del camino tiene valor 1, así que el costo del camino es el número de pasos.

¿Qué abstracciones se han incluido? Las acciones se han abstraído a los estados iniciales y finales, ignorando las localizaciones intermedias en donde se deslizan los bloques. Hemos abstraído acciones como la de sacudir el tablero cuando las piezas no se pueden mover, o extraer las piezas con un cuchillo y volverlas a poner. Nos dejan con una descripción de las reglas del puzle que evitan todos los detalles de manipulaciones físicas.

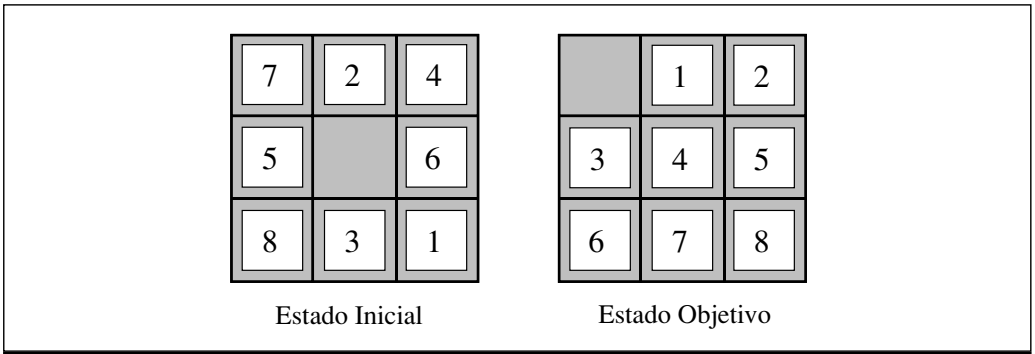


Figura 3.4 Un ejemplo típico del 8-puzle.

PIEZAS DESLIZANTES

El 8-puzle pertenece a la familia de puzles con **piezas deslizantes**, los cuales a menudo se usan como problemas test para los nuevos algoritmos de IA. Esta clase general se conoce por ser NP completa, así que no esperamos encontrar métodos perceptiblemente mejores (en el caso peor) que los algoritmos de búsqueda descritos en este capítulo y en el siguiente. El 8-puzle tiene $9!/2 = 181{,}440$ estados alcanzables y se resuelve fácilmente. El 15 puzle (sobre un tablero de 4×4) tiene alrededor de 1,3 trillones de estados, y configuraciones aleatorias pueden resolverse óptimamente en pocos milisegundos por los mejores algoritmos de búsqueda. El 24 puzle (sobre un tablero de 5×5) tiene alrededor de 10^{25} estados, y configuraciones aleatorias siguen siendo absolutamente difíciles de resolver de manera óptima con los computadores y algoritmos actuales.

PROBLEMA 8-REINAS

El objetivo del **problema de las 8-reinas** es colocar las ocho reinas en un tablero de ajedrez de manera que cada reina no ataque a ninguna otra. (Una reina ataca alguna pieza si está en la misma fila, columna o diagonal.) La Figura 3.5 muestra una configuración que no es solución: la reina en la columna de más a la derecha está atacando a la reina de arriba a la izquierda.

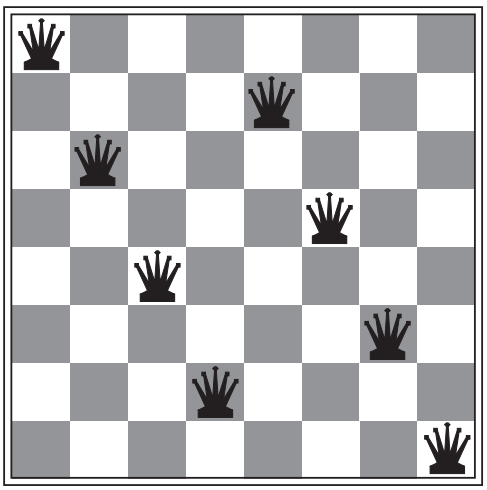


Figura 3.5 Casi una solución del problema de las 8-reinas. (La solución se deja como ejercicio.)

Aunque existen algoritmos eficientes específicos para este problema y para el problema general de las n reinas, sigue siendo un problema test interesante para los algoritmos de búsqueda. Existen dos principales formulaciones. Una **formulación incremental** que implica a operadores que aumentan la descripción del estado, comenzando con un estado vacío; para el problema de las 8-reinas, esto significa que cada acción añade una reina al estado. Una **formulación completa de estados** comienza con las ocho reinas en el tablero y las mueve. En cualquier caso, el coste del camino no tiene ningún interés porque solamente cuenta el estado final. La primera formulación incremental que se puede intentar es la siguiente:

FORMULACIÓN INCREMENTAL

FORMULACIÓN COMPLETA DE ESTADOS

- **Estados:** cualquier combinación de cero a ocho reinas en el tablero es un estado.
- **Estado inicial:** ninguna reina sobre el tablero.
- **Función sucesor:** añadir una reina a cualquier cuadrado vacío.
- **Test objetivo:** ocho reinas sobre el tablero, ninguna es atacada.

En esta formulación, tenemos $64 \cdot 63 \cdots 57 \approx 3 \times 10^{14}$ posibles combinaciones a investigar. Una mejor formulación deberá prohibir colocar una reina en cualquier cuadrado que esté realmente atacado:

- **Estados:** son estados, la combinación de n reinas ($0 \leq n \leq 8$), una por columna desde la columna más a la izquierda, sin que una reina ataque a otra.
- **Función sucesor:** añadir una reina en cualquier cuadrado en la columna más a la izquierda vacía tal que no sea atacada por cualquier otra reina.

Esta formulación reduce el espacio de estados de las 8-reinas de 3×10^{14} a 2.057, y las soluciones son fáciles de encontrar. Por otra parte, para 100 reinas la formulación inicial tiene 10^{400} estados mientras que las formulaciones mejoradas tienen cerca de 10^{52} estados (Ejercicio 3.5). Ésta es una reducción enorme, pero el espacio de estados mejorado sigue siendo demasiado grande para los algoritmos de este capítulo. El Capítulo 4

describe la formulación completa de estados y el Capítulo 5 nos da un algoritmo sencillo que hace el problema de un millón de reinas fácil de resolver.

Problemas del mundo real

PROBLEMA DE BÚSQUEDA DE UNA RUTA

Hemos visto cómo el **problema de búsqueda de una ruta** está definido en términos de posiciones y transiciones a lo largo de ellas. Los algoritmos de búsqueda de rutas se han utilizado en una variedad de aplicaciones, tales como rutas en redes de computadores, planificación de operaciones militares, y en sistemas de planificación de viajes de líneas aéreas. Estos problemas son complejos de especificar. Consideremos un ejemplo simplificado de un problema de viajes de líneas aéreas que especificamos como:

- **Estados:** cada estado está representado por una localización (por ejemplo, un aeropuerto) y la hora actual.
- **Estado inicial:** especificado por el problema.
- **Función sucesor:** devuelve los estados que resultan de tomar cualquier vuelo programado (quizá más especificado por la clase de asiento y su posición) desde el aeropuerto actual a otro, que salgan a la hora actual más el tiempo de tránsito del aeropuerto.
- **Test objetivo:** ¿tenemos nuestro destino para una cierta hora especificada?
- **Costo del camino:** esto depende del costo en dinero, tiempo de espera, tiempo del vuelo, costumbres y procedimientos de la inmigración, calidad del asiento, hora, tipo de avión, kilometraje del aviador experto, etcétera.

Los sistemas comerciales de viajes utilizan una formulación del problema de este tipo, con muchas complicaciones adicionales para manejar las estructuras bizantinas del precio que imponen las líneas aéreas. Cualquier viajero experto sabe, sin embargo, que no todo el transporte aéreo va según lo planificado. Un sistema realmente bueno debe incluir planes de contingencia (tales como reserva en vuelos alternativos) hasta el punto de que éstos estén justificados por el coste y la probabilidad de la falta del plan original.

PROBLEMAS TURÍSTICOS

Los **problemas turísticos** están estrechamente relacionados con los problemas de búsqueda de una ruta, pero con una importante diferencia. Consideremos, por ejemplo, el problema, «visitar cada ciudad de la Figura 3.2 al menos una vez, comenzando y finalizando en Bucarest». Como en la búsqueda de rutas, las acciones corresponden a viajes entre ciudades adyacentes. El espacio de estados, sin embargo, es absolutamente diferente. Cada estado debe incluir no sólo la localización actual sino también *las ciudades que el agente ha visitado*. El estado inicial sería «En Bucarest; visitado {Bucarest}», un estado intermedio típico sería «En Vaslui; visitado {Bucarest, Urziceni, Vaslui}», y el test objetivo comprobaría si el agente está en Bucarest y ha visitado las 20 ciudades.

PROBLEMA DEL VIAJANTE DE COMERCIO

El **problema del viajante de comercio** (PVC) es un problema de ruta en la que cada ciudad es visitada exactamente una vez. La tarea principal es encontrar el viaje *más corto*. El problema es de tipo NP duro, pero se ha hecho un gran esfuerzo para mejorar las capacidades de los algoritmos del PVC. Además de planificación de los viajes del viajante de comercio, estos algoritmos se han utilizado para tareas tales como la plani-

ficación de los movimientos de los taladros de un circuito impreso y para abastecer de máquinas a las tiendas.

DISTRIBUCIÓN VLSI

Un problema de **distribución VLSI** requiere la colocación de millones de componentes y de conexiones en un chip verificando que el área es mínima, que se reduce al mínimo el circuito, que se reduce al mínimo las capacitaciones, y se maximiza la producción de fabricación. El problema de la distribución viene después de la fase de diseño lógico, y está dividido generalmente en dos partes: **distribución de las celdas** y **dirección del canal**. En la distribución de las celdas, los componentes primitivos del circuito se agrupan en las celdas, cada una de las cuales realiza una cierta función. Cada celda tiene una característica fija (el tamaño y la forma) y requiere un cierto número de conexiones a cada una de las otras celdas. El objetivo principal es colocar las celdas en el chip de manera que no se superpongan y que quede espacio para que los alambres que conectan celdas puedan colocarse entre ellas. La dirección del canal encuentra una ruta específica para cada alambre por los espacios entre las celdas. Estos problemas de búsqueda son extremadamente complejos, pero definitivamente dignos de resolver. En el Capítulo 4, veremos algunos algoritmos capaces de resolverlos.

NAVEGACIÓN DE UN ROBOT

La **navegación de un robot** es una generalización del problema de encontrar una ruta descrito anteriormente. Más que un conjunto discreto de rutas, un robot puede moverse en un espacio continuo con (en principio) un conjunto infinito de acciones y estados posibles. Para un robot circular que se mueve en una superficie plana, el espacio es esencialmente de dos dimensiones. Cuando el robot tiene manos y piernas o ruedas que se deben controlar también, el espacio de búsqueda llega a ser de muchas dimensiones. Lo que se requiere es que las técnicas avanzadas hagan el espacio de búsqueda finito. Examinaremos algunos de estos métodos en el Capítulo 25. Además de la complejidad del problema, los robots reales también deben tratar con errores en las lecturas de los sensores y controles del motor.

SECUENCIACIÓN PARA EL ENSAMBLAJE AUTOMÁTICO

La **secuenciación para el ensamblaje automático** por un robot de objetos complejos fue demostrado inicialmente por FREDDY (Michie, 1972). Los progresos desde entonces han sido lentos pero seguros, hasta el punto de que el ensamblaje de objetos tales como motores eléctricos son económicamente factibles. En los problemas de ensamblaje, lo principal es encontrar un orden en los objetos a ensamblar. Si se elige un orden equivocado, no habrá forma de añadir posteriormente una parte de la secuencia sin deshacer el trabajo ya hecho. Verificar un paso para la viabilidad de la sucesión es un problema de búsqueda geométrico difícil muy relacionado con la navegación del robot. Así, la generación de sucesores legales es la parte costosa de la secuenciación para el ensamblaje. Cualquier algoritmo práctico debe evitar explorar todo, excepto una fracción pequeña del espacio de estados. Otro problema de ensamblaje importante es el **diseño de proteínas**, en el que el objetivo es encontrar una secuencia de aminoácidos que se plegarán en una proteína de tres dimensiones con las propiedades adecuadas para curar alguna enfermedad.

DISEÑO DE PROTEÍNAS

BÚSQUEDA EN INTERNET

En la actualidad, se ha incrementado la demanda de robots *software* que realicen la **búsqueda en Internet**, la búsqueda de respuestas a preguntas, de información relacionada o para compras. Esto es una buena aplicación para las técnicas de búsqueda, porque es fácil concebir Internet como un grafo de nodos (páginas) conectadas por arcos. Una descripción completa de búsqueda en Internet se realiza en el Capítulo 10.

tes. Un agente de aprendizaje-Q, por otro lado, puede comparar los valores de sus posibilidades disponibles sin la necesidad de saber sus resultados, así que no necesita un modelo del entorno. Por otro lado, ya que no sabe dónde le conducen sus acciones, los agentes de aprendizaje-Q no pueden mirar hacia delante; esto puede restringir seriamente su habilidad para aprender, como veremos.

APRENDIZAJE PASIVO

En el Apartado 21.2 empezaremos con **aprendizaje pasivo**, donde la política del agente está fijada y la tarea es aprender las utilidades de los estados (o parejas estado-acción); esto también puede suponer el aprendizaje de un modelo del entorno. El Apartado 21.3 cubre el **aprendizaje activo**, donde el agente debe aprender también qué hacer. El principal aspecto es la **exploración**: un agente debe tener tantas experiencias de su entorno como le sea posible para aprender cómo comportarse en él. El Apartado 21.4 discute cómo un agente puede usar aprendizaje inductivo para aprender más rápido de sus experiencias. El Apartado 21.5 cubre métodos de aprendizaje de representaciones directas de políticas en agentes reactivos. Es esencial para este capítulo entender los procesos de decisión de Markov (Capítulo 17).

APRENDIZAJE ACTIVO

EXPLORACIÓN

21.2 Aprendizaje por refuerzo pasivo

Para que las cosas sean sencillas, comenzamos con el caso de un agente de aprendizaje pasivo que utiliza una representación basada en estados en un entorno completamente observable. En aprendizaje pasivo, la política del agente π está fijada: en el estado s, siempre ejecuta la acción $\pi(s)$. Su meta es sencillamente aprender la bondad de la política, es decir, aprender la función de utilidad $U^{\pi}(s)$. Usaremos nuestro ejemplo del mundo 4 $\times$ 3, introducido en el Capítulo 17. La Figura 21.1 muestra una política para este mundo y sus utilidades correspondientes. De forma clara, la tarea del aprendizaje pasivo es

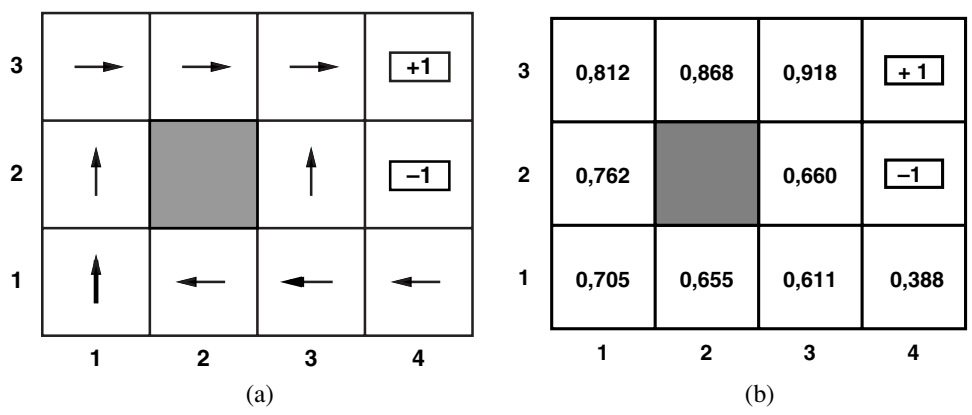


Figura 21.1 (a) Una política π para el mundo 4 $\times$ 3; esta política es óptima con recompensas de $R(s) = -0{,}04$ en los estados no terminales y sin descuentos. (b) Las utilidades de los estados en el mundo 4 $\times$ 3, dada la política π.

similar a la tarea de la **evaluación de la política**, parte del algoritmo de **iteración de la política** descrito en el Apartado 17.3. La principal diferencia es que el agente de aprendizaje pasivo no conoce el **modelo de transiciones** $T(s, a, s')$, que especifica la probabilidad de alcanzar el estado s' a partir del estado s después de realizar la acción a; ni la **función recompensa** $R(s)$, que especifica la recompensa de cada estado.

PRUEBA

El agente ejecuta un conjunto de **pruebas** en el entorno usando su política π. En cada prueba, el agente comienza en el estado (1, 1) y experimenta una secuencia de transiciones de estados hasta que alcanza uno de los estados terminales, (4, 2) o (4, 3). Sus percepciones proporcionan tanto el estado actual como la recompensa que se recibe en ese estado. Las pruebas típicas tienen el siguiente aspecto:

$$(1,1)_{-0,04} \rightarrow (1,2)_{-0,04} \rightarrow (1,3)_{-0,04} \rightarrow (1,2)_{-0,04} \rightarrow (1,3)_{-0,04} \rightarrow (2,3)_{-0,04} \rightarrow (3,3)_{-0,04} \rightarrow (4,3)_{+1}$$
$$(1,1)_{-0,04} \rightarrow (1,2)_{-0,04} \rightarrow (1,3)_{-0,04} \rightarrow (2,3)_{-0,04} \rightarrow (3,3)_{-0,04} \rightarrow (3,2)_{-0,04} \rightarrow (3,3)_{-0,04} \rightarrow (4,3)_{+1}$$
$$(1,1)_{-0,04} \rightarrow (2,1)_{-0,04} \rightarrow (3,1)_{-0,04} \rightarrow (3,2)_{-0,04} \rightarrow (4,2)_{-1}$$

Nótese que cada estado percibido se subcodifica con la recompensa recibida. El objetivo es usar la información sobre las recompensas para aprender la utilidad esperada $U^{\pi}(s)$ asociada con cada estado no terminal. La utilidad se define como la suma esperada de recompensas (descontadas) obtenidas si se sigue la política π. Como en la Ecuación (17.3) en el Apartado 17.2, esto se escribe como

$$U^{\pi}(s) = E\left[\sum_{t=0}^{\infty} \gamma^t R(s_t) | \pi, s_0 = s\right] \tag{21.1}$$

Incluiremos un **factor de descuento** γ en todas nuestras ecuaciones, pero para el mundo 4×3 pondremos $\gamma = 1$.

Estimación directa de la utilidad

ESTIMACIÓN DIRECTA DE LA UTILIDAD

TEORÍA DE CONTROL ADAPTATIVO

Widrow y Hoff (1960) inventaron un método sencillo para la **estimación directa de la utilidad** a finales de la década de los 50 en el área de la **teoría de control adaptativo**. La idea es que la utilidad de un estado es la recompensa total esperada del estado en adelante, y cada prueba proporciona un *ejemplo* del valor para cada estado visitado. Por ejemplo, la primera prueba de las tres dadas anteriormente, proporciona una muestra de recompensa total de 0,72 para el estado (1, 1), dos muestras de 0,66 y 0,84 para (1, 2), dos muestras de 0,80 y 0,88 para (1, 3), etc. Así, al final de cada secuencia, el algoritmo calcula la recompensa observada al llegar a cada estado y actualiza la utilidad estimada para dicho estado, manteniendo una media para cada estado en una tabla. En el límite de infinitas pruebas, la media de la muestra convergerá al valor real esperado en la Ecuación (21.1).

Está claro que la estimación directa de la utilidad es sólo un ejemplo de aprendizaje supervisado donde cada ejemplo tiene como entrada el estado y como salida la recompensa observada por llegar. Esto quiere decir que hemos reducido el aprendizaje por refuerzo a un problema estándar de aprendizaje inductivo, como se discutió en el Capítulo 18. El Apartado 21.4 discute el uso de tipos de representaciones más potentes para las funciones de utilidad, como redes de neuronas. Las técnicas de aprendizaje de dichas representaciones se pueden aplicar directamente a los datos observados.

La estimación directa de la utilidad tiene éxito reduciendo el problema del aprendizaje por refuerzo a un problema de aprendizaje inductivo, sobre el que se conoce más. Desafortunadamente, pierde una fuente muy importante de información, a saber, ¡el hecho de que las utilidades no son independientes! *La utilidad de cada estado es igual a su propia recompensa más la utilidad esperada de sus estados sucesores*. Es decir, los valores de la utilidad obedecen la ecuación de Bellman para una política fija (*véase* también la Ecuación (17.10)):

$$U^{\pi}(s) = R(s) + \gamma \sum_{s'} T(s, \pi(s), s') U^{\pi}(s') \tag{21.2}$$

Ignorando la conexión entre los estados, la estimación directa de la utilidad pierde oportunidades para el aprendizaje. Por ejemplo, la segunda de las tres pruebas dadas anteriormente alcanza el estado (3, 2), que no ha sido previamente visitado. La siguiente transición alcanza (3, 3), del que se conoce a partir de la primera prueba que tiene una alta utilidad. La ecuación de Bellman sugiere inmediatamente que es también probable que (3, 2) tenga una alta utilidad, ya que conduce a (3, 3), pero la estimación directa de la utilidad no aprende nada hasta que se finaliza la prueba. Más ampliamente, podemos ver la estimación directa de la utilidad como una búsqueda en un espacio de hipótesis para U que es más grande de lo que necesita ser, ya que incluye muchas funciones que violan la ecuación de Bellman. Por esta razón, el algoritmo a menudo converge muy lentamente.

Programación dinámica adaptativa

PROGRAMACIÓN DINÁMICA ADAPTATIVA

Para aprovechar las restricciones entre los estados, un agente debe aprender cómo estos están conectados. Un agente de **programación dinámica adaptativa** o **ADP** (*Adaptative Dynamic Programming*) trabaja aprendiendo el modelo de transiciones del entorno a medida que las recorre, y resolviendo el correspondiente proceso de decisión de Markov mediante un método de programación dinámica. Para un agente de aprendizaje pasivo, esto significa introducir el modelo de transiciones aprendido $T(s, \pi(s), s')$ y la recompensa observada $R(s)$ en las ecuaciones de Bellman (21.2), para calcular las utilidades de los estados. Como remarcamos en nuestra discusión sobre la iteración de la política del Capítulo 17, estas ecuaciones son lineales (no suponen maximización) así que se pueden resolver usando cualquier paquete de álgebra lineal. De forma alternativa, podemos adoptar el enfoque de la **iteración de políticas modificado** (*véase* Apartado 17.4), usando un proceso simplificado de iteración del valor para actualizar la utilidad estimada después de cada cambio en el modelo aprendido. Ya que el modelo normalmente cambia sólo un poco con cada observación, el proceso de iteración del valor puede usar la utilidad estimada previamente como valor inicial y debería converger más rápidamente.

El proceso de aprendizaje del modelo en sí mismo es sencillo, ya que el entorno es completamente observable. Esto significa que tenemos una tarea de aprendizaje supervisado donde la entrada es un par estado-acción y la salida es el estado resultante. En el caso más simple, podemos representar el modelo de transición como una tabla de probabilidades. Manteniendo la frecuencia con la que ocurre cada acción de salida y estimando la probabilidad de transición $T(s, a, s')$ a partir de la frecuencia con la que se

alcanza s' cuando se ejecuta a en s[1]. Por ejemplo, en las tres trazas dadas en la página 873, *Derecha* se ejecuta tres veces en (1, 3) y dos de cada tres veces el estado resultante es (2, 3), así se estima que $T((1, 3), \textit{Derecha}, (2, 3))$ es 2/3.

El programa de agente completo para un agente pasivo ADP se muestra en la Figura 21.2. En la Figura 21.3 se muestra su funcionamiento en el mundo 3×4. En términos de la rapidez con que mejora su estimación del valor, el agente ADP hace lo mejor posible, sujeto a su habilidad de aprender el modelo de transiciones. En este sentido, proporciona un estándar con el que medir otros algoritmos de aprendizaje por refuerzo. Sin embargo, esto es en cierto modo intratable para grandes espacios de estados. En el backgammon, por ejemplo, conllevaría resolver aproximadamente 10^{50} ecuaciones con 10^{50} variables desconocidas.

Aprendizaje de diferencia temporal



Es posible tener (casi) lo mejor de ambos mundos; es decir, uno puede aproximar las ecuaciones de restricciones mostradas anteriormente sin resolverlas para todos los posibles estados. *La clave es usar las transiciones observadas para ajustar los valores de los estados observados para que sean acordes con las ecuaciones de restricciones*. Considere, por ejemplo, la transición desde (1, 3) a (2, 3) en la segunda prueba de la página 873. Suponga que, como resultado de la primera prueba, las utilidades estima-

función AGENTE-PASIVO-ADP (percepción) **devuelve** una acción
  entradas: *percepción*, indica el estado actual s' y la señal de recompensa r'
  estática: π, una política fija
    mdp, un MDP con modelo T, recompensas R, descuento γ
    U, una tabla de utilidades, inicialmente vacía
    N_{sa}, una tabla de frecuencias para pares estado-acción, inicialmente a cero
    $N_{sas'}$, una tabla de frecuencias para tripletas estado-acción-estado, inicialmente a cero
    s, a, el estado y la acción previa, inicialmente a nulo *(null)*

  si s' es nuevo **entonces hacer** $U[s'] \leftarrow r'$; $R[s'] \leftarrow r'$
  si s no es nulo (null) **entonces hacer**
    incrementar $N_{sa}[s, a]$ y $N_{sas'}[s, a, s']$
    para cada t tal que $N_{sas'}[s, a, t]$ no sea cero **hacer**
      $T[s, a, t] \leftarrow N_{sas'}[s, a, t] / N_{sa}[s, a]$
  $U \leftarrow$ DETERMINACIÓN-VALOR(π, U, *mdp*)
  si TERMINAL?$[s']$ **entonces** $s, a \leftarrow$ nulo *(null)* **si no** $s, a \leftarrow s', \pi[s']$
  devolver a

Figura 21.2 Un agente de aprendizaje por refuerzo pasivo basado en programación dinámica adaptativa. Para simplificar el código, hemos asumido que cada percepción puede dividirse en un estado percibido y una señal de recompensa.

[1] Esta es la estimación de máxima verosimilitud, como se discutió en el Capítulo 20. Una actualización Bayesiana con una distribución a priori Dirichlet funcionaría mejor.

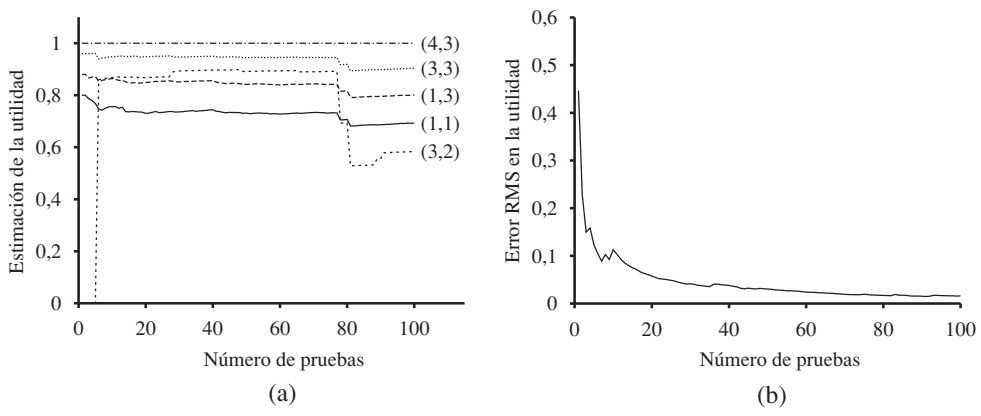


Figura 21.3 Las curvas de aprendizaje pasivo ADP para el mundo 4×3, dada la política óptima mostrada en la Figura 21.1. (a) La utilidad estimada para un subconjunto de estados seleccionados, como una función del número de pruebas. Nótense los grandes cambios que ocurren alrededor de la prueba 78, esta es la primera vez que el agente cae en el estado terminal -1 en (4, 2). (b) La raíz del error cuadrado medio en la estimación de $U(1, 1)$, promediada sobre 20 ejecuciones de 100 pruebas cada una.

das son $U^{\pi}(1, 3) = 0{,}84$ y $U^{\pi}(2, 3) = 0{,}92$. Ahora, si esta transición ocurriese todo el tiempo, esperaríamos que las utilidades obedecieran

$$U^{\pi}(1, 3) = -0{,}04 + U^{\pi}(2, 3)$$

así que $U^{\pi}(1, 3)$ sería 0,88. Así, su actual estimación de 0,84 podría ser un poco baja y debería incrementarse. De forma más general, cuando ocurre una transición desde un estado s a otro estado s', aplicamos la siguiente actualización a $U^{\pi}(s)$:

$$U^{\pi}(\mathrm{s}) \leftarrow U^{\pi}(\mathrm{s}) + \alpha(R(s) + \gamma U^{\pi}(\mathrm{s}') - U^{\pi}(s)) \tag{21.3}$$

Aquí, α es el parámetro de la **tasa de aprendizaje**. Ya que esta regla actualizada usa la diferencia de utilidades entre estados sucesivos; a menudo se denomina ecuación de **diferencia temporal** o **TD**.

DIFERENCIA TEMPORAL

La idea básica de todos los métodos de diferencia temporal es, primero definir las condiciones que se cumplen localmente cuando la estimación de la utilidad es correcta, y entonces, escribir y actualizar la ecuación que mueve la estimación hacia la ecuación ideal de «equilibrio». En el caso del aprendizaje pasivo, el equilibrio viene dado por la Ecuación (21.2). La Ecuación (21.3) hace que el agente alcance el equilibrio dado por la Ecuación (21.2), pero existe cierta sutileza en todo esto. Primero, nótese que la actualización sólo implica al sucesor observado s', mientras que las condiciones reales de equilibrio implican a todos los posibles estados siguientes. Se podría pensar que esto causa un gran cambio inapropiado en $U^{\pi}(s)$ cuando ocurre una transición muy poco común; pero, de hecho, ya que las transiciones poco comunes ocurren raramente, el *valor medio* de $U^{\pi}(s)$ convergerá al valor correcto. Además, si cambiamos α de un parámetro fijo a una función que decrece a medida que incrementa el número de veces que un estado

ha sido visitado, $U(s)$ convergerá al valor correcto[2]. Esto nos proporciona el programa de agente mostrado en la Figura 21.4. La Figura 21.5 ilustra el funcionamiento del agente pasivo TD en el mundo 4×3. No aprende tan rápido como el agente ADP y muestra variabilidad más alta, pero es más sencillo y requiere menos computación por cada observación. Nótese que *TD no necesita un modelo para llevar a cabo sus actualizaciones*. El entorno proporciona las conexiones entre los estados vecinos en forma de transiciones observadas.



función AGENTE-PASIVO-TD (percepción) **devuelve** una acción
  entradas: percepción, una percepción indica el estado actual s' y la señal de recompensa r'
  estática: π, una política fijada
    U, una tabla de utilidades, inicialmente vacía
    N_s, una tabla de frecuencias por estados, inicialmente a cero
    s, a, r, el estado, la acción y la recompensa previa, inicialmente a nulo (null)

  si s' es nuevo **entonces** $U[s'] \leftarrow r'$
  si s no es nulo (null) **entonces hacer**
    incrementar $N_s[s]$
    $U[s] \leftarrow U[s] + \alpha(N_s[s])(r + \gamma\, U[s'] - U[s])$
  si TERMINAL$[s']$ **entonces** $s, a, r \leftarrow$ nulo (null) **si no** $s, a, r \leftarrow s', \pi\,[s'], r'$
  devolver a

Figura 21.4 Un agente de aprendizaje por refuerzo pasivo que aprende estimaciones de la utilidad usando diferencias temporales.

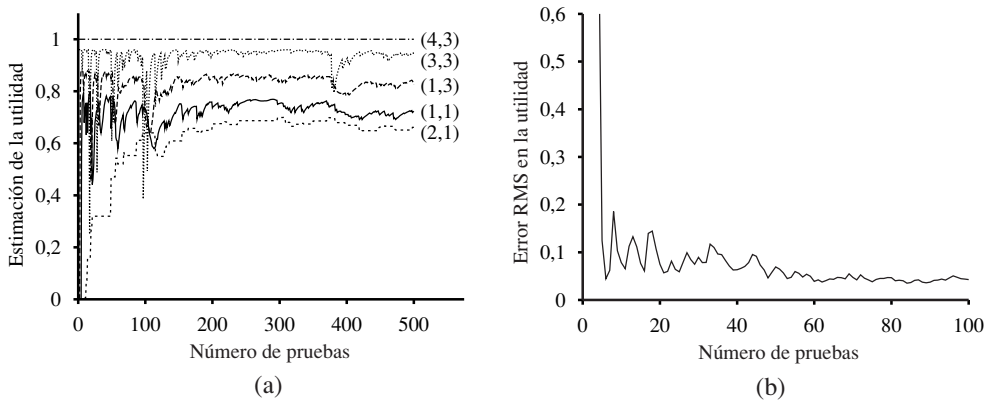


Figura 21.5 Las curvas de aprendizaje TD para el mundo 4×3. (a) La utilidad estimada para un subconjunto seleccionado de estados, como una función del número de pruebas. (b) La raíz del error cuadrado medio en la estimación de $U(1, 1)$, promediada sobre 20 ejecuciones de 500 pruebas cada una. Sólo se muestran las primeras 100 pruebas para permitir la comparación con la Figura 21.3.

[2] Técnicamente, se requiere que $\sum_{n=1}^{\infty}\alpha(n) = \infty$ y $\sum_{n=1}^{\infty}\alpha^2(n) < \infty$. El decrecimiento de $\alpha(n) = 1 / n$ satisface estas condiciones. En la Figura 21.5 hemos usado $\alpha(n) = 60/(59 + n)$.

El enfoque ADP y el enfoque TD están realmente muy relacionados. Ambos intentan hacer ajustes locales a la utilidad estimada para hacer que cada estado «esté de acuerdo» con sus sucesores. Una diferencia es que TD ajusta un estado para que esté de acuerdo con su sucesor *observado* (Ecuación (21.3)), mientras que ADP ajusta el estado para que esté de acuerdo con *todos* los sucesores que pudieran ocurrir, ponderados por sus probabilidades (Ecuación (21.2)). Esta diferencia desaparece cuando los efectos de los ajustes TD se promedian para un gran número de transiciones, ya que la frecuencia de cada sucesor en el conjunto de transiciones es aproximadamente proporcional a su probabilidad. Una diferencia más importante es que mientras TD hace un único ajuste por cada transición observada, ADP hace tantos como necesite para restaurar la consistencia entre la utilidad estimada U y el modelo del entorno T. Aunque la transición observada hace sólo un cambio local en T, sus efectos deberían ser propagados a través de U. Por ello, TD se puede ver como una tosca, pero eficiente, primera aproximación a ADP.

Cada ajuste que hace ADP se puede ver, desde el punto de vista de TD, como un resultado de una «pseudo-experiencia» generada a través de la simulación del modelo actual del entorno. Es posible extender el enfoque de TD para usar un modelo del entorno para generar varias pseudo-experiencias, transiciones que el agente TD puede imaginar que *pudieran* ocurrir, proporcionándolas al modelo actual. Para cada transición observada, el agente TD puede generar un gran número de transiciones imaginarias. De esta forma, la utilidad estimada resultante se aproximará más y más a la de ADP; por supuesto, con el precio de un incremento del tiempo de computación.

En una línea similar, se pueden generar versiones más eficientes de ADP aproximando directamente los algoritmos para iteración de valores o iteración de políticas. Recuérdese que la iteración completa de los valores puede ser intratable cuando el número de estados es grande. Muchos de los pasos de ajuste, sin embargo, son extremadamente pequeños. Un enfoque posible para generar respuestas razonablemente buenas de forma rápida es limitar al número de ajustes que se hacen después de cada transición observada. También se puede usar una heurística para ordenar los posibles ajustes y así llevar a cabo sólo los más significativos. Según la heurística del **barrido con prioridad** es preferible hacer ajustes a estados cuyos sucesores probables han experimentado un *gran número* de ajustes en su propia estimación de utilidad. Usando heurísticas como estas, los algoritmos ADP aproximados pueden aprender casi tan rápido como el ADP completo, en términos del número de secuencias de entrenamiento, pero pueden ser varios órdenes de magnitud más eficientes en términos de computación. (*Véase* Ejercicio 21.3.) Esto les permite manejar espacios de estados que son demasiado grandes para el ADP completo. Los algoritmos ADP aproximados tienen una ventaja adicional: en los primeros pasos del aprendizaje de un nuevo entorno, el modelo del entorno T a menudo estará lejos del correcto, así que hay un pequeño problema en el cálculo de una función de utilidad exacta. Un algoritmo aproximado puede usar un tamaño de ajuste mínimo que decrezca a medida que el modelo del entorno es más preciso. Esto elimina las iteraciones de valor muy largas que pueden ocurrir al principio del aprendizaje debido a los grandes cambios en el modelo.

BARRIDO CON PRIORIDAD

21.3 Aprendizaje por refuerzo activo

Un agente de aprendizaje pasivo tiene una política fija que determina su comportamiento. Un agente activo debe decidir qué acciones tomar. Empecemos con un agente de programación dinámica adaptativa y consideremos cómo debe ser modificado para manejar este nuevo grado de libertad.

Primero, el agente necesitará aprender un modelo completo con salidas probabilísticas para todas las acciones, en vez de aprender sólo el modelo para la política fija. El mecanismo de aprendizaje sencillo que usa el AGENTE-PASIVO-ADP hace esto bien. Luego, necesitamos tener en cuenta el hecho de que el agente tiene que realizar la elección entre las acciones. Las utilidades que necesita aprender son las definidas por la política *óptima*; que obedecen la ecuación de Bellman dada en el Apartado 17.2, que repetimos aquí:

$$U(s) = R(s) + \gamma \max_a \sum_{s'} T(s,a,s')U(s') \tag{21.4}$$

Estas ecuaciones se pueden resolver para obtener la función de utilidad U usando los algoritmos de iteración de valor o de iteración de política del Capítulo 17. Finalmente hay que determinar qué hacer en cada paso. Habiendo obtenido una función de utilidad U que es óptima para el modelo que se está aprendiendo, el agente puede extraer una acción óptima a partir de «un paso de mirar hacia delante» para maximizar la utilidad esperada; de forma alternativa, si se usa iteración de la política, la política óptima ya está disponible, así que simplemente debería ejecutar la acción que recomienda la política óptima. ¿O no?

Exploración

La Figura 21.6 muestra los resultados de una secuencia de pruebas para un agente ADP que sigue las recomendaciones de una política óptima para el modelo aprendido en cada paso. ¡El agente *no* aprende las verdaderas utilidades o la política óptima verdadera! Lo que ocurre en lugar de esto es que, en la prueba 39, encuentra una política que alcanza la recompensa +1 a través de la ruta más corta por (2, 1), (3, 1), (3, 2) y (3, 3). (*Véase* la Figura 21.6.) Después de experimentos con variaciones mínimas, a partir de la prueba 276 en adelante se queda parado en esa política, sin aprender las utilidades de los otros estados y sin encontrar la ruta óptima a través de (1, 2), (1, 3) y (2, 3). A este agente se le denomina **agente voraz**. Los experimentos repetidos muestran que el agente voraz *rara vez* converge a la política óptima para este entorno y algunas veces converge a políticas realmente horrorosas.

AGENTE VORAZ

¿Cómo puede ser que elegir la acción óptima lleve a resultados subóptimos? La respuesta es que el modelo aprendido no es el mismo que el entorno real; por lo tanto lo que es óptimo en el modelo aprendido puede ser subóptimo en el entorno real. Desafortunadamente, el agente no sabe cuál es el entorno real, así que no puede calcular la acción óptima para el mismo. Entonces, ¿qué se debe hacer?

Lo que el agente voraz ha pasado por alto es que las acciones además de proporcionar recompensas de acuerdo con el modelo aprendido actual, también contribuyen al aprendizaje del modelo verdadero, influyendo en las percepciones que se reciben. Me-

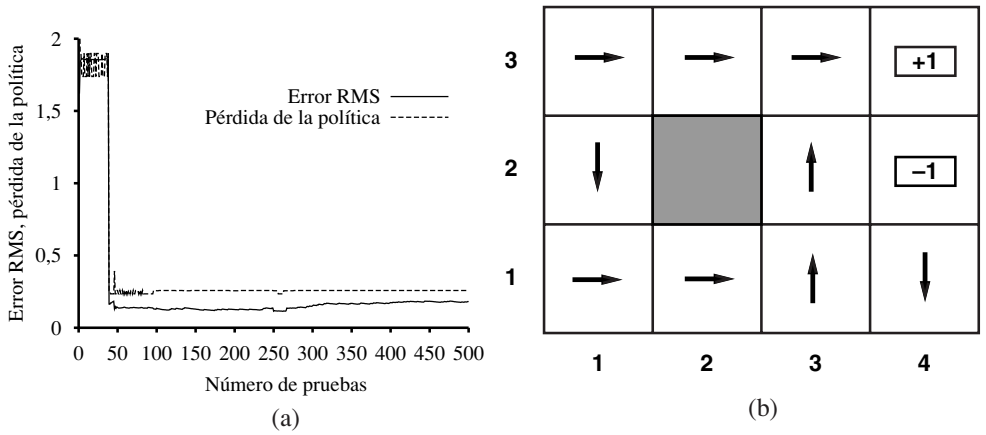


Figura 21.6 Funcionamiento de un agente voraz que ejecuta la acción recomendada por la política óptima del modelo aprendido. (a) Error RMS en la utilidad estimada promediada sobre los nueve cuadrados no terminales. (b) La política subóptima a la que el agente voraz converge en esta particular secuencia de pruebas.

jorando el modelo, el agente recibirá mejores recompensas en el futuro[3]. Por ello, un agente debe tener un compromiso entre la **explotación** para maximizar su recompensa (según refleja su estimación actual de la utilidad) y la **exploración** para maximizar su buen comportamiento a largo plazo. Los riesgos de la explotación pura están estancados. La exploración pura para mejorar el conocimiento de uno mismo no tiene sentido si uno nunca pone dicho conocimiento en práctica. En el mundo real, constantemente se tiene que decidir entre continuar en una confortable existencia o entrar en lo desconocido con la esperanza de descubrir una vida nueva y mejor. Con un buen entendimiento, se necesita menos exploración.

EXPLOTACIÓN

EXPLORACIÓN

¿Podemos ser un poco más precisos que esto? ¿Existe una política de exploración óptima? Resulta que esta pregunta ha sido estudiada en profundidad en el subcampo de la teoría de la decisión estadística que conduce a los denominados **problemas del bandido**. (*Véase* al lado.)

PROBLEMAS DEL BANDIDO

Aunque los problemas del bandido son extremadamente difíciles de resolver con exactitud para obtener un método de exploración *óptima*, sin embargo es posible que surja un esquema *razonable* que nos conducirá eventualmente a un comportamiento óptimo por parte del agente. Técnicamente, cualquier esquema necesita ser voraz en el límite de infinitas exploraciones, o **GLIE**. Un esquema GLIE debe intentar cada acción en cada estado un número ilimitado de veces para evitar tener una probabilidad finita de que una acción óptima se pierda por unas series malas de resultados atípicos. Un agente ADP que use dicho esquema aprenderá eventualmente el modelo del entorno real. Un esquema GLIE también debe convertirse eventualmente en voraz, así las acciones del agente se convertirán en óptimas respecto al modelo aprendido (y por ello el verdadero).

GLIE

[3] Nótese la directa analogía con la teoría del valor de la información del Capítulo 16.

EXPLORACIÓN Y BANDIDOS

En Las Vegas, un *bandido con un solo brazo* es una máquina tragaperras. Un jugador puede insertar una moneda, tirar de la palanca, y recoger las ganancias (si hay). Un **bandido con *n*-brazos** tiene *n* palancas. El jugador debe elegir con qué palanca jugar para cada moneda sucesiva, ¿la que mayor beneficios haya dado, o quizá con la que no haya intentado?

El problema del bandido con *n*-brazos es un modelo formal para problemas reales en muchas áreas de importancia vital, tales como decidir el presupuesto anual para la investigación en IA y su desarrollo. Cada brazo corresponde a una acción (tal como colocar 20 millones de dólares para el desarrollo de nuevos libros de IA), y el resultado de tirar del brazo corresponde a los beneficios obtenidos de tomar la acción (inmensos). La exploración, si es exploración de un nuevo campo de investigación o exploración de un nuevo complejo de tiendas, es arriesgada, es cara, y tiene desenlaces inciertos; por otro lado, no explorar nada significa que uno nunca descubrirá *ningunas* acciones que merezcan la pena.

Para formular un problema del bandido con propiedad, se debe definir exactamente qué se entiende por comportamiento óptimo. La mayoría de las definiciones en la literatura asumen que el propósito es maximizar la recompensa total esperada en el tiempo de vida del agente. Estas definiciones requieren que el valor esperado sea tomado de los posibles mundos en los que el agente pudiera estar, al igual que de los posibles resultados de cada secuencia de acción en cualquier mundo dado. Aquí, un «mundo» se define por el modelo de transiciones $T(s, a, s')$. Por ello, para poder actuar de forma óptima, el agente necesita una distribución a priori de los posibles modelos. Los problemas de optimización resultantes son normalmente intratables.

En algunos casos (por ejemplo, cuando la ganancia de cada máquina es independiente y se utilizan descuentos en las recompensas) es posible calcular el **índice Gittins** para cada máquina tragaperras (Gittins, 1989). El índice es una función del número de veces que se ha jugado con la máquina tragaperras y de la ganancia que ha proporcionado. El índice para cada máquina indica en qué medida merece la pena invertir más, basándose en una combinación de la respuesta esperada y el valor esperado de la información. Elegir la máquina con el valor del índice más alto proporciona una política de exploración óptima. Desafortunadamente, no ha habido forma de extender los índices de Gittins a problemas de decisión secuenciales.

Se puede usar la teoría de los bandidos con *n*-brazos para argumentar la sensatez de la estrategia de selección en los algoritmos genéticos. (*Véase* Capítulo 4.) Si se considera cada brazo en el problema del bandido con *n*-brazos como una posible cadena de genes, y la inversión de una moneda en un brazo como la reproducción de dichos genes, los algoritmos genéticos colocan las monedas de forma óptima, proporcionando un conjunto apropiado de asunciones de independencia.

Existen varios esquemas GLIE; uno de los más sencillos consiste en permitir que el agente elija una acción aleatoria una fracción $1/t$ del tiempo y que siga la política avara en el resto de los casos. Aunque esto converge eventualmente a una política óptima, puede ser extremadamente lento. Un enfoque más sensato proporcionaría algún peso a las acciones que el agente no ha intentado muy a menudo, mientras que tendería a evitar las acciones que tienen una baja utilidad. Esto puede ser implementado alterando la ecuación de restricción (21.4) para que asigne una mayor estimación de la utilidad a pares estado-acción inexplorados. Esencialmente, esto alcanza una distribución a priori optimista sobre los posibles entornos y hace que el agente se comporte inicialmente como si tuviera recompensas maravillosas dispersas por todos los lugares. Usaremos $U^+(s)$ para

denotar la estimación optimista de la utilidad (es decir, la «recompensa por llegar» esperada) del estado s, y sea $N(a, s)$ el número de veces que la acción a se ha intentado en s. Suponga que estamos usando iteración del valor en un agente de aprendizaje ADP; entonces necesitamos reescribir la ecuación de actualización (es decir, la Ecuación (17.6)) para incorporar la estimación optimista. La siguiente ecuación hace esto:

$$U^+(s) \leftarrow R(s) + \gamma \max_a f\left(\sum_{s'} T(s,a,s')U^+(s'), N(a,s)\right) \tag{21.5}$$

FUNCIÓN EXPLORACIÓN

A $f(u, n)$ se le denomina **función exploración**. Determina el compromiso entre la voracidad (preferencia por valores altos de u) y la curiosidad (preferencia por valores bajos de n: acciones que no se intentan a menudo). La función $f(u, n)$ debería ser incrementada en u y decrementada en n. Obviamente, existen muchas funciones posibles que cumplen estas condiciones. Una definición particular sencilla es

$$f(u,n) = \begin{cases} R^+ & \text{if } n < N_e \\ u & \text{en otro caso} \end{cases}$$

donde R^+ es una estimación optimista de la mejor recompensa posible que se puede obtener en cualquier estado y N_e es un parámetro fijo. Esto tendrá el efecto de hacer que el agente intente cada par acción-estado al menos N_e veces.

El hecho de que aparezca U^+ en vez de U en el lado derecho de la Ecuación (21.5) es muy importante. Mientras que la exploración continúa, los estados y las acciones cercanas al estado de comienzo deberían intentarse un gran número de veces. Si usáramos U, la estimación de utilidad más pesimista, entonces rápidamente el agente no estaría dispuesto a explorar más. El uso de U^+ supone que los beneficios de la exploración se propaguen hacia atrás desde los bordes de las regiones sin explorar, así las acciones que llevan *hacia* regiones inexploradas tienen más peso, en vez de las que no resultan familiares. El efecto de esta política de exploración se puede ver de forma clara en la Figura 21.7, que muestra una convergencia rápida a un rendimiento óptimo, a diferencia del enfoque vo-

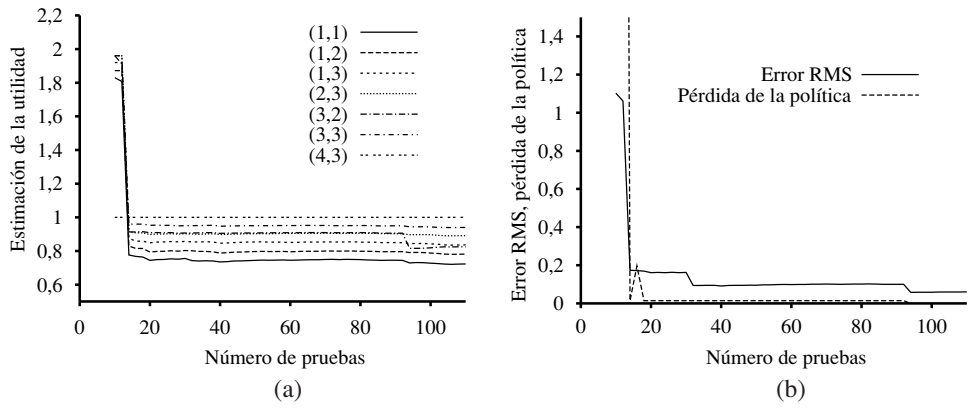


Figura 21.7 Funcionamiento de un agente ADP con exploración, usando $R^+ = 2$ y $N_e = 5$. (a) Estimación de la utilidad para estados seleccionados sobre el tiempo. (b) El error RMS en los valores de la utilidad y la política asociada perdida.

raz. Después de sólo 18 pruebas se encuentra una política cercana a la óptima. Nótese que la propia estimación de la utilidad no converge tan rápidamente. Esto es porque el agente deja de explorar las partes del espacio de estados que no tienen recompensa, visitándolos sólo «por accidente». Sin embargo, para el agente tiene perfecto sentido no preocuparse por las utilidades exactas de los estados que no son deseables y pueden ser evitados.

Aprendizaje de una Función Acción-Valor

Ahora que tenemos un agente ADP activo, consideremos cómo construir un agente de aprendizaje activo de diferencia temporal. El cambio más obvio del caso pasivo es que el agente no está equipado con una política fija, así que, si aprende una función de utilidad U, necesitará aprender un modelo para ser capaz de elegir una acción basada en U a través de un paso de mirada hacia delante. El problema de la adquisición del modelo para el agente TD es idéntico al del agente ADP. ¿Qué actualización del TD se regula a sí misma? Quizá sorprendentemente, la regla de actualización (21.3) permanezca sin cambios. Esto puede parecer extraño, por la siguiente razón: suponga que el agente realiza un paso que normalmente le lleva a un buen destino, pero debido a un entorno no determinístico el agente acaba en un estado catastrófico. La regla de actualización TD tomará esto tan seriamente como si el resultado hubiese sido el resultado normal de la acción, mientras que uno supondría que, ya que el resultado fue casual, el agente no debería preocuparse demasiado por ello. De hecho, por supuesto, ocurrirá un resultado improbable sólo de forma infrecuente en un gran conjunto de secuencias de entrenamiento; por lo tanto en ejecuciones largas sus efectos serán ponderados de forma proporcional a su probabilidad, como esperaríamos. Otra vez, puede mostrarse que el algoritmo TD convergerá a los mismos valores que ADP cuando el número de secuencias de entrenamiento tiende a infinito.

Existe un método TD alternativo denominado **aprendizaje**-Q que aprende una representación acción-valor en vez de aprender utilidades. Usaremos la notación $Q(a, s)$ para denotar el valor de realizar la acción a en el estado s. Los valores-Q están directamente relacionados con los valores de utilidad de la siguiente forma:

$$U(s) = \max_a Q(a,s) \tag{21.6}$$



LIBRE DE MODELO

Las funciones-Q parecen otra forma de almacenar información de utilidad, pero tienen una propiedad importante: *un agente TD que aprende una función-Q no necesita un modelo ni para el aprendizaje ni para la selección de acciones*. Por esta razón, el aprendizaje-Q se denomina método **libre de modelo** (*model-free*). Como con las utilidades, podemos escribir una ecuación de restricción que se mantenga en el equilibrio cuando los valores-Q sean correctos:

$$Q(a,s) = R(s) + \gamma \sum_{s'} T(s,a,s') \max_{a'} Q(a',s') \tag{21.7}$$

Como en el agente de aprendizaje ADP, podemos usar esta ecuación directamente como una ecuación de actualización para un proceso iterativo que calcule los valores-Q exactos, dado un modelo estimado. Esto hace, sin embargo, que se requiera también el

aprendizaje del modelo, ya que la ecuación usa $T(s, a, s')$. El enfoque de diferencia temporal, por otro lado, no requiere ningún modelo. La ecuación de actualización para el aprendizaje-Q TD es

$$Q(a,s) \leftarrow Q(a,s) + \alpha(R(s) + \gamma \max_{a'} Q(a',s') - Q(a,s)) \tag{21.8}$$

que se calcula siempre que se ejecuta una acción a en el estado s que conduce al estado s'.

En la Figura 21.8 se muestra el diseño completo del agente para un agente de aprendizaje-Q exploratorio que usa TD. Nótese que usa exactamente la misma función de exploración f que la usada por el agente ADP exploratorio, de ahí la necesidad de mantener las estadísticas de las acciones tomadas (la tabla N). Si se usa una política de exploración simple (es decir, actuar aleatoriamente en algunos pasos, donde el número de pasos decrece con el tiempo) podemos prescindir de las estadísticas.

El agente de aprendizaje-Q aprende la política óptima para el mundo 4×3, pero lo hace más lentamente que el agente ADP. Esto es porque TD no obliga a que se cumpla la consistencia entre los valores a través del modelo. La comparación plantea una cuestión general: ¿es mejor aprender un modelo y una función de utilidad que aprender una función acción-valor sin modelo? En otras palabras, ¿cuál es la mejor forma de representar la función del agente? Este es un aspecto de básico de la inteligencia artificial. Como empezamos en el Capítulo 1, una de las características claves de la historia de la mayoría de la investigación en IA es su adherencia al enfoque **basado en el conocimiento**. Esto equivale a la asunción de que la mejor forma de representar la función del agente es construir una representación de algunos aspectos del entorno en el cual el agente está ubicado.

Algunos investigadores, tanto dentro como fuera de IA, han reivindicado que la disponibilidad de métodos libres de modelo tales como el aprendizaje-Q significa que el enfoque basado en el conocimiento es innecesario. Sin embargo, esta intuición está poco justificada. Nuestra intuición, sobre lo que merece la pena, dice que cuando el

función AGENTE-APRENDIZAJE-Q (*percepción*) **devuelve** una acción
  entradas: *percepción*, una percepción indica el estado actual s' y la señal de recompensa r'
  estática: Q, una tabla de valores de acción indexada por el estado y la acción
    N_{sa}, una tabla de frecuencias de los pares estado-acción
    s, a, r, el estado, la acción y la recompensa previa, inicialmente nulos

  si s no es nulo **entonces hacer**
    incrementar $N_{sa}[s, a]$
    $Q[s, a] \leftarrow Q[a, s] + \alpha\,(N_{sa}[s, a])\,(r + \gamma \max_{a'} Q[a', s'] - Q[a, s])$
  si TERMINAL?[s′] **entonces** s, a, r ← nulo
  si no $s, a, r \leftarrow s', \text{argmax}_{a'} f(Q[a', s'], N_{sa}[a', s'], r'$
  devolver a

Figura 21.8 Un agente de aprendizaje-Q exploratorio. Es un aprendizaje activo que aprende el valor $Q(a, s)$ de cada acción en cada situación. Usa la misma función de exploración f que el agente ADP exploratorio, pero evita tener que aprender el modelo de transiciones ya que el valor-Q de un estado se puede relacionar directamente con los de sus vecinos.

entorno se convierte en más complejo, las ventajas de un enfoque basado en el conocimiento se manifiestan más. Esto ocurre incluso en juegos como el ajedrez, las damas, y el *backgammon* (*véase* la siguiente sección), donde los esfuerzos para aprender una función de evaluación mediante un modelo han tenido más éxito que los métodos de aprendizaje-Q.

21.4 Generalización en aprendizaje por refuerzo

Hasta ahora, hemos asumido que las funciones de utilidad y las funciones-Q aprendidas por los agentes se representan en forma tabular, con un valor de salida para cada tupla de entrada. Este enfoque funciona razonablemente bien para espacios de estados pequeños, pero el tiempo de convergencia y (para ADP) el tiempo por iteración se incrementa rápidamente a medida que el espacio se hace más grande. Con un control cuidadoso, los métodos ADP aproximados pueden hacer posible el manejo de 10.000 estados o más. Esto es suficiente para entornos bidimensionales como el del laberinto, pero los mundos más realistas se quedan fuera. El ajedrez y el *backgammon* son un pequeño subconjunto del mundo real, sus espacios de estados contienen del orden de 10^{50} a 10^{120} estados. ¡Sería absurdo suponer que se pueden visitar todos estos estados para aprender cómo jugar al juego!

APROXIMACIÓN DE FUNCIONES

Una forma de manejar estos problemas es utilizar **aproximación de funciones**, que consiste en representar la función de forma distinta a una tabla. La representación se ve como una aproximación porque puede no darse el caso de que la función *verdadera* de utilidad o función-Q se pueda representar en la forma elegida. Por ejemplo, en el Capítulo 6 se describió una **función de evaluación** para el ajedrez que se representa como una función lineal ponderada de un conjunto de características (o **funciones base**) $f_1, ..., f_n$:

FUNCIONES BASE

$$\hat{U}_\theta(s) = \theta_1 f_1(s) + \theta_2 f_2 + \cdots + \theta_n f_n(s)$$

Un aprendizaje por refuerzo puede aprender valores para los parámetros $\theta = \theta_1, ..., \theta_n$ de forma que la función de evaluación $\hat{U}_\theta$ aproxime la función de utilidad verdadera. En vez de 10^{120} valores en la tabla, esta aproximación se caracteriza por $n = 20$ parámetros, una compresión *enorme*. Aunque nadie conoce la verdadera función de utilidad para el ajedrez, nadie se cree que se pueda representar exactamente con 20 números. Sin embargo, si la aproximación es lo suficientemente buena, el agente podrá jugar un ajedrez excelente[4].



La aproximación de funciones es práctica para representar funciones para espacios de estados muy grandes, pero éste no es su beneficio principal. La *compresión que se consigue con un aproximador de una función permite al agente de aprendizaje generalizar a partir de estados que ha visitado a estados que no ha visitado*. Esto es, el aspecto

[4] Sabemos que la función de utilidad exacta se puede representar en una página o dos de Lisp, Java o C++. Esto es, se puede representar por un programa que resuelva el juego de forma exacta cada vez que es llamado. Estamos interesados sólo en funciones de aproximación que utilicen una cantidad de cálculo *razonable*. De hecho, podría ser mejor aprender una aproximación de función muy simple, y combinarla con cierta cantidad de búsqueda hacia delante. Los inconvenientes que esto implica todavía no se conocen bien.

más importante de la aproximación de funciones no es que requiera menos espacio, sino que permite hacer generalización inductiva sobre los espacios de entrada. Para darle una idea de la potencia de este efecto: examinando sólo uno de cada 10^{44} de los posibles estados del *backgammon*, es posible aprender una función de utilidad que permite a un programa jugar tan bien como los humanos (Tesauro, 1992).

Por otro lado, por supuesto, está el problema de que pudiera no haber una función en el espacio de hipótesis elegido que aproximara suficientemente bien la función de utilidad verdadera. Como en todo aprendizaje inductivo, hay un compromiso entre el tamaño del espacio de hipótesis y el tiempo que se toma para aprender la función. Un espacio de hipótesis grande incrementa la probabilidad de que se encuentre una buena aproximación, pero también supone que la convergencia se retrase.

Comencemos con el caso más simple, que es una estimación directa de la utilidad. (*Véase* el Apartado 21.2). Con aproximación de funciones, esto es una instancia de **aprendizaje supervisado**. Por ejemplo, suponga que representamos las utilidades para el mundo 4×3 utilizando una función lineal simple. Las características de los cuadrados son exactamente sus coordenadas x e y, tenemos

$$\hat{U}_\theta(x, y) = \theta_0 + \theta_1 x + \theta_2 y \tag{21.9}$$

Así, si $(\theta_0, \theta_1, \theta_2) = (0, 5, 0, 2, 0, 1)$, entonces $\hat{U}_\theta(1, 1) = 0{,}8$. Dada una colección de pruebas, obtenemos un conjunto de valores de muestra $\hat{U}_\theta(x, y)$ y podemos encontrar el mejor ajuste, en el sentido de minimizar el error cuadrado, utilizando regresión lineal estándar. (*Véase* Capítulo 20).

Para el aprendizaje por refuerzo, tiene más sentido utilizar un algoritmo de aprendizaje *en línea* que actualice los parámetros después de cada prueba. Suponga que ejecutamos una prueba y la recompensa total obtenida empezando en (1, 1) es 0,4. Esto sugiere que $\hat{U}_\theta(1, 1)$, actualmente con valor 0,8, es muy grande y debe ser reducido. ¿Cómo se deben ajustar los parámetros para conseguir esto? Como con el aprendizaje de redes neuronales, escribimos una función de error y calculamos su gradiente respecto a los parámetros. Si la recompensa total observada desde el estado s hacia delante en la j-ésima prueba es $u_j(s)$, el error se define como (la mitad) de la diferencia al cuadrado entre el total predicho y el total actual: $E_j(s) = (\hat{U}_\theta(s) - u_j(s))^2/2$. La tasa de cambio del error respecto a cada parámetro θ_i es $\partial E_j/\partial\theta_i$, por lo tanto, para variar el parámetro en la dirección en que el error se decrementa, queremos

$$\theta_i \leftarrow \theta_i - \alpha\frac{\partial E_j(s)}{\partial\theta_i} = \theta_i + \alpha(u_j(s) - \hat{U}_\theta(s))\frac{\partial\hat{U}_\theta(s)}{\partial\theta_i} \tag{21.10}$$

REGLA DE WIDROW-HOFF

REGLA DELTA

Esta se denomina **regla de Widrow-Hoff**, o **regla delta**, para mínimos cuadrados *en línea*. Para el aproximador lineal de función $\hat{U}_\theta(s)$ en la Ecuación (21.9) obtenemos tres sencillas reglas de actualización:

$$\theta_0 \leftarrow \theta_0 + \alpha(u_j(s) - \hat{U}_\theta(s))$$
$$\theta_1 \leftarrow \theta_1 + \alpha(u_j(s) - \hat{U}_\theta(s))x$$
$$\theta_2 \leftarrow \theta_2 + \alpha(u_j(s) - \hat{U}_\theta(s))y$$

Podemos aplicar estas reglas al ejemplo donde $\hat{U}_\theta$ (1, 1) es 0,8 y u_j(1, 1) es 0,4. θ_0, θ_1 y θ_2 se decrementan todas en $0{,}4\alpha$, lo que reduce el error para (1, 1). Observe que ¡cambiar las θ_is *también modifica los valores de* $\hat{U}_\theta$ *para cada uno de los otros estados*! Esto es lo que queremos decir cuando decimos que la aproximación de función permite a un sistema de aprendizaje por refuerzo generalizar a partir de sus experiencias.

Esperamos que el agente aprenda más rápidamente si utiliza un aproximador de función, dado que el espacio de hipótesis no es muy grande, pero incluye algunas funciones que son, de forma razonable, un buen ajuste de la función de utilidad verdadera. El Ejercicio 21.7 le invita a evaluar el rendimiento de la estimación directa de la utilidad, de ambas formas, utilizando y sin utilizar aproximador de función. La mejora en el mundo 4×3 es notable, pero no dramática, porque es un espacio de estados muy pequeño para comenzar con él. La mejora es mucho mayor en un mundo 10×10 con una recompensa de +1 en (10, 10). Este mundo está bien adecuado para una función de utilidad lineal, porque la función de utilidad verdadera es plana y prácticamente lineal. (*Véase* Ejercicio 21.10). Si ponemos una recompensa de +1 en (5, 5), la función de utilidad verdadera es más parecida a una pirámide, y lamentablemente, el aproximador de función fallará. ¡Sin embargo, no está todo perdido! Recuerde que lo que pasa en la aproximación de funciones lineal es que la función es lineal en los *parámetros;* las características en sí mismas pueden ser funciones arbitrarias no lineales de las variables de estado. Así que podemos incluir un término como $\theta_3 = \sqrt{(x - x_g)^2 + (y - y_g)^2}$ que mida la distancia a la meta.

Estas ideas se pueden aplicar igualmente a sistemas de aprendizaje de diferencia temporal. Todo lo que necesitamos hacer es ajustar los parámetros e intentar reducir la diferencia temporal entre estados sucesivos. Las nuevas versiones de las ecuaciones TD y aprendizaje-Q (21.3 y 21.8) son

$$\theta_i \leftarrow \theta_i + \alpha[R(s) + \gamma \hat{U}_\theta(s') - \hat{U}_\theta(s)] \frac{\partial \hat{U}_\theta(s)}{\partial \theta_i} \tag{21.11}$$

para utilidades y

$$\theta_i \leftarrow \theta_i + \alpha[R(s) + \gamma \max_{a'} \hat{Q}_\theta(a', s') - \hat{Q}_\theta(a, s)] \frac{\partial \hat{Q}_\theta(a, s)}{\partial \theta_i} \tag{21.12}$$

para valores-Q. Se puede demostrar que estas reglas de actualización convergen a la aproximación más cercana posible[5] a la función verdadera, cuando la función es *lineal* en los parámetros. Desafortunadamente no se puede decir lo mismo cuando se utilizan funciones *no lineales,* como redes neuronales. Hay algunos casos simples en los que los parámetros pueden irse a infinito, incluso aunque haya buenas soluciones en el espacio de hipótesis. Hay algoritmos más sofisticados que evitan estos problemas, pero actualmente en el aprendizaje por refuerzo con aproximadores generales de funciones sigue siendo un arte delicado.

La aproximación de funciones puede ser también muy útil para aprender un modelo del entorno. Recuerde que aprender un modelo de un entorno *observable* es un pro-

[5] La definición de distancia entre funciones de utilidad es bastante técnica; *véase* Tsitsiklis y Van Roy (1997).

blema de aprendizaje *supervisado*, porque la siguiente percepción proporciona el estado resultado. Se puede utilizar cualquiera de los métodos de aprendizaje supervisado del Capítulo 18, con los ajustes adecuados para tener en cuenta el hecho de que necesitamos predecir una descripción completa del estado en vez de una clasificación booleana o un valor real simple. Por ejemplo, si el estado se define mediante n variables booleanas, necesitaremos aprender n funciones booleanas para predecir todas las variables. Para un entorno *parcialmente observable*, el problema del aprendizaje es mucho más difícil. Si sabemos cuántas variables ocultas hay, y cómo se relacionan de forma causal con otras variables observables, podemos fijar la estructura de una red Bayesiana dinámica y utilizar el algoritmo EM para aprender los parámetros, como describimos en el Capítulo 20. La invención de las variables ocultas y el aprendizaje de la estructura del modelo aún son problemas abiertos.

Ahora volvemos a ejemplos de aplicaciones de gran escala del aprendizaje por refuerzo. Veremos que, en los casos en que se utiliza una función de utilidad (y por lo tanto un modelo), normalmente el modelo se toma como dado. Por ejemplo, en el aprendizaje de una función de evaluación para el *backgammon*, normalmente se asume que son conocidos sus movimientos legales y sus efectos.

Aplicaciones a juegos

La primera aplicación significativa del aprendizaje por refuerzo fue también el primer problema significativo de aprendizaje en general, el programa jugador de ajedrez escrito por Arthur Samuel (1959, 1967). Inicialmente, Samuel utilizó una función lineal ponderada para la evaluación de las posiciones, utilizando hasta 16 términos en un momento dado. Él aplicó una versión de la Ecuación (21.11) para actualizar los pesos. Sin embargo, existen algunas diferencias significativas entre su programa y los métodos actuales. Primero, él actualizaba los pesos utilizando la diferencia entre el estado actual y el valor almacenado generado mediante búsqueda hacia delante completa en el árbol de búsqueda. Esto funciona bien, porque equivale a ver el espacio de estados con diferente granularidad. Una segunda diferencia es que ¡el programa *no* utilizaba ninguna recompensa observada! Esto es, los valores de los estados terminales eran ignorados. Esto significa que es bastante posible que el programa de Samuel no converja, o que converja a una estrategia diseñada para perder en vez de para ganar. Para evitar esta suerte, él insistía en que los pesos para tener ventaja material deben ser siempre positivos. Sorprendentemente, esto era suficiente para dirigir el programa hacia áreas del espacio de pesos correspondientes a un buen jugador de ajedrez.

El sistema TD-Gammon de Gerry Tesauro (1992) ilustra, de forma contundente, el potencial de las técnicas de aprendizaje por refuerzo. En sus primeros trabajos (Tesauro y Sejnowski, 1989), Tesauro intentó aprender una representación de red neuronal de $Q(a, s)$, directamente a partir de ejemplos de movimientos etiquetados por un experto con valores relativos. Este enfoque resultó extremadamente tedioso para el experto. Dio lugar a un programa denominado NEUROGAMMON, que fue fuerte con los estándar de computador, pero no competitivo contra expertos humanos. El proyecto TD-Gammon fue un intento de aprender únicamente a partir del juego propio. La única señal de recompen-

sa se daba al final de cada juego. La función de evaluación se representaba mediante una red neuronal completamente conectada, con una capa oculta que contenía 40 nodos. Simplemente mediante la aplicación repetida de la Ecuación (21.11), TD-Gammon aprendió a jugar considerablemente mejor que Neurogammon, incluso aunque la representación de entrada contenía exactamente la posición del tablero cruda, sin características computadas. Esto supuso alrededor de 200.000 juegos de entrenamiento y dos semanas de tiempo de computación. Aunque puede parecerse a muchos juegos, es sólo una pequeña fracción despreciable del espacio de estados. Cuando se añadieron características precomputadas a la representación de entrada, una red con 80 unidades ocultas fue capaz, después de 300.000 juegos de entrenamiento, de alcanzar un estándar de juego comparable al de los tres mejores jugadores del mundo. Kit Woolsey, un gran jugador y analista dijo: «No cabe duda en mi mente de que su juicio posicional es mucho mejor que el mío.»

Aplicación a control de robots

EL CARRO Y LA PÉRTIGA

PÉNDULO INVERTIDO

En la Figura 21.9 se muestra el esquema del famoso problema de equilibrio del **carro y la pértiga** también conocido como el **péndulo invertido**. El problema consiste en controlar la posición x del carro, de forma que la pértiga se quede prácticamente vertical ($\theta \approx \pi/2$), cuando los límites de la pista en la que está el carro son los que se muestran. En relación con este problema, aparentemente simple, se han publicado más de 2.000 artículos en aprendizaje por refuerzo y teoría de control. El problema del carro y la pértiga difiere de los problemas descritos anteriormente en que las variables de estado x, θ, $\dot{x}$ y $\dot{\theta}$ son continuas. Las acciones normalmente son discretas: tirar a la izquierda o tirar a la derecha, el denominado régimen de **control bang-bang**.

CONTROL BANG-BANG

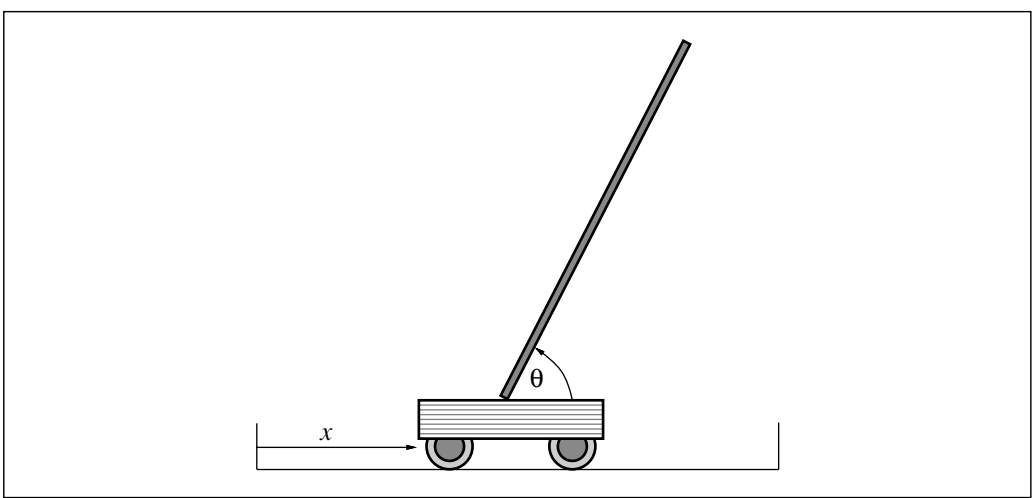


Figura 21.9 Esquema del problema de equilibrar una larga pértiga que está encima de un carro en movimiento. El carro se puede mover a izquierda y derecha por un controlador que observa x, θ, $\dot{x}$ y $\dot{\theta}$.

El primer trabajo de aprendizaje para este problema fue llevado a cabo por Michie y Chambers (1968). Su algoritmo BOXES fue capaz de equilibrar la pértiga más de una hora después de sólo 30 pruebas. Además, al contrario que muchos sistemas posteriores, BOXES se implementó con un carro y una pértiga reales, no con simulación. El algoritmo primero discretiza en cajas (*boxes*) el espacio de estados de cuatro dimensiones (de ahí el nombre). Ejecuta pruebas hasta que la pértiga cae o el carro llega al final de la pista. Con la acción final en la caja final, se asociaba una recompensa negativa que se propagaba hacia atrás en la secuencia. Se descubrió que la discretización causaba algunos problemas cuando el aparato se inicializaba en una posición diferente de las utilizadas en el entrenamiento, lo que sugiere que la generalización no era perfecta. Se puede obtener una mejora de la generalización y un aprendizaje más rápido utilizando un algoritmo que haga particiones *adaptadas* del espacio de estados de acuerdo con la variación observada de la recompensa. Hoy en día, un ejercicio común es equilibrar un péndulo invertido *triple* (una proeza que está lejos de las capacidades de la mayoría de los humanos).

21.5 Búsqueda de la política

BÚSQUEDA DE LA POLITICA

El enfoque final que consideraremos para problemas de aprendizaje por refuerzo se denomina **búsqueda de la política**. De alguna forma, la búsqueda de la política es el método más sencillo de todos los de este capítulo: la idea es mantener la política dando vueltas tanto tiempo como el rendimiento mejore, y después parar.

Empecemos con las propias políticas. Recuerde que una política π es una función que hace corresponder estados a acciones. Estamos interesados principalmente en representaciones *parametrizadas* de π, que tengan menos parámetros que estados hay en el espacio de estados (exactamente como en la sección anterior). Por ejemplo, podríamos representar π mediante una colección de funciones-Q parametrizadas, una para cada acción, y tomar la acción con mayor valor predicho:

$$\pi(s) = \max_a \hat{Q}_\theta(a, s) \tag{21.13}$$

Cada función-Q podría ser una función lineal de parámetros θ, como en la Ecuación (21.9), o podría ser una función no lineal como una red neuronal. Después, la búsqueda de la política ajustará los parámetros θ para mejorar la política. Observe que si la política se representa mediante funciones-Q, la búsqueda de la política es un proceso que aprende funciones-Q. ¡*Este proceso no es el mismo que el aprendizaje-Q*! En el aprendizaje-Q con aproximación de función, el algoritmo encuentra un valor de θ tal que $\hat{Q}_\theta$ es «cercana» a Q^*, la función-Q óptima. Por otro lado, la búsqueda de la política encuentra el valor de θ que proporciona un mejor rendimiento; los valores encontrados pueden diferir muy substancialmente[6]. Otro ejemplo claro de la diferencia es el caso en que $\pi(s)$

[6] De forma trivial, la función-Q aproximada, definida por $\hat{Q}_\theta(a, s) = Q^*(a, s)/10$ proporciona rendimiento óptimo, incluso aunque no está del todo cercana a Q^*.

se calcula utilizando búsqueda hacia delante de profundidad 10 con una función de utilidad aproximada $\hat{U}_\theta$. El valor de θ que proporciona buen rendimiento es una forma de hacer que $\hat{U}_\theta$ se parezca a la función de utilidad verdadera.

Un problema con las representaciones de la política como las de la Ecuación (21.13) es que la política es una función *discontinua* de los parámetros cuando las acciones son discretas[7]. Esto es, habrá valores de θ tales que un cambio infinitesimal en θ cause que la política cambie de una acción a otra. Esto significa que el valor de la política también puede cambiar de forma discontinua, lo que hace complicada la búsqueda basada en el gradiente. Por esta razón, los métodos de búsqueda de la política normalmente utilizan una representación **estocástica de la política** $\pi_\theta(s, a)$, que especifica la *probabilidad* de seleccionar la acción a en el estado s. Una representación popular es la **función softmax**:

POLÍTICA ESTOCASTICA

FUNCION SOFTMAX

$$\pi_\theta(s, a) = \exp(\hat{Q}_\theta(a, s))/\sum_{a'} \exp(\hat{Q}_\theta(a', s))$$

Softmax es prácticamente determinística si una acción es mucho mejor que las demás, pero siempre proporciona una función de θ diferenciable; por eso, el valor de la política (que depende de una forma continua de las probabilidades de selección de acciones) es una función de θ diferenciable.

Ahora, veamos los métodos para mejorar la política. Empezamos con el caso más simple: una política determinística en un entorno determinístico. En este caso, evaluar la política es trivial: simplemente la ejecutamos, y observamos la recompensa acumulada; esto nos proporciona el **valor de la política** $\rho(\theta)$. Mejorar la política es un problema estándar de optimización, como se describía en el Capítulo 4. Podemos seguir el vector **gradiente de la política** $\nabla_\theta \rho(\theta)$, dado que $\rho(\theta)$ es diferenciable. De forma alternativa, podemos seguir el **gradiente empírico** mediante escalada, es decir, evaluando el cambio en la política para incrementos pequeños de cada valor de parámetro. Con las advertencias usuales, este proceso convergerá a un óptimo local en el espacio de políticas.

VALOR DE LA POLÍTICA

GRADIENTE DE LA POLITICA

Cuando el entorno (o la política) es estocástico, las cosas son más complicadas. Suponga que estamos intentando hacer escalada, lo que requiere comparar $\rho(\theta)$ y $\rho(\theta + \Delta\theta)$ para algún $\Delta\theta$ pequeño. El problema es que la recompensa total de cada prueba puede variar enormemente, por lo que estimar el valor de la política a partir de un número de pruebas pequeño será muy poco fiable; intentar comparar dos de estas estimaciones será aún menos fiable. Una solución es simplemente ejecutar muchas pruebas, midiendo las varianzas de muestra y utilizarlas para determinar si se han ejecutado suficientes pruebas para obtener una indicación fiable de la dirección de mejora para $\rho(\theta)$. Desafortunadamente, esto es impracticable para muchos problemas reales, donde cada prueba puede ser muy cara, consumir mucho tiempo, y quizás ser peligrosa.

Para el caso de una política estocástica $\pi_\theta(s, a)$ es posible obtener una estimación imparcial del gradiente en θ, $\nabla_\theta \rho(\theta)$, directamente de los resultados de las pruebas ejecutadas en θ. Por simplicidad, obtendremos esta estimación para el caso simple de un entorno no secuencial en el que la recompensa se obtiene inmediatamente después de

[7] Para un espacio de acciones continuo, la política puede ser una función plana de los parámetros.

actuar en el estado inicial s_0. En este caso, el valor de la política es exactamente el valor esperado de la recompensa, y tenemos

$$\nabla_\theta \rho(\theta) = \nabla_\theta \sum_a \pi_\theta(s_0, a) R(a) = \sum_a (\nabla_\theta \pi_\theta(s_0, a)) R(a)$$

Ahora, realizamos un simple truco, de forma que este sumatorio pueda ser aproximado mediante muestras generadas a partir de la distribución de probabilidad definida por $\pi_\theta(s_0, a)$. Suponga que tenemos N pruebas y que la acción tomada en la prueba j-ésima es a_j. Entonces

$$\nabla_\theta \rho(\theta) = \sum_a \pi_\theta(s_0, a) \cdot \frac{(\nabla_\theta \pi_\theta(s_0, a)) R(a)}{\pi_\theta(s_0, a)} \approx \frac{1}{N} \sum_{j=1}^{N} \frac{(\nabla_\theta \pi_\theta(s_0, a_j)) R(a_j)}{\pi_\theta(s_0, a_j)}$$

Así, el verdadero gradiente del valor de la política se aproxima mediante una suma de términos que incluyen el gradiente de la probabilidad de selección de acción en cada prueba. Para el caso secuencial, esto se generaliza a

$$\nabla_\theta \rho(\theta) \approx \frac{1}{N} \sum_{j=1}^{N} \frac{(\nabla_\theta \pi_\theta(s_0, a_j)) R_j(s)}{\pi_\theta(s_0, a_j)}$$

para cada estado s visitado, donde a_j se ejecuta en s en la j-ésima prueba y $R_j(s)$ es la recompensa total recibida del estado s en adelante en la j-ésima prueba. El algoritmo resultante se denomina REINFORCE (Williams, 1992); normalmente es mucho más efectivo que la escalada utilizando muchas pruebas en cada valor de θ. Sin embargo, es todavía más lento de lo necesario.

Considere la siguiente tarea: dados dos programas de blackjack[8], determinar cuál es el mejor. Una forma de hacer esto es que cada uno juegue contra un «*croupier*» estándar un número determinado de manos, y después mida sus victorias respectivas. El problema con esto, como hemos visto, es que las victorias de cada programa fluctúan enormemente dependiendo de si recibe cartas buenas o malas. Una solución obvia es generar un determinado número de manos por adelantado. De esta forma, eliminamos el error que se debe a la diferencia de cartas recibidas. Esta es la idea en la que se basa el algoritmo PEGASUS (Ng y Jordan 2000). El algoritmo es aplicable a dominios para los que está disponible un simulador y se pueden repetir los resultados «aleatorios» de las acciones. El algoritmo funciona generando por adelantado N secuencias de números aleatorios, cada uno de los cuales se puede utilizar para ejecutar una prueba de cualquier política. La búsqueda de la política se lleva a cabo evaluando cada política candidata haciendo uso del *mismo* conjunto de secuencias aleatorias para determinar los resultados de las acciones. Se puede demostrar que el número de secuencias aleatorias que se requieren para asegurar que el valor de *cada* política está bien estimado, depende únicamente de la complejidad del espacio de políticas, y no de la complejidad del dominio subyacente. El algoritmo PEGASUS ha sido utilizado para desarrollar políticas efectivas en varios dominios, incluyendo vuelo autónomo de helicópteros (*véase* Figura 21.10).

[8] También conocido como ventiuna o pontón.



Figura 21.10 Imágenes superpuestas en un período de tiempo de un helicóptero autónomo realizando una peligrosa maniobra circular. El helicóptero está bajo el control de una política desarrollada por el algoritmo de búsqueda de la política PEGASUS. Se desarrolló un modelo simulador observando los efectos de varias manipulaciones de control en helicópteros reales; el algoritmo fue ejecutado en el modelo simulador por la noche. Se desarrollaron una gran variedad de controladores para diferentes maniobras. En todos los casos, el rendimiento obtenido excedió por mucho el de un piloto humano experto utilizando control remoto. (La imagen es cortesía de Andrew Ng.)

21.6 Resumen

Este capítulo ha examinado el problema de aprendizaje por refuerzo: cómo un agente puede hacerse experto en un entorno desconocido, únicamente a partir de sus propias percepciones y recompensas ocasionales. El aprendizaje por refuerzo se puede ver como un microcosmos para el problema completo de la IA, pero se estudia en esquemas simplificados para facilitar el progreso. Los puntos más importantes son:

- El diseño global del agente dicta el tipo de información que debe ser aprendida. Los tres principales diseños que hemos cubierto son diseños basados en el modelo, utilizando un modelo T y una función de utilidad U; un diseño de modelo libre que utiliza una función Q acción-valor; y el diseño reactivo que utiliza una política π.
- Las utilidades se pueden aprender utilizando tres enfoques:
  - — La **estimación directa de la utilidad** utiliza la recompensa total observada por llegar a un estado dado como evidencia directa para aprender su utilidad.
  - — La **programación dinámica adaptativa** (ADP) aprende un modelo y una función de recompensa a partir de las observaciones y después utiliza un valor o política de iteración para obtener las utilidades o una política óptima. ADP hace

un uso óptimo de las restricciones sobre utilidades de estados impuestas mediante la estructura de vecindad del entorno.
  - Los métodos de **diferencia temporal** (TD) actualizan las estimaciones de utilidad para que correspondan con las de los estados sucesores. Se puede ver como aproximaciones simples al enfoque ADP, que no requieren modelo para el proceso de aprendizaje. Sin embargo, utilizar un modelo aprendido para generar pseudoexperiencias puede dar como resultado un aprendizaje más rápido.

- Las funciones acción-valor, o funciones-Q, se pueden aprender mediante el enfoque ADP o mediante un enfoque TD. Con TD, el aprendizaje-Q no requiere modelo ni en la fase de aprendizaje ni en la de selección de acciones. Esto simplifica el problema de aprendizaje, pero restringe potencialmente la habilidad de aprender en entornos complejos, porque el agente no puede simular los resultados de los posibles cursos de acción.
- Cuando el agente de aprendizaje es responsable de seleccionar acciones mientras aprende, debe compensar entre el valor estimado de las acciones y el potencial de aprender nueva información útil. Una solución exacta para el problema de exploración es imposible, pero algunas heurísticas simples realizan un trabajo razonable.
- En espacios de estados grandes, los algoritmos de aprendizaje por refuerzo deben utilizar una representación funcional aproximada para generalizar sobre los estados. La señal de diferencia temporal se puede utilizar directamente para actualizar los parámetros en representaciones como redes neuronales.
- Los métodos de **búsqueda de la política** operan directamente sobre la política de representación, intentando mejorarla con base enl rendimiento observado. La varianza del rendimiento en un dominio estocástico es un problema serio; para dominios simulados se puede superar fijando la aleatoriedad de antemano.

Por su potencial para eliminar estrategias de control codificadas a mano, el aprendizaje por refuerzo continúa siendo una de las áreas más activas de investigación en aprendizaje automático. Las aplicaciones en robótica prometen ser particularmente valiosas; requerirán métodos para manejar entornos *continuos*, de *muchas dimensiones* y *parcialmente observables* en los que los comportamientos con éxito pueden consistir en miles o incluso millones de acciones primitivas.



NOTAS BIBLIOGRÁFICAS E HISTÓRICAS

Turing (1948, 1950) propuso el enfoque del aprendizaje por refuerzo, aunque no estaba convencido de su efectividad, escribiendo, «el uso de castigos o recompensas puede como mucho ser una parte del proceso de enseñanza». El trabajo de Arthur Samuel (1959), fue probablemente el primero con éxito en la investigación sobre aprendizaje automático. Aunque su trabajo fue informal y tenía algunos defectos, contenía la mayoría de las ideas modernas del aprendizaje por refuerzo. Hacia la misma época, los investigadores en teoría de control adaptativa (Widrow y Hoff, 1960), basándose en el trabajo de Hebb (1949), entrenaron redes simples con la regla delta. (Esta conexión inicial en-

tre las redes neuronales y el aprendizaje por refuerzo ha podido llevar a la persistente idea equivocada de que el segundo es un subcampo del primero.) El trabajo en el carro y la pértiga de Michie y Chambers (1968) también se puede ver como un método de aprendizaje por refuerzo con un aproximador de función. La literatura psicológica en aprendizaje por refuerzo es mucho más antigua; Hilgard y Bower (1975) proporcionan un buen resumen. Las investigaciones sobre el extraño comportamiento de las abejas han proporcionado la evidencia directa de la aplicación del aprendizaje por refuerzo en animales; hay una correlación neural clara de la señal de recompensa, en forma de una gran correspondencia neural a partir de los sensores de entrada de néctar, directamente con la corteza motora (Montague *et al*., 1995). La investigación utilizando grabación de celda única sugiere que el sistema de dopamina en cerebros de primates implementa algo parecido al aprendizaje de la función valor (Schultz *et al*. 1997).

La conexión entre el aprendizaje por refuerzo y los procesos de decisión de Markov fue inicialmente realizada por Werbos (1977), pero el desarrollo de aprendizaje por refuerzo en IA es resultado del trabajo en la Universidad de Massachusetts a comienzos de los años 80 (Barto *et al*., 1981). El artículo de Sutton (1988) proporciona un buen resumen histórico. La Ecuación (21.3) de este capítulo es un caso especial para $\lambda = 0$ del algoritmo general TD(λ) de Sutton. TD(λ) actualiza los valores de todos los estados en una secuencia, llevando a cada transición mediante una cantidad que disminuye como λ^t para estados de t pasos anteriores. TD(1) es idéntico a la regla de Widrow-Hoff o regla delta. Boyan (2002), basándose en el trabajo de Bradtke y Barto (1996) argumenta que TD(λ) y los algoritmos relacionados hacen un uso ineficiente de las experiencias; esencialmente, son algoritmos de regresión *en línea* que convergen mucho más lentamente que la regresión *fuera de línea*. Su algoritmo *en línea* LSTD(λ) proporciona los mismos resultados que la regresión *fuera de línea*.

La combinación del aprendizaje de diferencia temporal con la generación basada en modelos de experiencias simuladas fue propuesta en la arquitectura DYNA de Sutton (Sutton, 1990). La idea de barrido con prioridades fue introducida independientemente por Moore y Atkeson (1993) y Peng y Williams (1993). El aprendizaje-Q fue desarrollado en la tesis doctoral de Watkin (1989).

Los problemas del bandido, que modelizan el problema de exploración de decisiones no secuenciales, fueron analizados en profundidad por Berry y Fristedt (1985). Las estrategias óptimas de exploración para algunos esquemas se pueden obtener utilizando la técnica denominada **índices Gittins** (Gittins, 1989). Barto *et al*. (1995) discute una variedad de métodos de exploración para problemas de decisiones secuenciales. Kearns y Singh (1998) y Brafman y Tennenholtz (2000) describen algoritmos que exploran entornos desconocidos para los que está garantizada la convergencia a políticas cercanas a la óptima en un tiempo exponencial.

La aproximación de funciones en aprendizaje por refuerzo viene del trabajo de Samuel, que utilizó funciones de evaluación lineales y no lineales, y métodos de selección de características para reducir el espacio de las mismas. Los métodos posteriores incluyen el **CMAC** (*Cerebellar Model Articulation Controller*) (Albus, 1975), que esencialmente es una suma de funciones núcleo local que se superponen, y las redes neuronales asociativas de Barto *et al*. (1983). Actualmente, las redes neuronales son la forma más popular de aproximación de funciones. La aplicación que se conoce mejor es TD-Gam-

CMAC

mon (Tesauro, 1992, 1995), que se ha discutido en este capítulo. Un problema significatico que presenta el aprendizaje TD con redes neuronales es que tiende a olvidar las experiencias que han ocurrido hace tiempo, especialmente aquellas en partes del espacio de estados que se evitaron una vez que se ha conseguido competencia. Esto puede dar lugar a fallos catastróficos si reaparecen estas circunstancias. La aproximación de funciones basada en **aprendizaje basado en instancias** puede evitar este problema (Ormoneit y Sen, 2002; Forbes, 2002).

La convergencia de los algoritmos de aprendizaje por refuerzo que utilizan aproximación de funciones es una cuestión extremadamente técnica. Los resultados para aprendizaje TD han sido reforzados progresivamente para el caso de aproximadores lineales de funciones (Sutton, 1988; Dayan, 1992; Tsitsiklis y Van Roy, 1997), pero se han presentado algunos ejemplos de divergencia para funciones no lineales (veáse Tsitsiklis y Van Roy, 1997, para una discusión). Papavassiliou y Russell (1999) describen un nuevo tipo de aprendizaje por refuerzo que converge con cualquier forma de aproximador de función, dado que se puede encontrar una aproximación de mejor ajuste para los datos observados.

Los métodos de búsqueda de política fueron llevados a la fama por parte de Williams (1992), que desarrolló la familia de algoritmos REINFORCE. Los trabajos posteriores de Marbach y Tsitsiklis (1998), Sutton *et al*. (2000) y Baxter y Bartlett (2000) reforzaron y generalizaron los resultados de convergencia para la búsqueda de la política. El algoritmo PEGASUS se debe a Ng y Jordan (2000), aunque en la tesis doctoral de Van Roy (1998) aparecen técnicas similares. Como hemos mencionado en este capítulo, el rendimiento de una política *estocástica* es una función continua de sus parámetros, que se mejora con métodos de búsqueda basados en el gradiente. Este no es el único beneficio: Jaakkola *et al*. (1995) argumentan que realmente, las políticas estocásticas funcionan mejor que las políticas determinísticas en entornos parcialmente observables, si ambas se limitan a actuar con base en la percepción actual. (Una razón es que es menos probable que la política estocástica se atasque debido a algunos obstáculos que no se han visto.) En el Capítulo 17 apuntamos que las políticas óptimas en MDPs parcialmente observables son funciones determinísticas del *espacio de creencias* en vez de funciones de la percepción actual, por lo que aún esperaríamos mejores resultados manteniendo el espacio de creencias haciendo uso de los métodos de **filtrado** del Capítulo 15. Desafortunadamente, el espacio de estados de creencia es de dimensiones altas y continuo, y todavía no se han desarrollado algoritmos efectivos para aprendizaje con refuerzo con estados de creencia.

Los entornos del mundo real presentan una enorme complejidad, en términos del número de acciones primitivas que se requieren para conseguir una recompensa significativa. Por ejemplo, un robot puede realizar cientos de miles de movimientos de piernas antes de meter un gol. La **determinación de la recompensa** es un método común, que se utilizaba originalmente en entrenamiento de animales. Supone proporcionar al agente recompensas adicionales para «hacer progresos». Para el fútbol, éstas se pueden dar por hacer contacto con el balón o por lanzarlo hacia la portería. Estas recompensas pueden acelerar enormemente el aprendizaje, y se pueden incluir fácilmente, pero existe el riesgo de que el agente aprenda a maximizar las pseudo-recompensas en vez de las recompensas reales; por ejemplo, estar cerca del balón y «vibrar» causa muchos contac-

DETERMINACIÓN DE LA RECOMPENSA

tos con el mismo. Ng *et al*. (1999) muestran que el agente seguirá aprendiendo la política óptima dado que la pseuso-recompensa $F(s, a, s')$ satisface $F(s, a, s') = \gamma\Phi(s') = \Phi(s)$, donde Φ es una función arbitraria del estado. Se puede construir Φ de forma que refleje cualquier aspecto deseable del estado, tal como la consecución de submetas o la distancia a un estado meta.

APRENDIZAJE POR REFUERZO JERARQUICO

Los **métodos jerárquicos de aprendizaje por refuerzo**, que intentan resolver problemas en múltiples niveles de abstracción (muy parecidos a los **métodos de planificación HTN** del Capítulo 12), también pueden facilitar la generación de comportamientos complejos. Por ejemplo, «meter un gol» se puede descomponer en «obtener la posesión», «regatear hacia la portería» y «disparar»; y cada una de éstas se puede descomponer en más comportamientos motores de bajo nivel. Los resultados fundamentales en este área se deben a Forestier y Varaiya (1978), que demostraron que los comportamientos de bajo nivel de complejidad arbitraria se pueden tratar como acciones primitivas (aun aquellos que pueden variar en el tiempo) desde el punto de vista de los comportamientos de alto nivel que los invocan. Los enfoques actuales, Parr y Russell (1998), Dietterich (2000), Sutton *et al*. (2000), Andre y Russell (2002), se basan en este resultado para desarrollar

PROGRAMA PARCIAL

métodos que provean a un agente con un **programa parcial** que restringe el comportamiento del agente de forma que tenga una estructura jeráquica. Después, se aplica aprendizaje por refuerzo para aprender el mejor comportamiento consistente con el programa parcial. La combinación de aproximación de funciones, la *determinación* de la recompensa y el aprendizaje por refuerzo jerárquico permite enfrentarse a problemas de gran escala de forma satisfactoria.

El resumen de Kaelbling *et al*. (1996) es un buen punto de entrada a la literatura. El texto de Sutton y Barto (1998), dos de los pioneros del campo, se centra en las arquitecturas y algoritmos, mostrando cómo el aprendizaje por refuerzo entrelaza las ideas de aprendizaje, planificación y actuación. El trabajo algo más técnico de Bertsekas y Tsitsiklis (1996) proporciona de forma rigurosa los conocimientos básicos en la teoría de programación dinámica y convergencia estocástica. Los artículos de aprendizaje por refuerzo se publican frecuentemente en *Machine Learning*, *Journal of Machine Learning Research* y en las Conferencias Internacionales de *Machine Learning* y *Neural Information Processing Systems*.





EJERCICIOS

21.1 Implemente un agente de aprendizaje pasivo en un entorno simple como el mundo 4×3. Para el caso de un modelo del entorno inicialmente desconocido, compare el rendimiento del aprendizaje de los algoritmos de estimación directa de la utilidad, TD y ADP. Realice la comparación para la política óptima y para varias políticas aleatorias. ¿Para cuál converge más rápidamente la estimación de la utilidad? ¿Qué pasa cuando se aumenta el tamaño del entorno? (Intente entornos con y sin obstáculos).

21.2 El Capítulo 17 definía una **política adecuada** para un MDP, como aquella para la que está garantizado alcanzar un estado terminal. Muestre que para un agente ADP pasivo es posible aprender un modelo de transición para el que su política π es inade-

cuada (incluso si π es adecuada para el MDP verdadero; con estos modelos, el paso de determinación de valores puede fallar si $\gamma = 1$). Muestre que este problema no puede surgir si la determinación de valores se aplica al modelo aprendido sólo al final de una prueba.

21.3 Modifique el agente ADP pasivo para que utilice un algoritmo ADP aproximado como el discutido en el texto. Haga esto en dos pasos:

- ***a***) Implemente una cola de prioridades para ajustes de la estimación de la utilidad. Siempre que se ajusta un estado, todos sus predecesores también se convierten en candidatos para ajuste y deben ser añadidos a la cola. La cola se inicializa con el estado para el que ha tenido lugar la transición más reciente. Permita sólo un número fijo de ajustes.
- ***b***) Experimente con varias heurísticas para ordenar la cola de prioridades, examinando su efecto en las tasas de aprendizaje y el tiempo de computación.

21.4 El método de estimación directa de la utilidad del Apartado 21.1 utiliza estados terminales distintos para indicar el final de una prueba. ¿Cómo se podría modificar para entornos con descuento de recompensas y sin estados terminales?

21.5 ¿Cómo se puede utilizar el algoritmo de determinación de valores para calcular la pérdida esperada experimentada por un agente que utiliza un conjunto dado de estimaciones de utilidad U y un modelo estimado M, comparado con un agente que usa los valores correctos?

21.6 Adapte el mundo de la aspiradora (Capítulo 2) para aprendizaje por refuerzo incluyendo recompensas por recoger cada porción de suciedad, por guardarse y por apagarse. Haga el mundo accesible proporcionando las percepciones adecuadas. Ahora experimente con distintos agentes de aprendizaje por refuerzo. ¿Es la aproximación de funciones necesaria para el éxito? ¿Qué tipo de aproximador funciona para esta aplicación?



21.7 Implemente un agente de exploración de aprendizaje por refuerzo que utilice la estimación directa de la utilidad. Construya dos versiones: una con una representación tabular y otra que utilice el aproximador de función de la Ecuación (21.9). Compare su rendimiento en tres entornos.

- ***a***) El mundo 4×3 descrito en este capítulo.
- ***b***) El mundo 10×10 sin obstáculos y con una recompensa de +1 en (10, 10).
- ***c***) El mundo 10×10 sin obstáculos y con una recompensa de +1 en (5, 5).

21.8 Escriba la actualización de parámetros para el aprendizaje TD con

$$\hat{U}(x, y) = \theta_0 + \theta_1 x + \theta_2 y + \theta_3 \sqrt{(x - x_g)^2 + (y - y_g)^2}$$

21.9 Invente características adecuadas para un mundo de cuadrícula estocástica (generalización del mundo 4×3) que contenga múltiples obstáculos y estados terminales múltiples con recompensas de +1 o −1.

21.10 Calcule la función de utilidad verdadera y la mejor aproximación lineal en x e y (como en la Ecuación (21.9)) para los siguientes entornos:

a) Un mundo 10×10 con un único estado terminal +1 en (10, 10).
b) Como (a), pero añadiendo un estado terminal −1 en (10, 1).
c) Como (b), pero añadiendo obstáculos en 10 cuadrados seleccionados aleatoriamente.
d) Como (b), pero situando una pared en el tramo que va desde (5, 2) a (5, 9).
e) Como (a), pero con el estado terminal en (5, 5).

Las acciones son movimientos determinísticos en las cuatro direcciones. En cada caso, compare los resultados utilizando gráficos tridimensionales. Para cada entorno, proponga características adicionales (además de x e y) que mejorarían la aproximación y muestre los resultados.

21.11 Extienda el entorno estándar de juegos (Capítulo 6) para que incorpore una señal de recompensa. Sitúe dos agentes de aprendizaje por refuerzo en el entorno (por supuesto, pueden compartir el programa de agente) y deje que jueguen uno contra el otro. Aplique la regla de actualización TD generalizada (Ecuación (21.11)) para actualizar la función de evaluación. Podría empezar con una simple función de evaluación lineal ponderada y un juego sencillo como el tic-tac-toe.



21.12 Implemente los algoritmos REINFORCE y PEGASUS y aplíquelos al mundo 4×3, utilizando una familia de políticas de su propia elección. Comente los resultados.



21.13 Investigue la aplicación de las ideas de aprendizaje por refuerzo para modelizar el comportamiento humano y animal.



21.14 ¿Es el aprendizaje por refuerzo un modelo abstracto apropiado para la evolución?¿Qué conexión existe, si existe alguna, entre señales de recompensa cableadas y la capacidad evolutiva?



22 La comunicación

En este capítulo veremos por qué los agentes pueden querer intercambiar mensajes que lleven información y cómo lo pueden hacer.

Anochece en los bosques de la sabana del Parque Nacional de Amboseli a los pies del Kilimanjaro. Un grupo de monos vervet («vervet monkeys») está buscando comida cuando uno de ellos emite un ruidoso aullido. Los otros individuos del grupo lo reconocen como el aviso de la presencia de un leopardo (distinto del corto carraspeo usado para prevenir de las águilas o del chasquido para las serpientes) y trepan por los árboles. El vervet se ha comunicado satisfactoriamente con el grupo.

COMUNICACIÓN

SIGNOS

La **comunicación** es el intercambio intencional de información efectuado mediante la producción y percepción de **signos** pertenecientes a un sistema compartido de signos convencionales. La mayoría de los animales usan signos para representar mensajes importantes: comida aquí, depredador cercano, acercamiento, retirada, compañero. En un mundo observable parcialmente la comunicación puede ayudar a los agentes a prosperar ya que pueden adquirir información que es observada o inferida por los demás.

LENGUAJE

Lo que hace diferente a los humanos de los otros animales es el complejo sistema de mensajes estructurados, que conocemos como **lenguaje**, y que nos permite comunicar la mayor parte de las cosas que sabemos acerca del mundo. Aunque los chimpancés, los delfines y otros mamíferos han dado muestra de manejar vocabularios de cientos de signos y cierta aptitud para encadenarlos, únicamente los humanos pueden comunicar fiablemente un número ilimitado de mensajes cualitativamente diferentes.

Desde luego, hay otros atributos que son exclusivamente humanos: ninguna otra especie viste ropas, crea arte figurativo o ve tres horas diarias la televisión. Pero cuando Turing propuso su prueba (*véase* Apartado 1.1) la basó en el lenguaje, ya que el lenguaje está íntimamente vinculado con el pensamiento. En este capítulo explicaremos cómo trabaja y describe un fragmento de castellano un agente comunicativo.

22.1 La comunicación como acción

ACTO DE HABLA

Una de las acciones que puede realizar un agente es la de producir lenguaje. Esta acción es llamada un **acto de habla**. «Habla» es usado en el mismo sentido que «libertad de palabra», no estrictamente hablado de forma que, el correo electrónico, las señales de humo y el uso del lenguaje de signos son considerados todos ellos como actos de habla. En el idioma inglés no existe una palabra específica para designar a un agente que produce lenguaje sea cual sea el medio, por ello usaremos **hablante**, **oyente** y **declaración** como términos genéricos para referirnos a cualquier modo de comunicación. También emplearemos el término **palabra** para referirnos a cualquier clase de signo comunicativo convencional.

HABLANTE

OYENTE

DECLARACIÓN

PALABRA

¿Por qué querrá un agente molestarse en realizar un acto de habla cuando puede hacer una acción «normal»? Hemos visto en el Capítulo 12 que los agentes en un entorno multiagente pueden servirse de la comunicación para lograr planes comunes. Por ejemplo, un grupo de agentes que está explorando simultáneamente el mundo de *wumpus*[1] obtiene más ventajas (colectiva e individualmente) si es capaz de:

- **Preguntar** a otros agentes acerca de aspectos particulares del mundo. Esto se suela hacer con cuestiones como: *¿Has olfateado al wumpus en alguna parte?*
- **Informarse** entre sí acerca del mundo. Esto se hace con declaraciones representativas: *Hay una brisa aquí, en 3 4*. La respuesta a una pregunta es otra clase de acción informativa.
- **Solicitar** a otros agentes la realización de acciones: *Por favor, ayúdame a llevar el oro*. Algunas veces se considera más correcto usar un **acto de habla indirecto** (una solicitud en forma de declaración o pregunta): *Podría utilizar alguna ayuda para cargar esto*. Un agente con autoridad puede dar órdenes (*Alpha ve a la derecha; Bravo y Charlie id a la izquierda*) y un agente con poder puede amenazar (*Dame el oro, pues si no…*). A esta clase de actos de habla se les llama **directivas**.
- **Aceptación** de peticiones: *De acuerdo*.
- **Prometer** o comprometerse a un plan: *Yo dispararé al wumpus; mientras tú coge el oro*.

ACTO DE HABLA INDIRECTO

Todos los actos de habla afectan al mundo al hacer vibrar las moléculas del aire (o el efecto equivalente en cualquier otro medio) y cambiando el estado mental y finalmente las acciones futuras de otros agentes. Algunos tipos de actos de habla transfieren información al oyente, asumiendo que la toma de decisiones del oyente se verá afectada por dicha información. Otros tienen la intención directa de hacer que el oyente haga alguna acción. Otra clase de actos de habla, las **declaraciones**, surgen para tener un efecto más directo sobre el mundo, como en *Os declaro marido y mujer* o *tres, estás fuera*. Desde luego, el efecto es conseguido creando o confirmando una compleja red de estados mentales entre los agentes involucrados: estar casado o estar fuera del juego son estados caracterizados principalmente por convenciones más que por propiedades «físicas» del mundo.

DECLARACIONES

[1] *Nota del traductor: Wumpus* es el monstruo central de una familia de videojuegos de los años 70 llamado «Hunt The Wumpus».

La tarea de los agentes que se comunican es decidir *cuándo* se le solicita un acto de habla de alguna clase y *qué* acto de habla, de entre todas las posibilidades, es el correcto. El problema de entender los actos de habla es similar a otros problemas de **comprensión**, tales como interpretar imágenes o diagnosticar enfermedades. Se nos da un conjunto de entradas ambiguas y a partir de ellas debemos retroceder para decidir qué estado del mundo pudo haber creado esas entradas. Sin embargo, debido a que el habla es una acción planificada, su comprensión también implica el reconocimiento de planes.

COMPRENSIÓN

Fundamentos del lenguaje

LENGUAJE FORMAL

CADENAS

SÍMBOLOS TERMINALES

LENGUAJES NATURALES

Un **lenguaje formal** se define como un (posiblemente infinito) conjunto de **cadenas**. Cada cadena es a su vez una concatenación de **símbolos terminales**, algunas veces llamados palabras. Por ejemplo, en el lenguaje de la lógica de primer orden, $\wedge$ y P son símbolos terminales y «$P \wedge Q$» es una cadena típica. La cadena «$P\ Q\ \wedge$» no pertenece al lenguaje. Los lenguajes formales, tales como la lógica de primer orden o Java, tienen definiciones matemáticas estrictas. Esto contrasta con los **lenguajes naturales**, tales como chino, danés, inglés y castellano, que no tienen definiciones estrictas y son usados por una comunidad de hablantes. En este capítulo intentaremos tratar los lenguajes naturales como si ellos fueran lenguajes formales, aunque reconocemos que el emparejamiento no será perfecto.

GRAMÁTICA

Una **gramática** es un conjunto finito de reglas que especifican un lenguaje. Los lenguajes formales siempre tienen una gramática oficial, especificada en manuales o libros. Los lenguajes naturales tienen gramáticas no oficiales, pero los lingüistas se afanan en descubrir propiedades del lenguaje por medio de un proceso de investigación científica y entonces codificar sus descubrimientos en una gramática. Hasta la fecha, ningún lingüista lo ha conseguido completamente. Obsérvese que los lingüistas son científicos, que intentan definir cómo *es* un lenguaje. Hay también gramáticos prescriptivos, los cuales intentan dictar cómo *debe ser* un lenguaje. Ellos crean reglas tales como «No dividir infinitivos» que son algunas veces escritas en guías de estilo, pero que tienen poca relevancia en el uso actual del lenguaje.

Tanto los lenguajes formales como los naturales asocian un significado o **semántica** a cada cadena válida. Por ejemplo, en el lenguaje de la aritmética podemos tener una regla que diga que si «X» e «Y» son expresiones, entonces «$X + Y$» es también una expresión y su semántica es la suma de X e Y. En los lenguajes naturales es además importante entender la **pragmática** de una cadena: el significado real de la cadena cuando es dicha en una situación determinada. El significado no está solamente en las palabras en sí mismas, sino en la interpretación de las palabras *in situ*.

PRAGMÁTICA

ESTRUCTURA DE LA FRASE

FRASE SUSTANTIVA

La mayoría de los formalismos para reglas gramaticales están basados en la idea de **estructura de la frase** (que las cadenas están compuestas de subcadenas llamadas **frases**, las cuales pueden pertenecer a distintas categorías). Por ejemplo, las frases «el *wumpus*», «el rey» y «el agente de la esquina» son todas ellas ejemplos de la categoría **frase nominal** o *FN*. Hay dos razones para identificar a las frases de esta manera. Primera, las frases normalmente corresponden a elementos semánticos naturales a partir de los cuales puede ser interpretado el significado de una declaración; por ejemplo, las fra-

ses nominales hacen referencia a objetos del mundo. Segundo, el categorizar las frases nos ayuda a describir las cadenas permitidas del lenguaje. Podemos decir que cualquier frase nominal puede combinarse con una **frase verbal** (o *FV*), tal como «está muerto», para formar una frase de la categoría **sentencia** (o *S*). Sin las nociones intermedias de frase nominal y frase verbal sería difícil explicar por qué «el *wumpus* está muerto» es una sentencia mientras que «*wumpus* el muerto es» no lo es.

FRASE VERBAL

SENTENCIA

A los nombres de las categorías, tales como *FN*, *FV* y *S*, les llamamos **símbolos no terminales**. Las gramáticas definen los no terminales por medio de **reglas de reescritura**. Adoptaremos la notación de la forma Backus-Naur (BNF) para las reglas de reescritura, tal como se describe en el Apéndice B en la página 1.117. En esta notación, el significado de una regla como

SÍMBOLOS NO TERMINALES

REGLAS DE REESCRITURA

$$S \rightarrow FN \ FV$$

es que una sentencia puede consistir en una *FN* seguida por cualquier *FV*.

Etapas de la comunicación

Un episodio típico de comunicación, en el cual un hablante *H* quiere informar a un oyente *O* acerca de una sentencia *S* utilizando palabras *P*, está compuesto de siete procesos:

INTENCIÓN

Intención. Por alguna razón, un hablante *H* decide que hay una sentencia *S* que merece ser dicha al oyente *O*. En nuestro ejemplo, el hablante tiene la intención de hacer saber al oyente que el *wumpus* no vivirá más.

GENERACIÓN

Generación. El hablante planifica cómo convertir la sentencia *S* en una declaración que probablemente hará que el oyente, al percibir la declaración en la situación actual, pueda inferir el significado *S* (o algo parecido). Se asume que el hablante es capaz de proponer las palabras «El *wumpus* está muerto» y decirlas.

SÍNTESIS

Síntesis. El hablante produce la realización física P' de las palabras *P*. Esto puede ser vía tinta sobre papel, vibraciones en el aire o cualquier otro medio. En la Figura 22.1, mostramos un agente que sintetiza una cadena de sonidos P' escritos en el alfabeto fonético definido en el Apartado 15.6: «[elwumpusestamuerto]». Las palabras están juntas; esto es típico del habla rápida.

PERCEPCIÓN

Percepción. *O* percibe la realización física P' como P'_2 y la decodifica como las palabras P_2. Cuando el medio es hablado, el paso de percepción se llama **reconocimiento del habla**; cuando es impreso, se llama **reconocimiento óptico de caracteres**. Ambos pasaron en los años 1990 de ser esotéricos a ser temas comunes, debido en gran parte a la creciente potencia de cálculo de los computadores.

ANÁLISIS

Análisis. *O* infiere que P_2 tiene posibles significados $S_1, \ldots, S_n$. Dividimos el análisis en tres partes principales: interpretación sintáctica (o análisis sintáctico), interpretación semántica e interpretación pragmática. El **análisis sintáctico** es el proceso de construir el **árbol sintáctico** para una cadena de entrada, como se muestra en la Figura 22.1. Los nodos interiores del árbol sintáctico representan frases y los nodos hoja representan palabras. La **interpretación semántica** es el proceso de extracción del significado de una declaración como una expresión en algún lenguaje de representación. La Figura 22.1 muestra dos posibles interpretaciones semánticas: que el *wumpus* no está vivo y que el *wumpus* está agotado (un significado coloquial de *muerto*). Se dice que las declaraciones con varias interpretaciones posibles son **ambiguas**. La **interpretación pragmática** tiene en

ANÁLISIS SINTÁCTICO

ÁRBOL SINTÁCTICO

INTERPRETACIÓN SEMÁNTICA

INTERPRETACIÓN PRAGMÁTICA

CAPACIDAD GENERATIVA

Los formalismos gramaticales pueden ser clasificados por su **capacidad generativa**: el conjunto de lenguajes que pueden representar. Chomsky (1957) describe cuatro clases de formalismos gramaticales que difieren únicamente en la forma de las reglas de reescritura. Las clases pueden ser organizadas jerárquicamente, donde cada clase puede ser usada para describir todos los lenguajes que pueden ser descritos por una clase menos potente, así como algunos lenguajes adicionales. Vamos a listar la jerarquía; aparece primero la clase más potente:

Las **gramáticas recursivamente enumerables** usan reglas que no tienen restricciones: ambos lados de las reglas de reescritura pueden tener cualquier número de símbolos terminales y no terminales, como en la regla $A\ B \rightarrow C$. Estas gramáticas son equivalentes a las máquinas de Turing en cuanto a su potencia expresiva.

Las **gramáticas dependientes del contexto** están restringidas únicamente a que la parte derecha debe contener al menos tantos símbolos como la parte izquierda. El nombre «dependiente del contexto» procede del hecho de que una regla tal como $A\ S\ B \rightarrow A\ X\ B$ dice que un S puede ser reescrito como un X en el contexto de un A precediéndole y un B siguiéndole. Las gramáticas dependientes del contexto pueden representar lenguajes tales como $a^n\ b^n\ c^n$ (una secuencia de n copias de a seguida por el mismo número de b y de c).

En las **gramáticas independientes del contexto** (o **GIC**), el lado izquierdo consta de un único símbolo no terminal. Así, cada regla permite rescribir el no terminal como el lado derecho en *cualquier* contexto. Las GIC son muy utilizadas como gramáticas de lenguajes de programación y lenguajes naturales, aunque ahora está ampliamente aceptado que al menos algunos lenguajes naturales tienen construcciones que no son independientes del contexto (Pullum, 1991). Las gramáticas independientes del contexto pueden representar $a^n\ b^n$, pero no $a^n\ b^n\ c^n$.

Las **gramáticas regulares** son la clase más restringida. Cada regla tiene un único no terminal en el lado izquierdo y un símbolo terminal opcionalmente seguido por un no terminal en el lado derecho. Las gramáticas regulares son equivalentes en potencia a las máquinas de estados finitos. Se ajustan mal a los lenguajes de programación, debido a que no pueden representar construcciones tales como aperturas y cierres balanceados de paréntesis (una variación del lenguaje $a^n\ b^n$). Lo más cercano que pueden llegar es representando $a^*\ b^*$, una secuencia de cualquier número de a seguido de cualquier número de b.

Las gramáticas más altas en la jerarquía tienen más poder expresivo, pero los algoritmos para tratar con ellas son menos eficientes. Hasta mitad de los 80, los lingüistas se concentraron en los lenguajes independientes del contexto y dependientes del contexto. Desde entonces, ha crecido la importancia de las gramáticas regulares, ocasionado por la necesidad de procesar muy rápidamente megabytes y gigabytes de texto *en línea*, incluso a costa de un menor análisis completo. Como Fernando Pereira expresó, «Cuanto más viejo me vuelvo, más abajo voy en la jerarquía de Chomsky». Para ver lo que él quiere decir, compare Pereira y Warren (1980) con Mohri, Pereira y Riley (2002).

cuenta el hecho de que las mismas palabras pueden tener diferentes significados en diferentes situaciones. Mientras la interpretación sintáctica es una función de un argumento, la cadena, la interpretación pragmática es función de la declaración y del contexto o situación en la cual es enunciada. En el ejemplo, la pragmática hace dos cosas: reem-

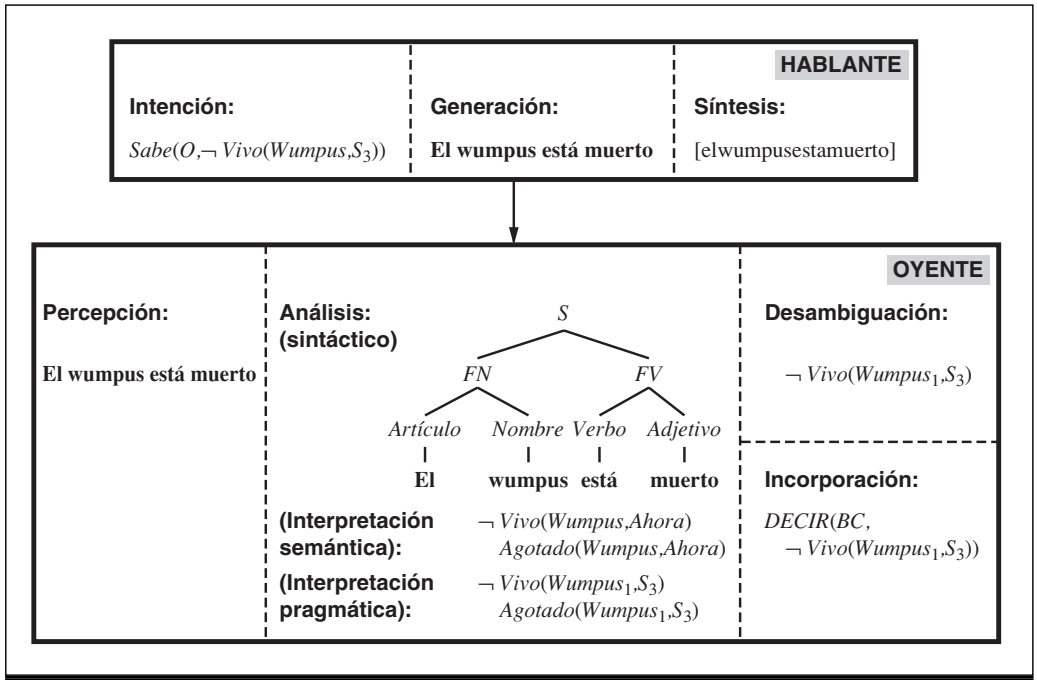


Figura 22.1 Los siete procesos implicados en la comunicación, usando la sentencia ejemplo «El *wumpus* está muerto»

plaza la constante *Ahora* por la constante S_3, que representa la situación actual, y reemplaza *Wumpus* por $Wumpus_1$, que representa al único *wumpus* que sabemos que está en la cueva. En general, la pragmática puede contribuir mucho más a la interpretación final de la declaración; consideremos «Estoy mirando el diamante» cuando es dicho por un joyero o por un jugador de béisbol. En el Apartado 22.7 veremos que la pragmática nos permite interpretar «Está muerto» como que el *wumpus* está muerto si nosotros estamos en una situación donde el *wumpus* está saliente.

DESAMBIGUACIÓN

Desambiguación. O infiere que H intenta expresar S_i (donde idealmente $S_i = S$). La mayoría de los hablantes no son intencionalmente ambiguos, pero muchas declaraciones tienen varias interpretaciones factibles. La comunicación funciona porque el oyente hace la labor de imaginarse qué interpretación es la que probablemente pretende expresar el hablante. Conviene destacar que esta es la primera vez que hemos usado la palabra *probablemente*, y que la desambiguación es el primer proceso que depende fuertemente del razonamiento incierto. El análisis genera interpretaciones posibles; si se encuentra más de una interpretación, entonces la desambiguación elige aquella que considera la mejor.

INCORPORACIÓN

Incorporación. O decide creer S_i (o no). Un agente totalmente ingenuo deberá creer todas las cosas que oiga, pero un agente sofisticado considerará el acto de habla como evidencia para S_i, no como confirmación de ello.

Al ponerlo todo junto, obtenemos el programa agente mostrado en la Figura 22.2. Aquí el agente actúa como un robot esclavo que puede ser controlado por un amo. En cada turno, el esclavo responderá una pregunta o cumplirá una orden que le haya dado

```
función AGENTE-COMUNICATIVO-INGENUO (percepción) devuelve acción
    estático: BD, base de datos
              estado, el estado actual del retorno
              acción, la acción más reciente, inicialmente ninguna

estado ← ACTUALIZAR-ESTADO(estado, acción, percepción)
palabras ← PARTE-HABLADA(percepción)
semántica ← DESAMBIGUACIÓN(PRAGMÁTICA(SEMÁNTICA(ANALIZADOR(palabras))))
si palabras = Ninguna y no se dice acción entonces /* Describe el estado */
    devuelve DECIR(GENERAR-DESCRIPCIÓN(estado))
sino si TIPO[semántica] = Orden entonces /* Obedece la orden */
    devuelve CONTENIDOS[semántica]
sino si TIPO[semántica] = Pregunta entonces /* Contesta la pregunta */
    respuesta ← PREGUNTAR(BD, semántica)
    devuelve DECIR(GENERAR-DESCRIPCIÓN(respuesta))
sino si TIPO[semántica] = Sentencia entonces /* Cree la sentencia */
    NOTAR(BD, CONTENIDOS[semántica])
/* Si se fracasa aquí, hacer una acción «regular» */
    devuelve PRIMERO(PLANIFICADOR(BD, estado))
```

Figura 22.2 Un agente comunicativo que acepta órdenes, preguntas y sentencias. El agente puede también describir el estado actual o realizar una acción que no sea un acto de habla «regular» cuando no tiene nada que decir.

el amo y creerá cualquier afirmación hecha por el amo. También comentará la situación actual si no tiene nada más urgente que hacer y planificará su propia acción si se queda solo. Aquí tenemos un diálogo típico:

ROBOT ESCLAVO	AMO
Siento una brisa.	Ve a 1 2.
Aquí no hay nada.	Ve al norte.
Siento una brisa, huelo un hedor y veo un resplandor.	Toma el oro.

22.2 Una gramática formal para un fragmento del español

En esta sección definimos una gramática formal para un pequeño fragmento del inglés (castellano) que es adecuado para hacer afirmaciones sobre el mundo del *wumpus*. A este lenguaje le llamaremos ε_0. Secciones posteriores mejorarán ε_0 para hacerlo algo más cercano al español real. Es improbable idear una gramática completa para el español, aunque sólo sea porque dos personas no se pondrían totalmente de acuerdo sobre lo que constituye español válido.

El léxico de ε_0

LÉXICO

Primero definiremos el **léxico** o lista de palabras permitidas. Las palabras se agrupan en categorías o partes del habla conocidas por los usuarios del diccionario: sustantivos, pronombres y nombres para denotar cosas, verbos para denotar sucesos, adjetivos para modificar sustantivos y adverbios para modificar verbos. Las categorías que quizá sean menos familiares para algunos lectores son los artículos (tales como *el*), preposiciones (*en*) y conjunciones (*y*). La Figura 22.3 muestra un pequeño léxico.

Cada una de las categorías finaliza con … para indicar que hay otras palabras en la categoría. Sin embargo, debe hacerse notar que hay dos razones distintas para la falta de palabras. En principio no es factible listar todos los sustantivos, verbos, adjetivos y adverbios. No hay decenas de miles de miembros en cada clase, sino porque constantemente son añadidos otros nuevos (como *MP3* o *anime*). Estas cuatro categorías son llamadas **clases abiertas**. Las otras categorías (pronombres, artículos, preposiciones y conjunciones) son llamadas **clases cerradas**. Estas tienen un pequeño número de palabras (desde unas pocas a unas pocas docenas) que pueden en principio ser enumeradas completamente. Las clases cerradas cambian con el paso de los siglos, no de meses. Por ejemplo, «thee» y «thou»[2] fueron pronombres habitualmente usados en el siglo XVII, fueron decayendo en el siglo XIX y hoy en día sólo son vistos en poesía y algunos dialectos regionales.

CLASES ABIERTAS

CLASES CERRADAS

Sustantivo	→	**hedor** | **brisa** | **resplandor** | **nada** | **agente** | ***wumpus*** | **foso** | **oro** | **este** | …
Verbo	→	**es** | **ver** | **oler** | **disparar** | **sentir** | **heder** | **ir** | **tomar** | **llevar** | **matar** | **girar** | …
Adjetivo	→	**derecho** | **izquierdo** | **oriental** | **muerto** | **posterior** | **hediondo** | …
Adverbio	→	**aquí** | **allí** | **cerca** | **adelante** | **correctamente** | **a la izquierda** | **al este** | **al sur** | **atrás** | …
Pronombre	→	**mí** | **tú** | **yo** | **ello** | …
Nombre	→	**Juan** | **María** | **Alicante** | **Aristóteles** | …
Artículo	→	**el** | **un** | …
Preposición	→	**a** | **en** | **sobre** | …
Conjunción	→	**y** | **o** | **pero** | …
Dígito	→	**0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**

Figura 22.3 El léxico para ε_0.

La Gramática de ε_0

El siguiente paso es combinar las palabras en frases. Usaremos cinco símbolos no terminales para definir las diferentes clases de frases: sentencia (*S*), frase nominal (*FN*), frase verbal (*FV*), frase preposicional (*FP*) y oración relativa (*OracionRel*)[3]. La Figu-

[2] *Nota del traductor:* ti y tú en inglés antiguo.

[3] Una oración relativa sigue y modifica a una frase nominal. Consta de un pronombre relativo (tal como «quien» o «que») seguido por una frase verbal. (En el Ejercicio 22.12 se discute otra clase de oración relativa.) Un ejemplo de oración relativa es *que huele mal* en «El *wumpus* que huele mal está en 2 2».

S	→	*FN FV*	Yo + siento una brisa
	|	*S Conjunción S*	Yo siento una brisa + y + yo huelo a *wumpus*
FN	→	*Pronombre*	Yo
	|	*Nombre*	Juan
	|	*Sustantivo*	foso
	|	*Artículo* del *sustantivo*	el + *wumpus*
	|	*Dígito Dígito*	3 4
	|	*FN FP*	el *wumpus* + al este
	|	*FN OracionRel*	el *wumpus* + que es hediondo
FV	→	*Verbo*	huele mal
	|	*FV FN*	siento + una brisa
	|	*FV Adjetivo*	es hediondo
	|	*FV FP*	gira + al este
	|	*FV Advervio*	ve + adelante
FP	→	*Preposición FN*	a + el este
OracionRel	→	**que** *FV*	que + es hediondo

Figura 22.4 La gramática para $\mathcal{E}_0$, con frases de ejemplo para cada regla.

ra 22.4 muestra una gramática para ε_0, con un ejemplo para cada regla de reescritura. ε_0 genera sentencias correctas de español como:

Juan está en el foso
El *wumpus* que huele mal está en 2 2
María está en Boston y Juan huele mal

SOBREGENERA

SUBGENERACIÓN

Desafortunadamente, la gramática **sobregenera**: esto es, genera sentencias que no son gramaticales, tales como «Mi ir Boston» y «Yo huelo foso oro *wumpus* nada este». También **subgenera** hay muchas sentencias de inglés (castellano) que rechaza, tales como «Yo pienso que el *wumpus* es hediondo». (Otra deficiencia es que la gramática no pone en mayúsculas la primera palabra de una sentencia, ni añade puntuación al final. Esto es porque está diseñada principalmente para hablar, no para escribir.)

22.3 Análisis sintáctico

Hemos definido anteriormente el **análisis sintáctico** como el proceso de encontrar un árbol sintáctico para una cadena de entrada dada. Esto es, una llamada a la función ANÁLISISSINTÁCTICO, tal que

ANÁLISISSINTÁCTICO («el *wumpus* está muerto», ε_0, *S*)

debe devolver un árbol sintáctico con raíz *S* cuyas hojas son «el *wumpus* está muerto» y cuyos nodos internos son símbolos no terminales de la gramática ε_0. Se puede ver dicho árbol en la Figura 22.1. En texto lineal, escribimos el árbol como

[*S*: [*FN*: [*Artículo*: **el**][*Sustantivo*: ***wumpus***]]
[*FV*: [*Verbo*: **está**][*Adjetivo*: **muerto**]]]

El análisis sintáctico puede ser visto como un proceso de búsqueda del árbol sintáctico. Hay dos formas extremas de especificar el espacio de búsqueda (y muchas variantes entre ellas). En la primera, podemos empezar con símbolo *S* y buscar un árbol que tenga las palabras en sus hojas. Este modo se llama **análisis descendente** (ya que *S* es dibujado en lo alto del árbol). En la segunda, podemos empezar con las palabras y buscar un árbol con raíz *S*. Este modo se llama **análisis ascendente**[4]. El análisis descendente puede ser definido precisamente como un problema de búsqueda tal que:

ANÁLISIS ASCENDENTE

ANÁLISIS DESCENDENTE

- El **estado inicial** es un árbol sintáctico que consta de la raíz *S* y un hijo desconocido: [*S*: ?]. En general, cada estado del espacio de búsqueda es un árbol sintáctico.
- La **función sucesor** selecciona el nodo más a la izquierda del árbol con hijo desconocido. Entonces busca en la gramática reglas que tengan la etiqueta raíz del nodo en el lado izquierdo. Por cada una de estas reglas, crea un estado sucesor donde el ? es reemplazado por la lista correspondiente al lado derecho de la regla. Por ejemplo, en ε_0 hay dos reglas para *S*, así que el árbol [*S*: ?] podría ser reemplazado por los dos sucesores siguientes:

  [*S*: [*S*: ?][*Conjunción*: ?][*S*: ?]]
  [*S*: [*FN*: ?][*FV*: ?]]

  La segunda de estas tiene siete sucesores, uno por cada regla de reescritura de *FN*.
- La **prueba (criterio) de finalización** controla que las hojas del árbol sintáctico correspondan exactamente a la cadena de entrada, sin entradas desconocidas ni descubiertas.

Un gran problema para el análisis descendente es hacer frente a las llamadas **reglas recursivas por la izquierda**, esto es, reglas de la forma $X \rightarrow X...$ Con la búsqueda primero en profundidad, dichas reglas podrían llevarnos a seguir reemplazando *X* con [*X*: *X*…] en un bucle infinito. Con la búsqueda de primero en anchura podríamos encontrar exitosamente árboles sintácticos para sentencias válidas, pero cuando se nos da una sentencia inválida, podríamos quedar atascados en un espacio de búsqueda infinito.

La formulación del análisis ascendente como una búsqueda es como sigue:

- El **estado inicial** es una lista de las palabras de la cadena de entrada, cada una vista como un árbol sintáctico consistente en un único nodo hoja, por ejemplo [**el**, ***wumpus***, **está**, **muerto**]. En general, cada estado del espacio de búsqueda es una lista de árboles sintácticos.

[4] El lector debe darse cuenta de que el análisis descendente y el ascendente son análogos al encadenamiento hacia atrás y hacia delante, respectivamente, descritos en el Capítulo 7. Veremos brevemente que la analogía es exacta.

- La **función sucesor** mira en cada posición *i* de la lista de árboles y en cada lado derecho de las reglas de la gramática. Si la subsecuencia de la lista de árboles que empieza en *i* empareja con el lado derecho, entonces la subsecuencia es reemplazada por un nuevo árbol cuya categoría es el lado izquierdo de la regla y cuyo hijo es la subsecuencia. Por «emparejar» queremos decir que la categoría del nodo es la misma que el elemento del lado derecho. Por ejemplo, la regla *Artículo* → **el** empareja la subscuencia que consta del primer nodo de la lista [**el**, ***wumpus***, **está**, **muerto**], así que un estado sucesor podría ser [[*Artículo*: **el**], ***wumpus***, **está**, **muerto**].
- La **prueba (criterio) de finalización** comprueba el estado que conste de un árbol sencillo con raíz *S*.

En la Figura 22.5 podemos ver un ejemplo de un análisis ascendente.

paso	*lista de nodos*	*subsecuencia*	*regla*
INIT	**el *wumpus* está muerto**	**el**	*Artículo* → **el**
2	*Artículo* ***wumpus*** **está muerto**	***wumpus***	*Sustantivo* → ***wumpus***
3	*Artículo Sustantivo* **está muerto**	*Artículo Sustantivo*	*FN* → *Artículo Sustantivo*
4	*FN* **está muerto**	**está**	*Verbo* → **está**
5	*FN Verbo* **muerto**	**muerto**	*Adjetivo* → **muerto**
6	*FN Verbo Adjetivo*	*Verbo*	*FV* → *Verbo*
7	*FN FV Adjetivo*	*FV Adjetivo*	*FV* → *FV Adjetivo*
8	*FN FV*	*FN FV*	*S* → *FN FV*
OBJETIVO	*S*		

Figura 22.5 Traza de un análisis ascendente de la cadena «El *wumpus* está muerto». Empezamos con una lista de nodos consistentes en palabras. Después reemplazamos subsecuencias que emparejan el lado derecho de una regla con un nuevo nodo cuya raíz es el lado izquierdo. Por ejemplo, en la tercera línea los nodos *Artículo* y *Sustantivo* son reemplazados por un nodo *FN* que tiene estos dos nodos como hijos. El análisis descendente produciría una traza similar, pero en la dirección opuesta.

Tanto el análisis ascendente como el descendente pueden ser ineficientes, debido a la multiplicidad de maneras en las cuales múltiples árboles sintácticos para diferentes frases pueden ser combinados. Ambos pueden malgastar tiempo buscando irrelevantes porciones del espacio de búsqueda. El análisis descendente puede generar nodos intermedios que nunca podrán ser sujetados por palabras, y el análisis ascendente puede generar análisis parciales de las palabras que no pueden aparecer en un *S*.

Incluso si tuviésemos una función heurística perfecta que nos permitiera buscar sin ninguna digresión irrelevante, estos algoritmos podrían todavía ser ineficientes, ya que algunas sentencias tienen *exponencialmente muchos* árboles sintácticos. La siguiente subsección muestra qué hacer en estos casos.

Análisis sintáctico eficiente

Consideremos las siguientes dos sentencias[5]:

> *Have the students in section 2 of Computer Science 101 take the exam.*
> (Los estudiantes de la sección 2 de Ciencia de la Computación 101 han realizado el examen.)
> *Have the students in section 2 of Computer Science 101 taken the exam?*
> (¿Han realizado el examen los estudiantes de la sección 2 de Ciencia de la Computación 101?)

Incluso aunque comparten las 10 primeras palabras, estas sentencias tiene análisis sintácticos muy diferentes, ya que la primera es una orden y la segunda una pregunta. Un algoritmo de análisis sintáctico de izquierda a derecha tendría que suponer si la primera palabra es parte de una orden o de una pregunta y no podría decir si la suposición es correcta al menos hasta la undécima palabra, *take* o *taken*. Si el algoritmo supone equivocadamente, tendrá que deshacer todo el camino hasta la primera palabra. Esta clase de vuelta atrás es inevitable, pero si nuestro algoritmo de análisis sintáctico pretende ser eficiente, debe evitar reanalizar «the students in section 2 of Computer Science 101» como un *FN* cada vez que retroceda.



En esta sección, desarrollaremos un algoritmo de análisis sintáctico que evita esta fuente de ineficiencias. La idea básica es un ejemplo de **programación dinámica**: *cada vez que analizamos una subcadena, almacenamos el resultado de forma que no tenemos que reanalizarla posteriormente*. Por ejemplo, una vez que descubrimos que «the students in section 2 of Computer Science 101» es una *FN*, podemos registrar este resultado en una estructura de datos conocida como un **grafo**. Los algoritmos que hacen esto son llamados **analizadores sintácticos con grafos**. Debido a que estamos tratando con gramáticas independientes del contexto, cualquier frase que se encontrase en el contexto de una rama del espacio de búsqueda puede funcionar igualmente bien en cualquier otra rama del espacio de búsqueda.

GRAFO

El grafo para una sentencia de n palabras consta de $n + 1$ **nodos** y un número de **arcos** que conectan nodos. La Figura 22.6 muestra un grafo con seis nodos (círculos) y tres arcos (líneas). Por ejemplo, el arco etiquetado

NODOS

ARCOS

$$[0,5, S \rightarrow FN\ FV\ \bullet]$$

significa que una *FN* seguida por una *FV* se combinan para formar una *S* que cruza la cadena desde 0 hasta 5. El símbolo • en un arco separa lo que ha sido encontrado hasta el momento de lo que queda por encontrar[6]. Los arcos con • al final son llamados **arcos completos**. El arco

$$[0,2, S \rightarrow FN \bullet FV]$$

[5] *Nota del traductor:* mantendremos, en este caso y otros posteriores, las frases del original inglés para poder seguir el argumento del autor, en este caso de la coincidencia de las palabras, ya que en muchos casos hacen referencia a particularidades de la gramática inglesa.

[6] Debido al • los arcos son llamados a veces **reglas punteadas.**

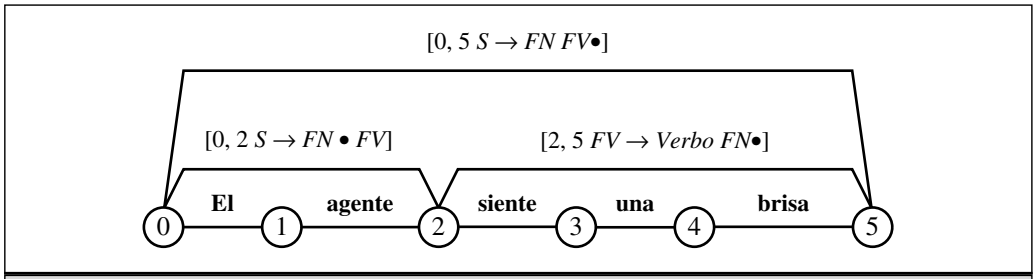


Figura 22.6 Parte del grafo para la sentencia «El agente siente una brisa». Se muestran los seis nodos, pero solamente tres de los arcos que pueden componer un análisis sintáctico completo.

dice que una *FN* cruza la cadena de 0 a 2 (las dos primeras palabras) y que si podemos encontrar a continuación una *FV*, entonces podemos formar una *S*. Los arcos como este con el punto antes del final son llamados arcos incompletos, y decimos que el arco está buscando una *FV*.

La Figura 22.7 muestra un algoritmo de análisis sintáctico con grafo. La idea principal es combinar lo mejor del análisis sintáctico ascendente y del descendente. El procedimiento PREDECIR descendente: transforma las entradas en el grafo que dicen qué símbolos son deseables y en qué localizaciones. ANALIZAR es el procedimiento ascendente que se inicia en las palabras, pero que utilizará una única palabra para extender una entrada de un grafo ya existente. De forma similar, EXTENDER construye componentes descendentes, pero solamente para extender una entrada de un grafo ya existente.

Usamos un truco para iniciar el algoritmo completo: añadimos el arco $[0, 0, S' \rightarrow \bullet\, S]$ al grafo, donde S es el símbolo inicial de la gramática, y S' es un nuevo símbolo que acabamos de idear. La llamada a AÑADIR-ARCO causa que el PREDECIR añada arcos por las reglas que puedan producir un S, esto es, $[S \rightarrow FN\ FV]$. Después miramos el primer componente de esa regla, *FN*, y añadimos reglas para cada manera de producir una *FN*. Finalmente, el predictor (pronosticador) añade, en forma descendente, todos los posibles arcos que pueden ser usados para crear la *S* final.

Cuando el predictor para S' finaliza, entramos en un bucle que llama a SCANNER para cada palabra de la sentencia. Si la palabra en la posición j es miembro de la categoría B que algún arco está buscando en j, entonces extendemos ese arco, anotando la palabra como una instancia de B. Observemos que cada llamada a ANALIZAR puede acabar llamando a PREDECIR y EXTENDER recursivamente, de ese modo intercalar procesamiento descendente y ascendente.

El otro componente ascendente, EXTENDER[7], toma un arco completo con lado izquierdo B y lo usa para extender cualquier regla incompleta del grafo que finaliza donde el arco completo empieza si la regla incompleta está buscando a B.

Las Figuras 22.8 y 22.9 muestran un grafo y una traza del algoritmo analizando la sentencia «Yo la percibo» («I feel it») (que es una respuesta a la pregunta «¿Sientes una

[7] Tradicionalmente nuestro procedimiento EXTENDER ha sido llamado COMPLETER. Este nombre es engañoso, ya que el procedimiento no completa arcos: toma un arco completo como entrada y extiende los arcos incompletos.

```
función ANALIZADOR-CON-GRAFO(palabras, gramática) devuelve grafo

    grafo ← array[0... LONGITUD(palabras)] de listas vacias
    AÑADIR-ARCO  S([0, 0, S′ → • S])
    para i desde 0 a LONGITUD(palabras) hacer
        ANALIZAR(i, palabras[i])
    devuelve grafo

procedimiento AÑADIR-ARCO(arco))
    /* Añadir un arco al grafo y ver si extiende o predice otro arco. */
    si arco no está en grafo[FIN(arco)] entonces
        añadir arco a el grafo[FIN(arco)]
        si arco no tiene nada después del punto entonces EXTENDER(arco)
        sino PREDECIR(arco)

procedimiento ANALIZAR(j, word)
    /* Para cada arco que espera una palabra de esta categoría, se extiende el arco. */
    para cada [i, j, A → α • B β] en grafo[ j] hacer
        si palabra es de la categoría B entonces
            AÑADIR-ARCO([i, j + 1, A → α B • β])

procedimiento PREDECIR([i, j, A → α • B β])
    /* Añadir al grafo todas las reglas de B que podrían ayudar a extender el arco */
    para cada (B → γ) en REESCRIBIR-PARA(B, gramática) hacer
        AÑADIR-ARCO([j, j, B → • γ])

procedimiento EXTENDER([j, k, B → γ •])
    /* Ver qué arcos se pueden extender mediante este arco */
    e_B ← el arco que es la entrada a este procedimiento
    para cada ([i, j, A → α • B′ β]) en grafo[j] hacer
        si B = B′ entonces
            AÑADIR-ARCO([i, j, A → α e_B • β])
```

Figura 22.7 El algoritmo de análisis sintáctico con grafo. S es el símbolo inicial y S' es un símbolo no terminal nuevo. ***grafo[j]*** es la lista de los arcos que finalizan en el nodo j. Las letras griegas equivalen a una cadena de cero o más símbolos.

brisa?»). Trece arcos (etiquetados a-m) son registrados en el grafo, incluyendo cinco arcos completos (mostrados sobre los nodos del grafo) y ocho incompletos (debajo de los nodos). Observar el ciclo de las acciones predecir, analizar y extender. Por ejemplo, predictor utiliza el hecho de que el arco (a) está buscando una *S* para autorizar la predicción de una *FN* (arco b) y después un *Pronombre* (arco c). Entonces analizar reconoce que hay un *Pronombre* en la posición derecha (arco d) y extender combina el arco incompleto b con el arco completo d para producir un nuevo arco, e.

El algoritmo de análisis sintáctico con grafo evita construir una extensa clase de arcos que podrían haber sido examinados por el procedimiento ascendente simple. Consideremos la sentencia «The ride the horse gave was wild» («La carrera que el caballo realizó fue desenfrenada»). Un análisis sintáctico ascendente podría etiquetar «ride the horse» como una *FV* y después descartar el árbol sintáctico cuando encontrara que no po-

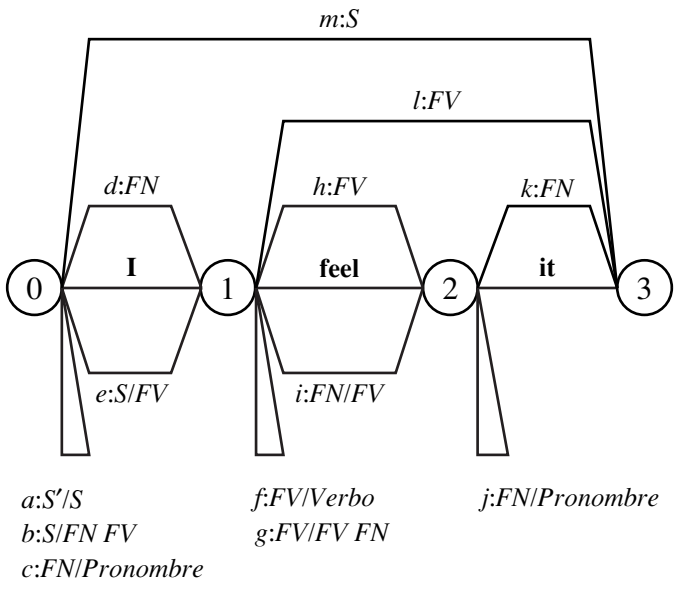


Figura 22.8 Grafo para el análisis sintáctico de «$_0$ Yo $_1$ lo $_2$ siento $_3$» («$_0$ I $_1$ feel $_2$ it $_3$»). La notación *m:S* significa que el arco *m* tiene una *S* en el lado izquierdo, mientras que la notación *f:FV/Verbo* significa que el arco *f* tienen una *FV* en el lado izquierdo, pero está buscando un *Verbo*. Hay cinco arcos completos sobre los nodos y ocho arcos incompletos debajo de ellos.

Arco	Procedimiento	Derivación
a	INICIAR	[0, 0, $S' \rightarrow \bullet\ S$]
b	PREDECIR(a)	[0, 0, $S \rightarrow \bullet\ FN\ FV$]
c	PREDECIR(b)	[0, 0, $FN \rightarrow \bullet\ Pronombre$]
d	ANALIZAR(c)	[0, 1, $FN \rightarrow Pronombre\ \bullet$]
e	EXTENDER(b,d)	[0, 1, $S \rightarrow FN \bullet FV$]
f	PREDECIR(e)	[1, 1, $FV \rightarrow \bullet\ Verbo$]
g	PREDECIR(e)	[1, 1, $FV \rightarrow \bullet\ FV\ FN$]
h	ANALIZAR(f)	[1, 2, $FV \rightarrow Verbo\ \bullet$]
i	EXTENDER(g,h)	[1, 2, $FV \rightarrow FV \bullet FN$]
j	PREDECIR(g)	[2, 2, $FN \rightarrow \bullet\ Pronombre$]
k	ANALIZAR(j)	[2, 3, $FN \rightarrow Pronombre\ \bullet$]
l	EXTENDER(i,k)	[1, 3, $FV \rightarrow FV\ FN\ \bullet$]
m	EXTENDER(e,l)	[0, 3, $S \rightarrow FN\ FV\ \bullet$]

Figura 22.9 Traza del análisis sintáctico de «$_0$ Yo $_1$ lo $_2$ siento $_3$» («$_0$ I $_1$ feel $_2$ it $_3$»). Para cada arco a-m, mostramos el procedimiento usado para derivar el arco desde otros arcos ya existentes en el grafo. Por brevedad omitimos algunos arcos.

día encajarlo en una *S* más amplia. Pero ε_0 no permite que una *FV* siga a «el», así que el algoritmo de análisis sintáctico con grafo nunca predecirá una *FV* en este punto y por tanto evitará perder tiempo construyendo la componente *FV* aquí. Los algoritmos que tra-

EXTREMO IZQUIERDO

bajan de izquierda a derecha y evitan construir estos componentes imposibles son llamados analizadores sintácticos de **extremo izquierdo**, debido a que montan el árbol sintáctico que empieza con el elemento inicial de la gramática y extienden la palabra más a la izquierda de la sentencia (el extremo izquierdo). Únicamente se añadirá un arco al grafo si puede servir para extender este árbol sintáctico. (*Véase* la Figura 22.10 como ejemplo).

El analizador sintáctico con grafo utiliza únicamente tiempo y espacio polinomial. Requiere espacio para almacenar arcos del orden $O(kn^2)$, donde n es el número de palabras de la sentencia y k es una constante que depende de la gramática. Cuando no puede construir más arcos, para, así sabemos que el algoritmo termina (incluso cuando hay reglas recursivas por la izquierda). De hecho, tarda de orden de $O(n^3)$ en el peor de los casos, lo cual es lo mejor que se puede conseguir para gramáticas independientes del contexto. El cuello de botella para ANALIZADOR-CON-GRAFO es EXTENDER, el cual debe intentar extender cada uno de los $O(n)$ arcos incompletos que finalizan en la posición j con cada uno de los $O(n)$ arcos completos que empiezan en j, para cada uno de los $n + 1$ valores diferentes de j. Multiplicándolos obtenemos $O(n^3)$. Esto nos da una especie de paradoja: ¿cómo puede un algoritmo de $O(n^3)$ devolver una respuesta que puede contener un número exponencial de árboles sintácticos? Consideremos un ejemplo: la sentencia

«Fall leaves fall and spring leaves spring»
(Las hojas otoñales caen y las hojas primaverales brotan)

es ambigua ya que cada palabra (excepto «and») puede ser tanto sustantivo como verbo, y «fall» y «spring» pueden ser también adjetivos. En total, la sentencia tiene cuatro análisis sintácticos[8]:

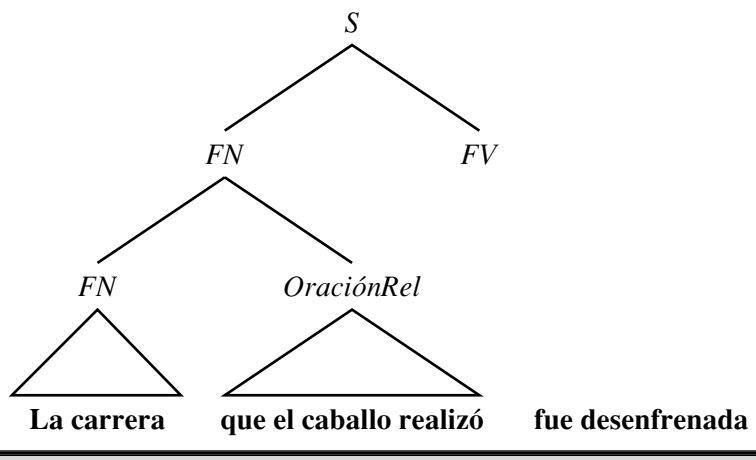


Figura 22.10 Un algoritmo de análisis sintáctico de extremo izquierdo evita predecir una *FV* empezando en «ride», pero no predice una *FV* empezando en «was», ya que la gramática espera que una *FV* siga a una *FN*. El triángulo sobre «the horse gave» significa que las palabras han pasado un análisis sintáctico con una *OraciónRel*, pero con componentes intermedios adicionales que no son mostrados.

[8] El análisis sintáctico [*S*: Fall [*FV*: leaves fall]] es equivalente a «Autumn abandons autumn» (El otoño abandona al otoño).

[*S*: [*S*: [*FN*: Fall leaves] fall] and [*S*: [*FN*: spring leaves] spring];
[*S*: [*S*: [*FN*: Fall leaves] fall] and [*S*: spring [*FV*: leaves spring]];
[*S*: [*S*: Fall [*FV*: leaves fall]] and [*S*: [*FN*: spring leaves] spring];
[*S*: [*S*: Fall [*FV*: leaves fall]] and [*S*: spring [*FV*: leaves spring]].

Si tuviésemos conjuntamente n subsentencias ambiguas, podríamos tener 2^n maneras de elegir análisis sintácticos para las subsentencias[9]. ¿Cómo debe el analizador sintáctico con grafo evitar un tiempo de procesamiento exponencial? Realmente hay dos respuestas. La primera es que el algoritmo CHART-PARSE en sí mismo es realmente un reconocedor, no un analizador. Si hay un arco completo de la forma $[0, n, S \rightarrow \alpha \bullet]$ en el grafo, entonces hemos reconocido una *S*. La recuperación del árbol sintáctico desde este arco no es considerado parte del trabajo de CHART-PARSE, pero puede hacerlo. Observar, en la última línea de EXTENDER, que montamos α como una lista de arcos, e_B, no como una lista de nombres de categorías. Así, para convertir un arco en un árbol sintáctico, simplemente miramos recursivamente a los arcos componentes, convirtiendo cada $[i, j, X \rightarrow \alpha \bullet]$ en el árbol $[X : \alpha]$. Esto es sencillo, pero nos da sólo *un* árbol sintáctico.

La segunda respuesta es que si quiere todos los posibles análisis sintácticos, tendrá que escarbar profundamente en el grafo. Mientras estamos convirtiendo el arco $[i, j, X \rightarrow \alpha \bullet]$ en el árbol $[X : \alpha]$, también miraremos para ver si hay otros arcos de la forma $[i, j, X \rightarrow \beta \bullet]$. Si los hay, estos arcos generarán análisis sintácticos adicionales. Ahora tenemos una selección de lo que podemos hacer con ellos. Podemos enumerar todas las posibilidades, y esto significa que la paradoja puede ser resuelta y podríamos necesitar una cantidad exponencial de tiempo para listar los análisis sintácticos. O podemos prolongar el misterio un poco más y representar los análisis sintácticos con una estructura llamada **bosque empaquetado**, la cual se parece a esto:

BOSQUE EMPAQUETADO

$$[S\text{: }[S\text{: }\begin{Bmatrix}[FN\text{: Fall leaves}][FV\text{: fall}]\\ [FN\text{: Fall}][FV\text{: leaves fall}]\end{Bmatrix}] \text{ and } [S\text{: }\begin{Bmatrix}[FN\text{: spring leaves}][FV\text{: spring}]\\ [FN\text{: spring}][FV\text{: leaves spring}]\end{Bmatrix}]]$$

La idea es que cada nodo pueda ser o bien un nodo de un árbol sintáctico regular o un conjunto de nodos árbol. Esto nos permitirá devolver una representación de un número exponencial de análisis sintácticos en una cantidad de tiempo y espacio polinomial. Desde luego, cuando $n = 2$, no hay muchas diferencias entre 2^n y $2n$, pero para n grande, esta representación proporciona un ahorro considerable. Desafortunadamente, esta simple aproximación de bosque empaquetado no manejará todas las $O(n!)$ ambigüedades de cómo se asocian las conjunciones. Maxwell y Kaplan (1995) muestran cómo una representación más compleja, basada en los principios de los sistemas de mantenimiento de la coherencia puede empaquetar los árboles aún más ajustadamente.

[9] También podría ser una ambigüedad $O(n!)$ en la manera en que los componentes se juntan entre ellos, por ejemplo, (*X* y (*Y* y *Z*)) frente ((*X* y *Y*) y *Z*). Pero esto es otra historia, una que está bastante bien explicada por Church y Patil (1982).

22.4 Gramáticas aumentadas

En el Apartado 22.2 vimos que una gramática simple para ε_0 genera «Yo huelo un hedor» («I smell a stench») y otras frases de inglés. Desafortunadamente, también genera muchas frases incorrectas como «Mi huelo un hedor» («Me smell a stench»). Para evitar este problema, nuestra gramática debería conocer que «me» no es un *FN* válida cuando es el sujeto de una frase. Los lingüistas dicen que el pronombre «Yo» se emplea en el caso subjetivo, y «mi» en el caso objetivo[10]. Cuando tenemos los casos en cuenta comprendemos que ε_0 no es una gramática independiente del contexto: no es verdad que cualquier *FN* sea igual a cualquier otra sin hacer caso del contexto. Podemos arreglar el problema introduciendo nuevas categorías tales como FN_S y FN_O, para soportar frases sustantivas en los casos subjetivo y objetivo, respectivamente. También necesitamos separar la categoría *Pronombre* en las dos categorías $Pronombre_S$ (que incluye «yo») y $Pronombre_O$ (que incluye «mi»). La parte superior de la Figura 22.11 muestra la gramática BNF completa para el caso convenido; podemos llamar al lenguaje resultante ε_1. Hay que reseñar que todas las reglas de las *FN* deben ser duplicadas, una vez para las FN_S y otra para las FN_O.

CONCORDANCIA

Desafortunadamente, ε_1 todavía sobregenera. El español y otros muchos lenguajes requieren **concordancia** entre el sujeto y el verbo principal de una frase. Por ejemplo, si «yo» es el sujeto, entonces «yo huelo» es gramaticalmente correcto, pero «yo huele» no lo es. Si «ello» es el sujeto, tenemos el caso inverso. En inglés, las distinciones por convenio son mínimas: la mayoría de los verbos tienen una forma para los sujetos singulares en tercera persona (él, ella o ello), y una segunda forma para todas las otras com-

$$
\begin{aligned}
S &\rightarrow FN_S\ FV \mid \ldots \\
FN_S &\rightarrow Pronombre_S \mid Nombre \mid Sustantivo \mid \ldots \\
FN_O &\rightarrow Pronombre_O \mid Nombre \mid Sustantivo \mid \ldots \\
FV &\rightarrow FV\ FN_O \mid \ldots \\
FP &\rightarrow Preposición\ FN_O \\
Pronombre_S &\rightarrow \textbf{Yo} \mid \textbf{tu} \mid \textbf{el} \mid \textbf{ella} \mid \textbf{ello} \mid \ldots \\
Pronombre_O &\rightarrow \textbf{mi} \mid \textbf{tu} \mid \textbf{su} \mid \ldots
\end{aligned}
$$

$$
\begin{aligned}
S &\rightarrow FN(Subjetivo)\ FV \mid \ldots \\
FN(caso) &\rightarrow Pronombre(caso) \mid Nombre \mid Sustantivo \mid \ldots \\
FV &\rightarrow FV\ FN(Objetivo) \mid \ldots \\
FP &\rightarrow Preposición\ FN(Objetivo) \\
Pronombre(Subjetivo) &\rightarrow \textbf{Yo} \mid \textbf{tu} \mid \textbf{el} \mid \textbf{ella} \mid \textbf{ello} \mid \ldots \\
Pronombre(Objetivo) &\rightarrow \textbf{mi} \mid \textbf{tu} \mid \textbf{su} \mid \ldots
\end{aligned}
$$

Figura 22.11 Arriba: una gramática en BNF para el lenguaje ε_1, la cual maneja casos objetivos y subjetivos en frases sustantivas y de esta forma no sobregenera demasiado incorrectamente. Se han omitido las porciones que son idénticas a ε_0. Abajo: una gramática de cláusulas definidas (DCG) de ε_1.

[10] El caso subjetivo también es a veces llamado el caso nominativo y el caso objetivo es llamado el caso acusativo. Muchos lenguajes también tienen un caso dativo para palabras en la posición del objeto indirecto.

binaciones de persona y número. Hay una excepción: «I am / you are / he is» tiene tres formas. Si multiplicamos estas tres distinciones por las dos distinciones FN_S y FN_O, acabamos con seis formas de *FN*. Como descubriremos más distinciones, acabaremos empleando números exponenciales.

AUMENTAR

La alternativa es **aumentar** las reglas de la gramática en vez de introducir nuevas reglas. Daremos primero un ejemplo de cómo queremos representar una regla aumentada. Las reglas aumentadas permiten *parámetros* para las categorías no terminales. La Figura 22.11 muestra cómo describir ε_1 empleando reglas aumentadas. Las categorías *FN* y *Pronombre* tienen un parámetro indicando su caso. (Los sustantivos no tienen caso en inglés, aunque lo tienen en muchos otros lenguajes.) En la regla para *S*, la *FN* debe estar en caso subjetivo, mientras que en las reglas para *FV* y *FP*, la *FN* debe estar en caso objetivo. La regla para *FN* toma una variable, *caso*, como su argumento. La intención es que la *FN* pueda tener cualquier caso, pero si la *FN* es rescrita como *Pronombre*, entonces ésta tenga el mismo caso. Este uso de una variable (evitando una decisión donde la distinción no es importante) es lo que mantiene el tamaño del conjunto de reglas evitando un crecimiento exponencial con el número de características.

GRAMÁTICA DE CLÁUSULAS DEFINIDAS

Este formalismo para aumentar es llamado **gramática de cláusulas definidas** o DCG (*definite clause grammar*), debido a que cada regla gramatical puede ser interpretada como una cláusula definida en la lógica de Horn[11]. Primero mostraremos cómo una regla normal, no aumentada, puede ser interpretada como una cláusula definida. Consideremos cada símbolo de categoría como un predicado sobre cadenas, de forma que *FN*(s) es verdad si la cadena *s* forma una *FN*. La regla CFG

$$S \rightarrow FN\ FV$$

es la forma reducida de la cláusula definida

$$FN(s_1) \wedge FV(s_2) \Rightarrow S(s_1 + s_2)$$

Aquí $s_1 + s_2$ denota la concatenación de dos cadenas, por tanto esta regla dice que si la cadena s_1 es una *FN* y la cadena s_2 es una *FV*, entonces su concatenación es una *S*, que es exactamente como estamos interpretando la regla CFG. Es importante hacer notar que dichas *DCG's nos permiten hablar del análisis sintáctico como inferencia lógica.* Esto hace posible razonar sobre lenguajes y cadenas de formas muy diversas. Por ejemplo, significa que podemos hacer un análisis sintáctico ascendente empleando encadenamiento hacia adelante o análisis sintáctico descendente empleando encadenamiento hacia atrás. Veremos que también esto significa que podemos usar la misma gramática tanto para el análisis sintáctico como para la generación.



El beneficio real del enfoque DCG es que podemos *aumentar* la categoría de símbolos con argumentos adicionales aparte de los argumentos de cadena. Por ejemplo, la regla

$$FN(caso) \rightarrow Pronombre(caso)$$

es la abreviatura de

$$Pronombre(caso, s_1) \Rightarrow FN(caso, s_1)$$

[11] Recordar que una cláusula definida, cuando se escribe como una implicación, tiene exactamente un átomo en su consecuente, y una conjunción de cero o más átomos en su antecedente. Dos ejemplos son $A \wedge B \Rightarrow C$ y sólo *C*.